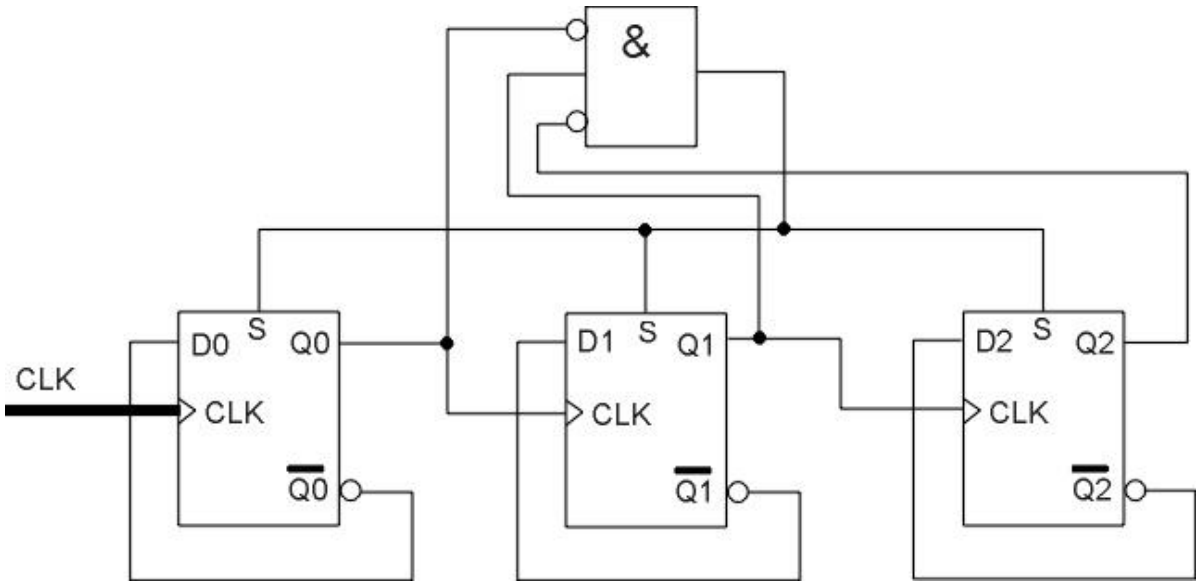
Rafbók



Tölvur og nettækni
TNT3
Eiríkur Guðmundsson
Kennsluhefti

Tölvur og nettækni TNT 3

Þetta hefti er án endurgjalds á rafbókinni www.rafbok.is
Allir rafiðnaðarmenn og rafiðnaðarnemar geta fengið aðgang án endurgjalds að rafbókinni.

Heimilt er að afrita textann til fræðslu í skólum sem reknir eru fyrir opinbert fé án leyfis höfundar eða Rafmenntar, fræðsluseturs rafiðnaðarins. Hvers konar sala á textanum í heild eða að hluta til er óheimil nema að fengnu leyfi höfundar og Rafmenntar.

Vinsamlegast sendið leiðréttingar og athugasemdir til höfundar eða til
Báru Laxdal Halldórsdóttur á netfangið bara@rafmennt.is

Höfundur er Eiríkur Guðmundsson
Umbrot: Bára Laxdal Halldórsdóttir

Efnisyfirlit

1. TTL og CMOS rásir og tengingar við rofa og aflstýringar 4
1.1 TTL rásir 4
1.2 Inngangsspenna 5
1.3 Inngangsstraumur 6
1.4 Útgangsspenna 6
1.5 Útgangsstraumur 6
1.6 FAN OUT 7
1.7 Seinkun (Propagation delay) 7
1.8 CMOS rásir 8
1.9 Afskoppunarrásir (Debouncing) 11
1.10 Samtenging hliða ef fleiri innganga er þörf (fjölgun innganga) 16
1.11 Verkefni 1 17
1.12 Ónotaðir inngangar (fækkun innganga) 17
1.13 Verkefni 2 18
1.14 Útgangsmögnun 18
1.15 Transistorar 19
1.16 ULN2003 20
1.17 Ljóshlið (Optocoupler) 21
2. Leit að bilunum í rökrásum með mælitækjum og með hjálp hermiforrita 22
2.1 Hermiforrit 22
2.2 Sveiflusjár 22
2.3 Próbur 24
3. Lásar og vippur 25
3.1 Lásar 25
3.2 Sanntöflur 26
3.3 NOR og NAND lás í Electronic Workbench 26
3.4 RS vippur 27
3.5 T vippur 29
3.6 D vippur 30
3.7 JK vippur 31
3.8 Einföldun á sannleikstöflu JK vippu 32
3.9 Master Slave vippur 33
3.10 Breyting úr einni vippugerð í aðra 34
4. Ósamfasa og samfasa teljarar 35
4.1 Ósamfasa teljarar 35
4.2 Hönnun ósamfasa teljara 36
4.3 Hönnun samfasa teljara 39
Dæmi 1: 39
4.4 Fleiri teljarar 43
4.5 Greining samfasa teljara 46

Dæmi 2: 47
Verkefni 48
5. Notkun teljara 49
5.1 74192 BCD teljari 49
5.2 MOD6 teljari búinn til úr 74192 BCD teljara 50
5.3 7447 og 7448 sjö ráka afkóðunarrás 52
5.4 Seven segment display búið til úr LED díóðum 56
5.5 Darlington ULN2003A 57
Verkefni. Teljari 0 - 99 58
5.6 Sérstakir teljarar 60
5.6.1 Johnson teljari 60
5.6.2 Overbeck teljari 61
5.6.3 Klukka 62
6. Gistur (register) 64
6.1 Samhliða inn / Samhliða út gistur 65
6.2 Samsíða gistur með þristöðu útgangi (Tristate logic , buffer) 66
6.3 Tvístefnd gistur (Bidirectional registers) 69
6.4 Rað inn / Rað út gistur 73
6.5 Parity 77
7. Minni 80
7.1 Högun minnis 82
7.2 SRAM (Static Random Access Memory) 85
7.3 ROM minni (Read Only Memory) 88
7.4 DRAM (Dynamic random access memory) 91
8. Forritun stýriörgjörva 98

1. TTL og CMOS rásir og tengingar við rofa og aflstýringar

1.1 TTL rásir

TTL rásir (Transistor Transistor Logic) eru rökrásir þar sem nafnið byrjar á 54 eða 74. 74 rásirnar eru miklu algengari og við munum því einblína meira á þær. Þær heita TTL vegna þess að bæði rökræni hluti þeirra (AND, OR og NOT) er búinn til með transistorum og einnig útgangsstig (mögnunarstig) þeirra. 54xxx rásirnar virka eins og 74xxx rásirnar en eru „Military standard“ og vinna því betur við erfiðar aðstæður.
Skoðum fyrst tæknilegar upplýsingar um 74xxx (þar sem xxx er tala, tveggja eða þriggja stafa) rásirnar og veljum eina, 7408, sem er með fjórum, tveggja innganga AND hliðum. Aðrar rásir í 74 seríunni hafa mjög svipuð gildi. Þessi data blöð má „gúggla” t.d. með textanum 7408 data sheet download.

7408 QUAD DUAL INPUT AND GATE

Absolute Maximum Ratings (Note 1)

		DM54 and 54	-55°C to +125°C
		DM74	0°C to +70°C
Supply Voltage	7V	Storage Temperature Range	-65°C to +150°C
Input Voltage	5.5V		
Operating Free Air Temperature Range			

Recommended Operating Conditions

Symbol	Parameter	DM5408			DM7408			Units
		Min	Nom	Max	Min	Nom	Max	
V_{CC}	Supply Voltage	4.5	5	5.5	4.75	5	5.25	V
V_{IH}	High Level Input Voltage	2			2			V
V_{IL}	Low Level Input Voltage			0.8			0.8	V
I_{OH}	High Level Output Current			-0.8			-0.8	mA
I_{OL}	Low Level Output Current			16			15	mA
T_A	Free Air Operating Temperature	-55		125	0		70	°C

Note 1: The “Absolute Maximum Ratings” are those values beyond which the safety of the device cannot be guaranteed. The device should not be operated at these limits. The parametric values defined in the “Electrical Characteristics” table are not guaranteed at the absolute maximum ratings. The “Recommended Operating Conditions” table will define the conditions for actual device operation.

Supply voltage (V_{CC}) er fæðispenna rásarinnar. Fyrir 74xxx rásirnar á þessi spenna að vera á bilinu 4,75V til 5,25V helst sem næst 5V. Því er 7805 spennureglunarrásin mikið notuð með 74xxx rásunum. Rásin virkar ekki nema fá fæðispennu.
Inngangsspenna. Samkvæmt DATA blaðinu er V_{IH} (High level input voltage) = $2V_{min}$. Þetta þýðir að ef rásin á að vinna rétt verður inngangsspennan í hástöðu (lógískur 1) að vera 2V eða hærri.

V_{IL} (Low level input voltage) = 0,8V_{max}. Þetta þýðir að ef rásin á að vinna rétt verður inngangsspennan í lágstöðu (lógískt 0) að vera 0,8V eða lægri.
Sjáum að þarna á milli er gap frá 0,8V til 2V, einhverskonar grátt svæði.

1.2 Inngangsspenna



Við hönnun rökrásastýringa þarf að gæta að því að inngangsspenna lendi ekki inni á gráa svæðinu. Hún þarf að vera lægri en 0,8V í lágstöðu og hærri en 2V í hástöðu. Lendi hún á gráa svæðinu ábyrgjast framleiðendur ekki rétta virkni. T.d. myndi 1,5V rafhlaða ekki vera góð sem inngangsspenna inn á rökrás.

Electrical Characteristics
over recommended operating free air temperature range (unless otherwise noted)

Symbol	Parameter	Conditions		Min	Typ (Note 2)	Max	Units
V_I	Input Clamp Voltage	V_{CC}=Min, I_I=-12mA				-1.5	V
V_{OH}	High Level Output Voltage	V_{CC}=Min, I_{OH}=Max, V_{IL}=Max		2.4	3.4		V
V_{OL}	Low Level Output Voltage	V_{CC}=Min, I_{OL}=Max, V_{IH}=Min			0.2	0.4	V
I_I	Input Current @Max Input Voltage	V_{CC}=Max, V_I=5.5V				1	mA
I_{IH}	High Level Input Current	V_{CC}=Max, V_I=2.4V				40	µA
I_{IL}	Low Level Input Current	V_{CC}=Max, V_I=0.4V				-1.6	mA
I_{OS}	Short Circuit Output Current	V_{CC}=Max (Note 3)	DM54	-20		-55	mA
			DM74	-18		-55	
I_{CCH}	Supply Current with Output High	V_{CC}=Max			11	21	mA
I_{CCL}	Supply Current with Output Low	V_{CC}=Max			20	33	mA

Switching Characteristics
at V_{CC}=5V and T_A=25°C (See Section 1 for Test Waveforms and Output Load)

Symbol	Parameter	Conditions	Min	Max	Units
t_{PLH}	Propagation Delay Time Low to High Level Output	C_L=15 pF RL=400Ω		27	ns
t_{PLH}	Propagation Delay Time High to Low Level Output			19	ns

Note2: All typical are at V_{CC}=5V, T_A=25°C
Note 3: Not more than one output should be shorted at a time.

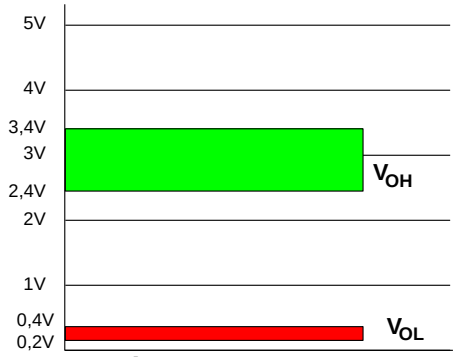
1.3 Inngangsstraumur

Inngangsstraumur í hástöðu inngangs kallast I_{IH} (High Level Input Current) og er gefinn 40µA miðað við að V_{CC} sé 5,25V og inngangsspennan sé 2,4V.

Inngangsstraumur í lágstöðu inngangs kallast I_{IL} (Low Level Input Current) og er gefinn –1,6mA miðað við að V_{CC} sé 5,25V og inngangsspennan sé 0,4V.

1.4 Útgangsspenna

Útgangsspennan í **hástöðu** er kölluð V_{OH} (Voltage Output High). Hún er gefin lágmark 2,4V en oftast (typical) er hún 3,4V. Framleiðandinn ábyrgist semsagt að við eðlilegar aðstæður sé útgangsspennan aldrei lægri en 2,4V og venjulega 3,4V.
Útgangsspennan í **lágstöðu** er kölluð V_{OL} (Voltage Output Low). Hún er gefin hámark 0,4V en er venjulega (typical) 0,2V. Framleiðandinn ábyrgist að við eðlilegar aðstæður sé þessi spenna oftast 0,2V en geti farið upp í 0,4V. Sjáum að þessar stærðir lenda langt frá „gráa svæðinu" og eru því vel nothæfar sem inngangsspenna inn á rásir sem á eftir koma. Við þurfum því ekki að hafa áhyggjur af útgangsspennunni ef við pössum upp á inngangsspennuna.



Útgangsspenna

1.5 Útgangsstraumur

Útgangsstraumurinn í **hástöðu** útgangs (lógískur 1) kallast I_{OH} (High Level Output Current) og er gefinn –0,8mA.
Útgangsstraumurinn í **lágstöðu** útgangs (lógískt 0) kallast I_{OL} (Low Level Output Current) og er gefinn 16mA.

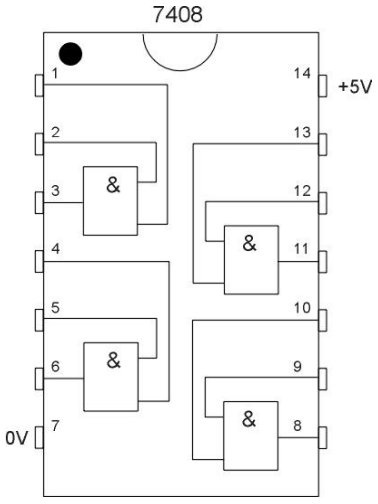
1.6 FAN OUT

Fan out er stærð sem segir til um hve marga innganga einn útgangur á rökrás getur drifið. Fan out er skilgreint sem hlutfallið á milli útstraums og innstraums bæði í há og lágstöðu.

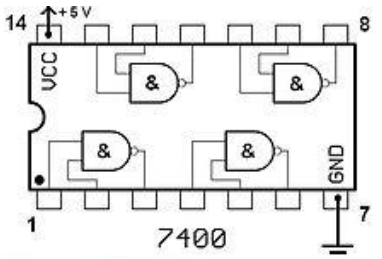
Fan out = $\frac{I_{OH}}{I_{IH}} = \frac{0{,}8mA}{40\mu A} = 20$ eða $\frac{I_{OL}}{I_{IL}} = \frac{16mA}{1{,}6mA} = 10$. Veljum lægra gildið sem er 10 og er fast gildi fyrir 74xxx rásirnar. Einn útgangur á 74xxx rás á því að ráða við að drífa 10 innganga sem koma á eftir honum.

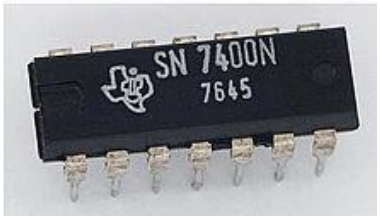
1.7 Seinkun (Propagation delay)

Seinkun í 74xxx rásunum er mjög lítil eða frá 19ns (max) þegar útgangur breytist frá 1 í 0 upp í 27ns (max) þegar útgangur breytist frá 0 í 1. Þetta eru tímar sem við þurfum yfirleitt ekki að hafa áhyggjur af og yfirleitt eru þeir miklu lægri en þetta. Nanósekúndur eru milljörðustu hlutar úr sekúndu og tíðnir eins og t.d. 10MHz ættu ekki að vera vandamál í sambandi við seinkunn.



Pinnamynd (pin out) af 7408 rásinni (4 x AND). Pinni 1 er yfirleitt merktur með punkti á rásinni. Ef í vafa þá er hann efst til vinstri ef hakið á rásinni snýr upp.





7400 rásin (4 x NAND) með
Amerískum táknum fyrir NAND hlið.

Nánari merkingar 74 seríunnar. (Wikipedia)

74	Upphaflega 74 rásin
74L	Low Power rás 1mW, 10ns
74H	High Speed rás 22mW, 6ns
74S	High Speed. Schottky díóða í inngangi 20mW, 3ns
74LS	Low Speed, Schottky díóða í útgangi
74AS	Advanced Schottky 8mW, 1,7ns
74ALS	Advanced Low Power Schottky 1,2mW, 4ns
74F	Fast Low Power 1,2mW, 4ns

1.8 CMOS rásir

CMOS rásir þekkjast á því að nafn þeirra byrjar á 40 og venjulega er talað um CD4000 seríuna. CMOS rásirnar taka miklu minna afl en 74 serían og hafa stærra FAN OUT. Hver CMOS útgangur getur þannig drifið fleiri innganga á eftir sér en 74 rásirnar. Þær eru hinsvegar hægari en 74 rásirnar. Annar kostur við CMOS rásirnar er að þær hafa meiri vídd í fæðispennu eða V_{CC} frá 3V til 15V. Þær eru ekki eins næmar fyrir static (ESD) eins og 74 rásirnar. CD4000 rásirnar eru þó minna notaðar í dag vegna framþróunar í 74 seríunni.

Nánari merkingar CD4000 seríunnar (Wikipedia)

40C	CMOS 4–15 V operation similar to buffered 4000 (4000B) series.
40HC	High-speed CMOS, similar performance to LS, 12 ns. 2.0–6.0 V.
40HCT	High speed, compatible logic levels to bipolar parts.
40AC	Advanced CMOS, performance generally between S and F.
40ACQ	Advanced CMOS with Quiet outputs.
40AHC	Advanced high-speed CMOS, three times as fast as HC.
40ALVC	Low-voltage – 1.8–3.3 V, time Propagation Delay (TPD) < 3 ns at 3.3 V.
40ALVT	Low-voltage – 2.5–3.3 V, 5 V tolerant inputs, high current ≤ 64 mA, TPD < 3 ns at 2.5 V.
40AUC	Low-voltage – 0.8–2.5 V, TPD < 2.5 ns at 1.8 V.
40AUP	Low-voltage – 0.8–3.6 V (3.3 V typically), TPD 15.6/8.2/4.3 ns at 1.2/1.8/3.3V, partial power-down specified (IOFF), inputs protected.
40AVC	Low-voltage – 1.8–3.3 V, TPD < 3.2 ns at 1.8 V, bus hold, IOFF.
40FC	Fast CMOS, performance similar to F.
40LCX	CMOS with 3 V supply and 5 V tolerant inputs
40LV	Low-voltage CMOS – 2.0–5.5 V supply and 5 V tolerant inputs.
40LVC	Low voltage – 1.65–3.3 V and 5 V tolerant inputs, TPD < 5.5 ns at 3.3 V, TPD < 9 ns at 2.5 V.
40LV-A	2.5–5 V, 5 V tolerant inputs, TPD < 10 ns at 3.3 V, bus hold, IOFF, low noise.
40LVT	Low-voltage – 3.3 V supply, 5 V tolerant inputs, high output current < 64 mA, TPD < 3.5 ns at 3.3 V, IOFF, low noise.
40LVQ	Low-voltage – 3.3 V.
40LVX	Low-voltage – 3.3 V with 5 V tolerant inputs.
40VHC	Very-high-speed CMOS

DYNAMIC ELECTRICAL CHARACTERISTICS at T_A = 25°C, Input t_r, t_f = 20 ns, C_L = 50 pF, R_L = 200 kΩ

CHARACTERISTIC	TEST CONDITIONS		LIMITS			UNITS
		V_{DD} (V)	Min.	Typ.	Max.	
INPUT-PULSE OPERATION						
Propagation Delay Time Φ to Q_1 Out; t_{PHL}, t_{PHL}		5	-	180	360	ns
		10	-	80	160	
		15	-	65	130	
Qn to Qn+1; t_{PHL}, t_{PHL}		5	-	100	330	ns
		10	-	40	80	
		15	-	30	60	
Transition Time, t_{THL}, t_{THL}		5	-	100	200	ns
		10	-	50	100	
		15	-	40	80	
Minimum Input-Pulse Width, t_W		5	-	70	140	ns
		10	-	30	60	
		15	-	20	40	
Input-Pulse Rise or Fall Time, $t_{r\Phi}$, $t_{f\Phi}$		5	Unlimited			µs
		10				
		15				
Maximum Input-Pulse Frequency, f_Φ		5	3.5	7	-	MHz
		10	8	16	-	
		15	12	24	-	
Input Capacitance, C_I	Any Input		-	5	7.5	pF
Reset Operation						
Propagation Delay Time, t_{PHL}		5	-	140	280	ns
		10	-	60	120	
		15	-	50	100	
Minimum Reset Pulse Width, t_W		5	-	100	200	ns
		10	-	40	80	
		15	-	30	60	
Reset Removal Time, t_{REM}		5	-	175	350	ns
		10	-	75	150	
		15	-	50	100	

Data blað fyrir CD4000 rás. Gefnar eru upp stærðir miðað við V_{CC} = 5V, 10V og 15V.

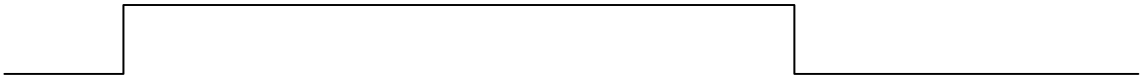
CD4000 rásirnar hafa því breytilega getu eftir því hver fæðispennan V_{CC} er.

1.9 Afskoppunarrásir (Debouncing)

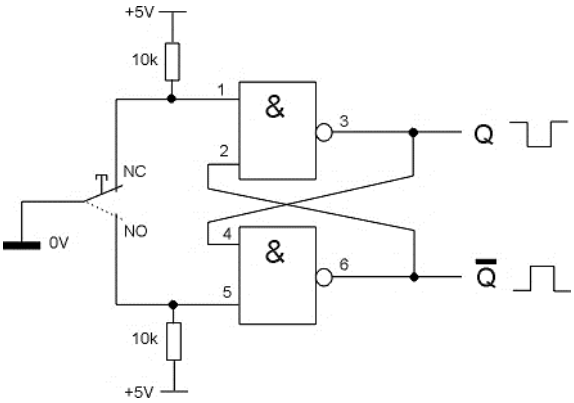
Almennt séð er ekki hægt að nota rofa beint á inngang rökrása. T.d. ef búa á til teljara þar sem á að hækka eða lækka gildið með rofa þá mun hann ekki senda einn púls heldur marga inn á teljarann. Ástæðan fyrir þessu er svokallað snertuskopp (contact bouncing). Venjuleg snerta nær ekki strax sambandi heldur skoppar hún vegna gormvirkni rofans. Ef við skoðum merki frá rofa í sveiflusjá þá lítur það einhvern veginn svona út.



Merkið sem við viljum hinsvegar fá lítur svona út:



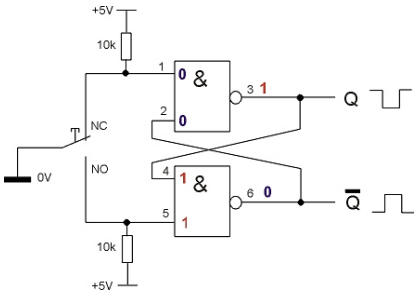
Þessu má redda með svokallaðri afskoppunarrás (Debouncer). Algeng leið til að gera þetta er að nota svokallaðan NAND lás. Hann er búinn til úr tveimur tveggja innganga NAND hliðum. Viðnámin á myndinni eru til að koma í veg fyrir að inngangarnir verði fljótandi (sambandslausir) þegar snertan er ekki tengd við þá. Sem dæmi þá heldur neðra 10k viðnámið fastri +5V spennu á inngangi 5 miðað við þessa stöðu snertunnar. Ef snertan skiptir um stöðu þá mun hún á einhverjum tímapunkti lenda í millibili á milli inngangs 1 og inngangs 5. Þá munu viðnámin sjá til þess að inngangar 1 og 5 hafi +5V spennu en verði ekki fljótandi. Um leið og snertan tengist við inngang 5 þá verður spennan á þeim inngangi 0V en inngangur 1 verður fastur í +5V.



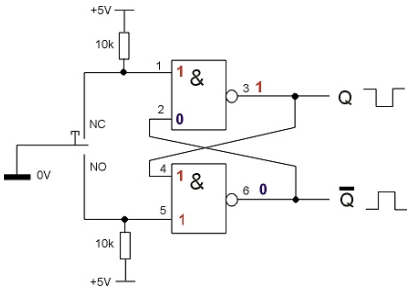
Ef við skoðum sanntöflu tveggja innganga NAND hliðs þá skýrist virknin betur.

A	B	$Y = \overline{A \cdot B}$
0	**0**	**1**
0	**1**	**1**
1	**0**	**1**
1	**1**	**0**

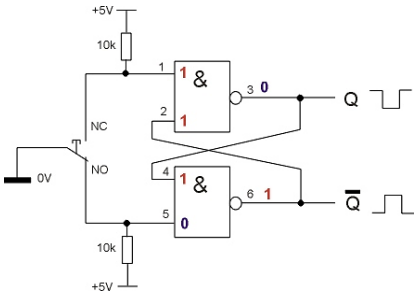
Ef annarhvor eða báðir inngangar NAND hliðs eru **0** þá er útgangurinn **1**. Útgangurinn er bara **0** þegar báðir inngangar eru **1**. Skoðum einn púls á rofanum þ.e. þrýst er á hann og honum síðan sleppt.



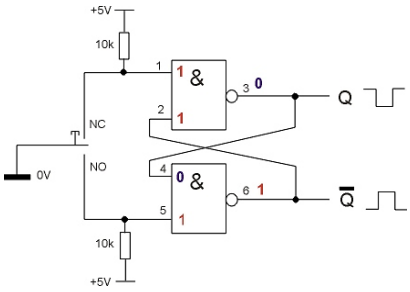
1. Upphafsstaða.



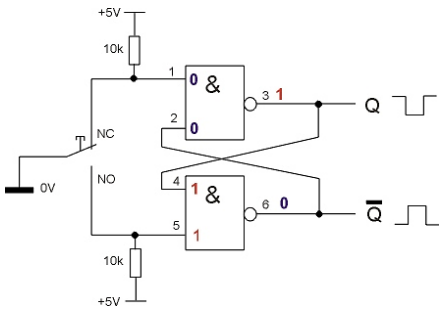
2. Miðstaða snertu á leið niður.



3. Neðsta staða.

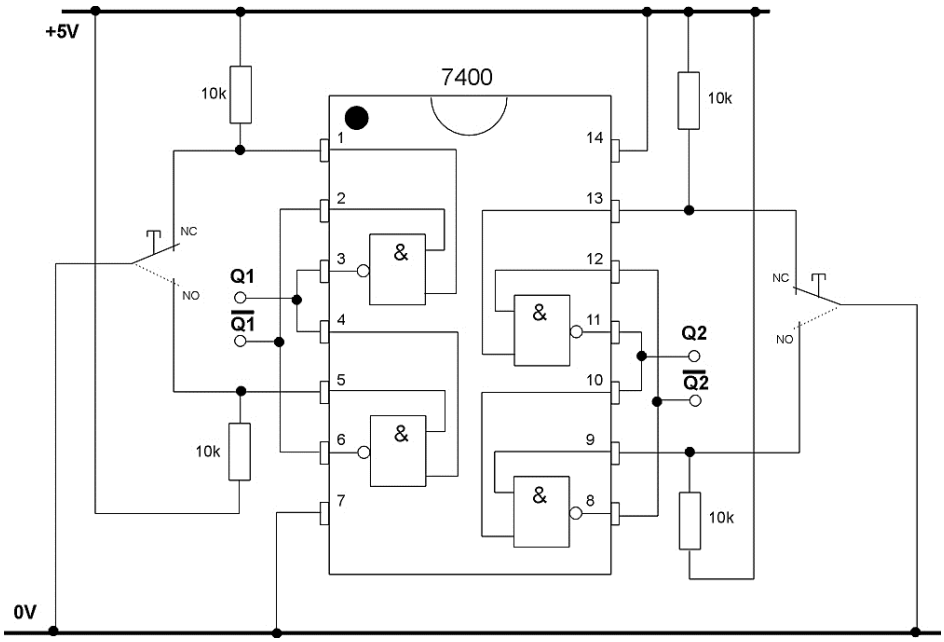


4. Miðstaða snertu á leið upp.

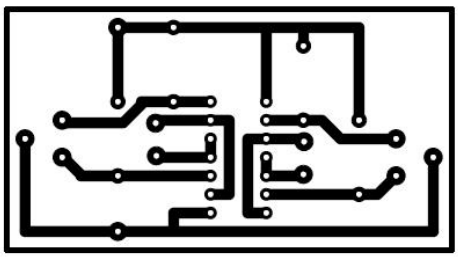


5. Upphafsstaða.

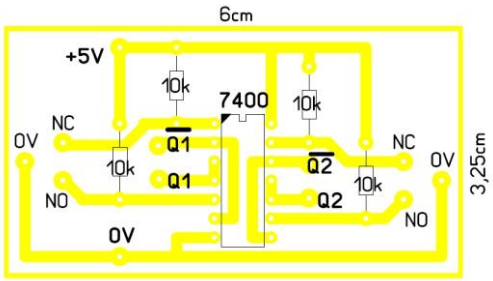
Q = 11001 frá mynd 1 til myndar 5
$\bar{Q}$= 00110 frá mynd 1 til myndar 5
Sjáum hvernig Q útgangarnir mynda hreinan púls.



Tvöfaldur NAND lás með 7400 QUAD DUAL INPUT NAND GATE rásinni.

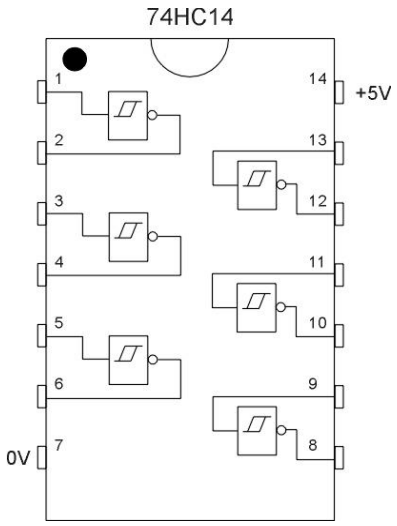


Prent fyrir tvöfaldan NAND lás.

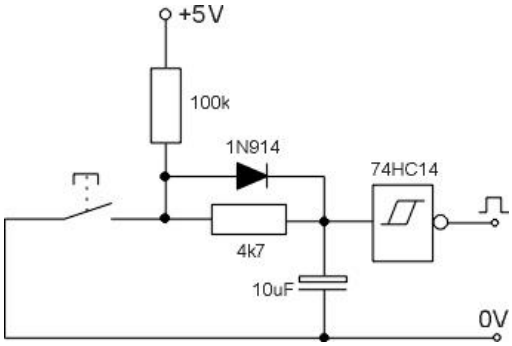


Íhlutamynd fyrir tvöfaldan NAND lás.

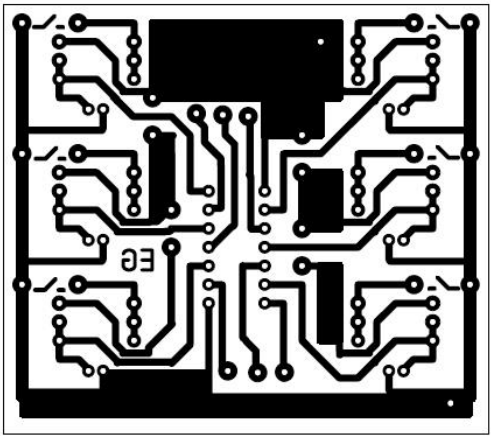
Önnur leið til að búa til hreina púlsa er að nota svokallaðan Schmitt Trigger. Rásin 74HC14 inniheldur sex Schmitt trigger rásir með viðsnúnum (Inverted) útgangi. Hana má því nota til að hreinsa púlsa frá sex snertum. Kosturinn við að nota Schmitt trigger er að við þurfum ekki víxlandi snertu (NC og NO) eins og í NAND lásnum.



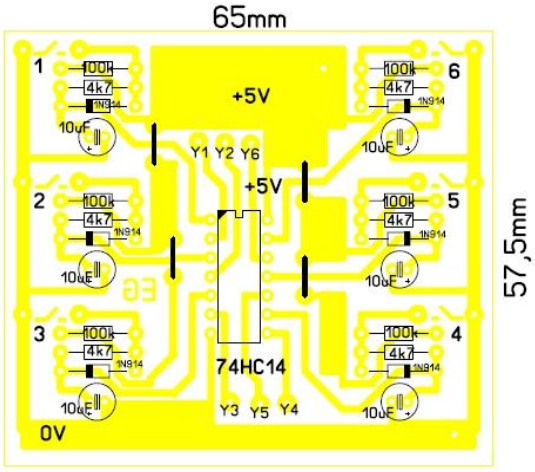
74HC14 HEX SCHMITT TRIGGER INVERTERS.



Afskoppunarrás sem notar eitt af sex hliðum 74HC14 rásarinnar.



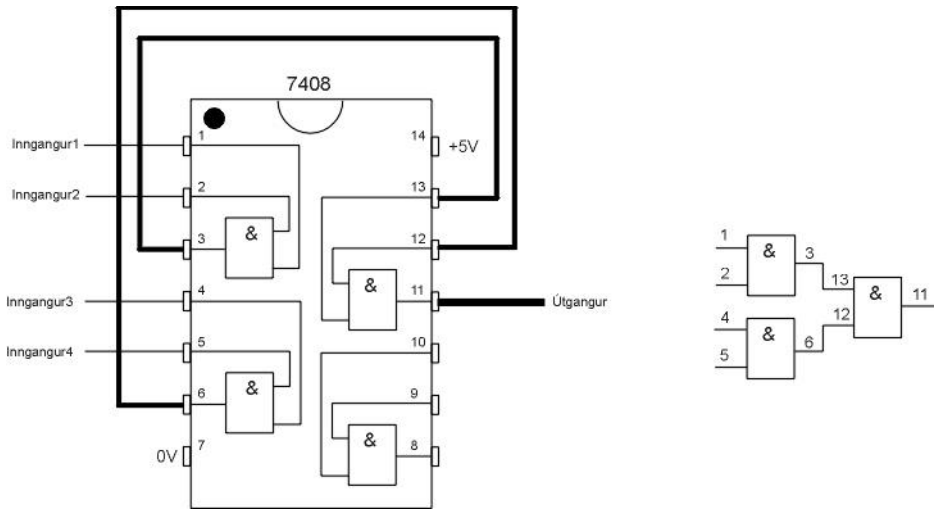
Prentplata fyrir sexfalda Schmitt trigger afskoppunarrás.



Íhlutamynd fyrir sexfalda Schmitt trigger afskoppunarrás.

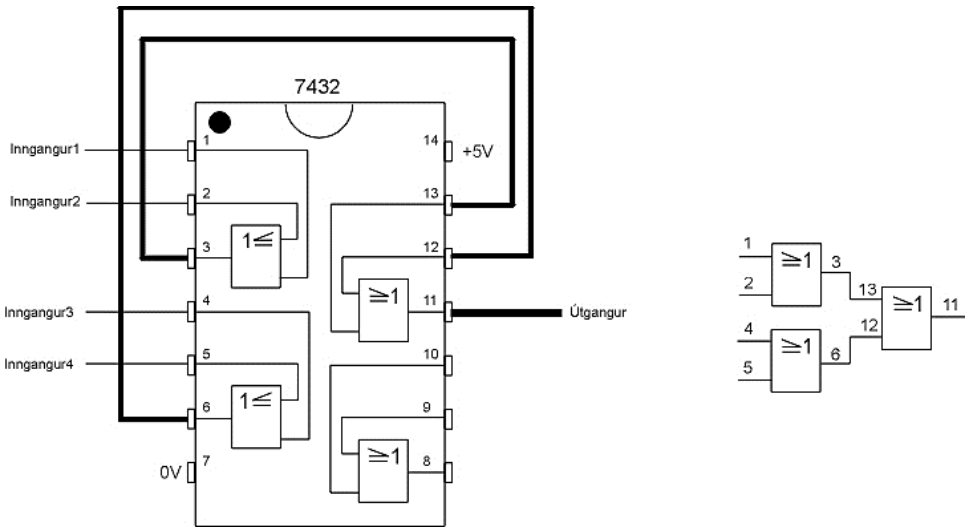
1.10 Samtenging hliða ef fleiri innganga er þörf (fjölgun innganga)

Ef við þurfum fleiri innganga en í boði eru á einu hliði þá má tengja sama tvö eða fleiri hlið til að fjölga þeim. Hugsum okkur að við þurfum fjögurra innganga AND hlið en eigum bara 7408 rásina sem er með fjórum tveggja innganga AND hliðum.



Tengjum inngangana fjóra inn á tvö AND hlið. Tökum útganga þeirra inn á þriðja AND hliðið og við höfum fjögurra innganga AND hlið búið til úr þremur tveggja innganga AND hliðum. Það er auðvelt að sanna með Boolean algebru að þetta gefur sömu niðurstöðu og ef við værum með eitt fjögurra innganga AND hlið.

Sama gildir um OR hlið. Við getum búið til fjögurra innganga OR úr þremur tveggja innganga OR hliðum. 7432 rásin inniheldur fjögur tveggja innganga OR hlið.



1.11 Verkefni 1

Skoðið dæmið hér að framan.
Gildir þetta líka um NAND og NOR?
Það er best að sanna eða afsanna með því að búa til tengimynd og nota síðan Boolean algebru.

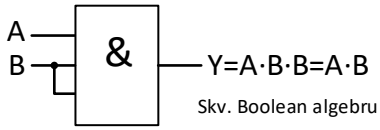
1.12 Ónotaðir inngangar (fækkun innganga)

Aldrei má skilja eftir ótengda innganga á rökrásum. Þá eru þeir það sem kallað er „fljótandi" og óvíst hvort spennan á þeim sé í lág eða hástöðu. Það þarf því að gera eitthvað við þessa innganga. Það eru tveir möguleikar í boði.

1. Tengja ónotaða innganginn við næsta samliggjandi inngang.
2. Tengja ónotaða innganginn við „hlutlausa" stöðu.

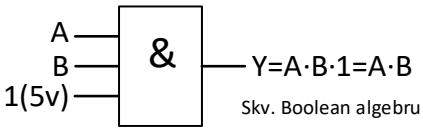
Dæmi 1

Okkur vantar tveggja innganga AND en höfum bara þriggja innganga. Tengjum neðsta innganginn við miðjuinnganginn. Þá er hliðið orðið tveggja innganga.



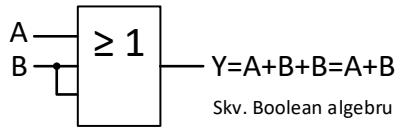
Dæmi 2

Okkur vantar tveggja innganga AND en höfum bara þriggja innganga
Tengjum neðsta innganginn í hástöðu (1) því 1 er margföldunarhlutleysa og AND hliðið er margföldunarrás.



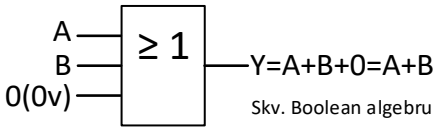
Dæmi 3

Okkur vantar tveggja innganga OR en höfum bara þriggja innganga.
Tengjum neðsta innganginn við miðjuinnganginn. Þá er hliðið orðið tveggja innganga.



Dæmi 4

Okkur vantar tveggja innganga OR en höfum bara þriggja innganga.
Tengjum neðsta inn ganginn í lágstöðu (0) því 0 er samlagningarhlutleysa og OR hliðið er samlagningarrás.



1.13 Verkefni 2

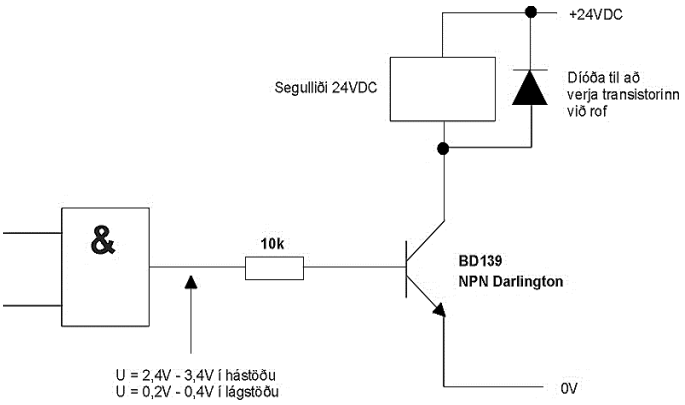
Gilda þessar aðferðir í dæmum 1-4 líka fyrir NAND og NOR?
Hver er hlutlausa staðan fyrir NAND og hver er hlutlausa staðan fyrir NOR?

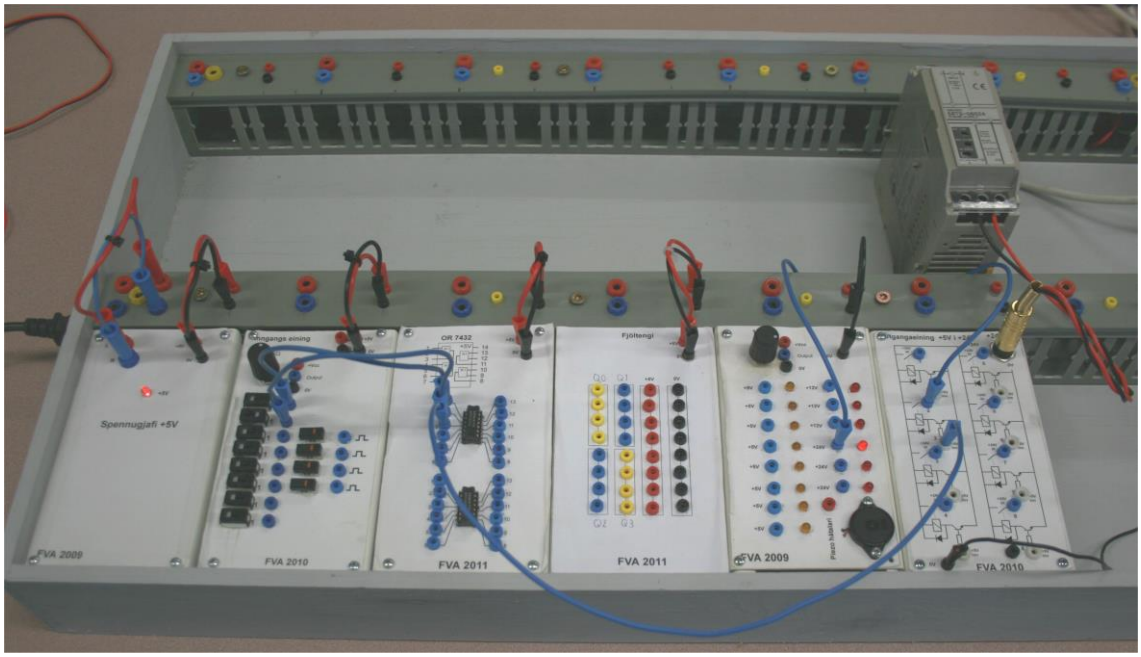
1.14 Útgangsmögnun

Venjulegur útgangur á rökrás getur ekki drifið straumþunga innganga á eftir sér og því þurfum við að setja mögnunarstig á milli. Við getum í raun látið útgang á rökrás stýra hvaða álagi sem er með réttu mögnunarstigi eða stigum

1.15 Transistorar

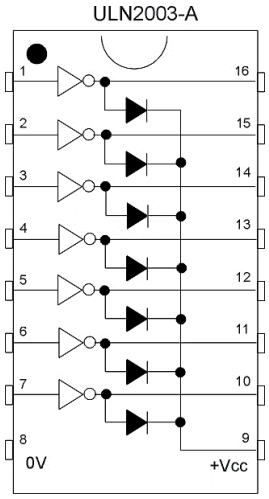
Hugsum okkur að við viljum láta rökrásarútgang kveikja á 24VDC segulliða. Það er ljóst að við tengjum segulliðann ekki beint á útganginn því hann hefur hvorki spennu né straumgetu til að drífa hann beint. En við getum t.d. sett transistor á milli. Þá er gott að nota Darlington transistor t.d. BD139 á milli og láta útganginn á rökrásinni vinna á base transistorsins.



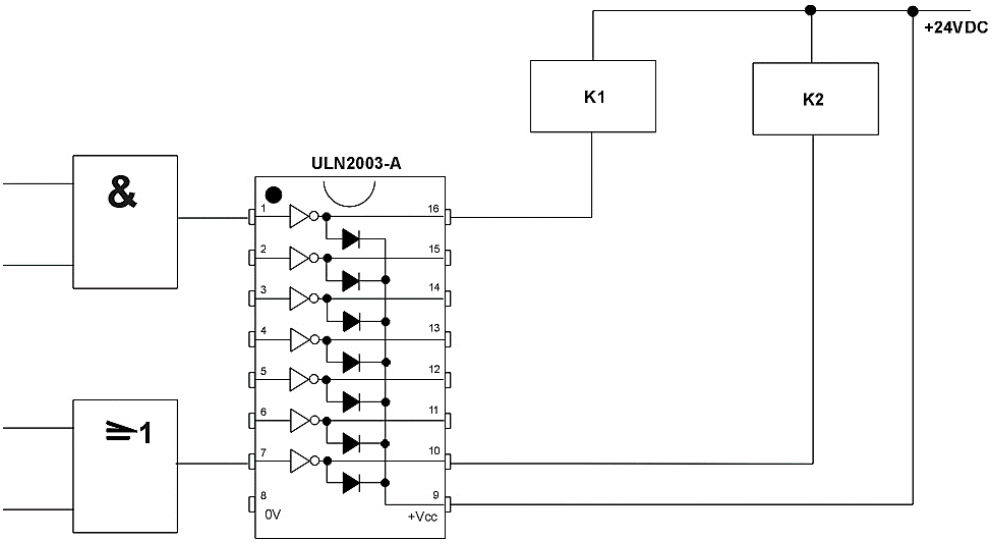


Á myndinni stýrir 7432 OR hlið 24VDC segulliða

1.16 ULN2003



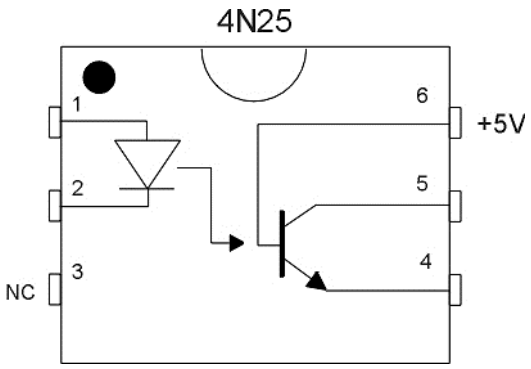
ULN2003 rásin er sjöfaldur Darlington driver. Hann hefur sjö inn og útganga og V_{CC} og 0V innganga. V_{CC} er fæðispenna hlutarins sem stýra á.



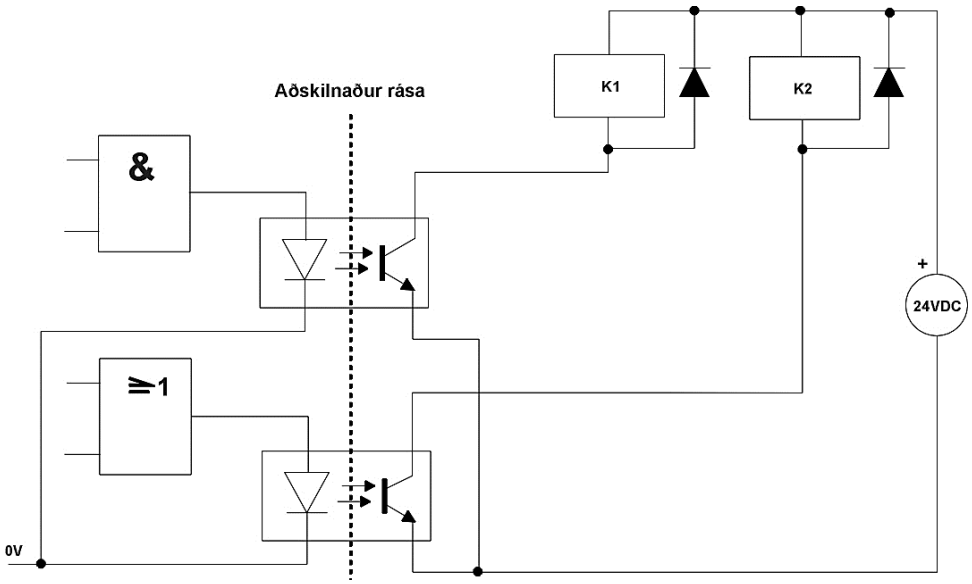
Á myndinni stýra AND og OR hlið 24VDC segulliðunum K1 og K2. Díóðurnar fyrir álagið eru innbyggðar í ULN2003 rásina og þarf því ekki að setja sérstaklega. ULN2003 rásin hentar sérstaklega vel í að stýra stórum sjö ráka LED stöfum vegna fjölda útganga. Stafirnir eru þá tengdir í CA (sameiginlegur +) og ULN2003 rásin sér um að tengja hvern staf við 0V og kveikja þannig á honum.

1.17 Ljóshlið (Optocoupler)

Ljóshlið aðskilur algerlega straumrás merkjagjafans og álagsins. Ljósdíóða í inngangsstigi ljóshliðsins sendir ljóspúls inn á BASE transistors í útgangsstigi ljóshliðsins. Með þessu móti gengur enginn straumur á milli inngangs og útgangs, bara ljós. Eina sem útgangur rökrásarinnar þarf að gera er að kveikja á ljósdíóðu.



4N25 Optocoupler rásin. Hún er með auka BASE tengingu þ.a. hægt er að nota transistorinn í útganginum sjálfstætt. Annars er pinni 6 skilinn eftir ótengdur.



Hér stýra tvö rökrásahlið tveimur segulliðum, K1 og K2, í gegnum ljóshlið. Enginn straumur gengur á milli rásanna.

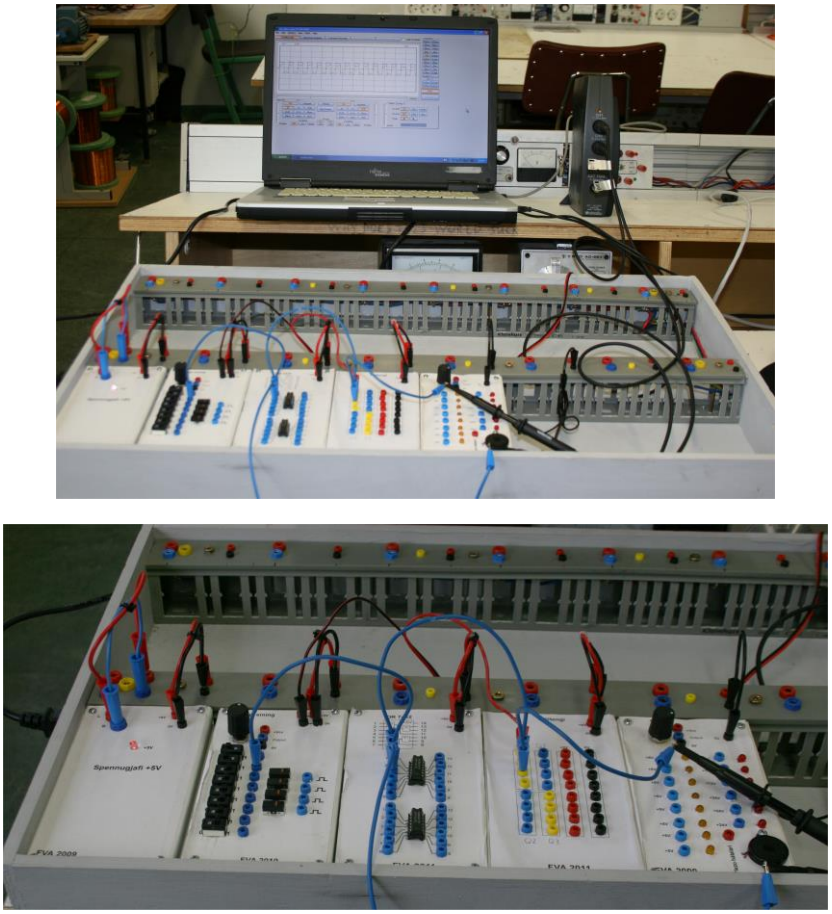
2. Leit að bilunum í rökrásum með mælitækjum og með hjálp hermiforrita

2.1 Hermiforrit

Það getur verið gott í hönnun rökrásastýringa að setja rásina upp í hermiforriti eins og Multisim eða Electronic Workbench. Þá er hægt að prófa rásina á tölvuskjá og skoða hvað gerist ef settar eru inn bilanir. Til þess að fá raunverulegar niðurstöður má samt ekki nota „ideal rásir". T.d. á ekki að vera leyfilegt að skilja eftir ótengdan inngang á rökrás.

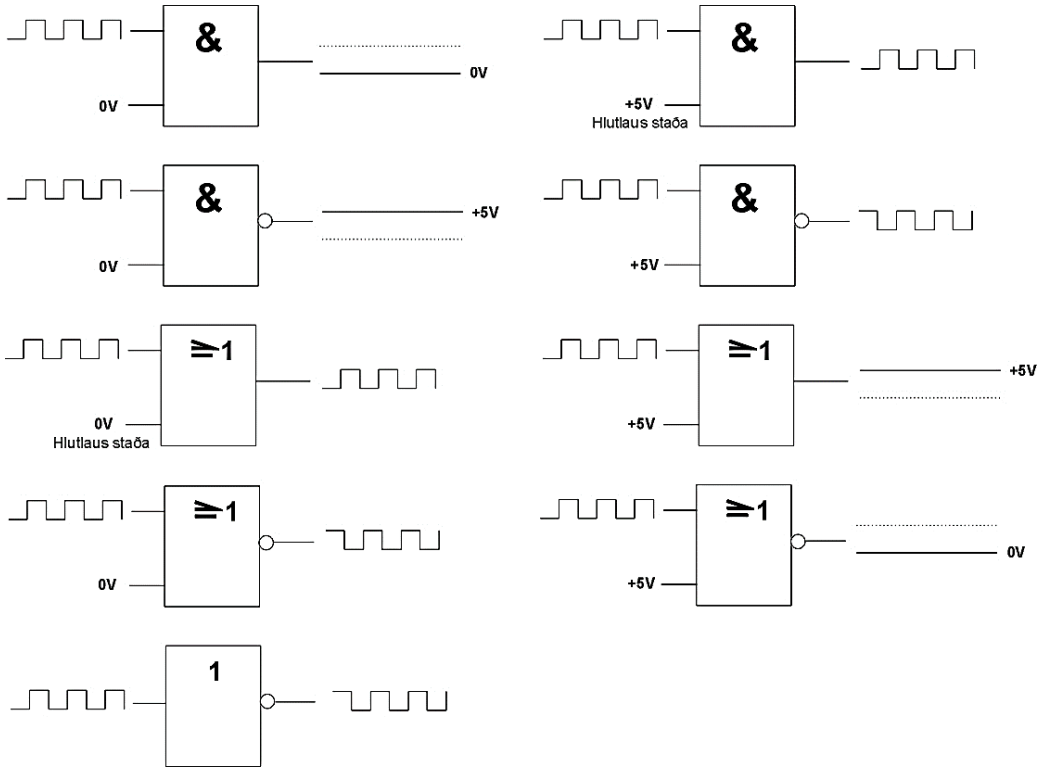
2.2 Sveiflusjár

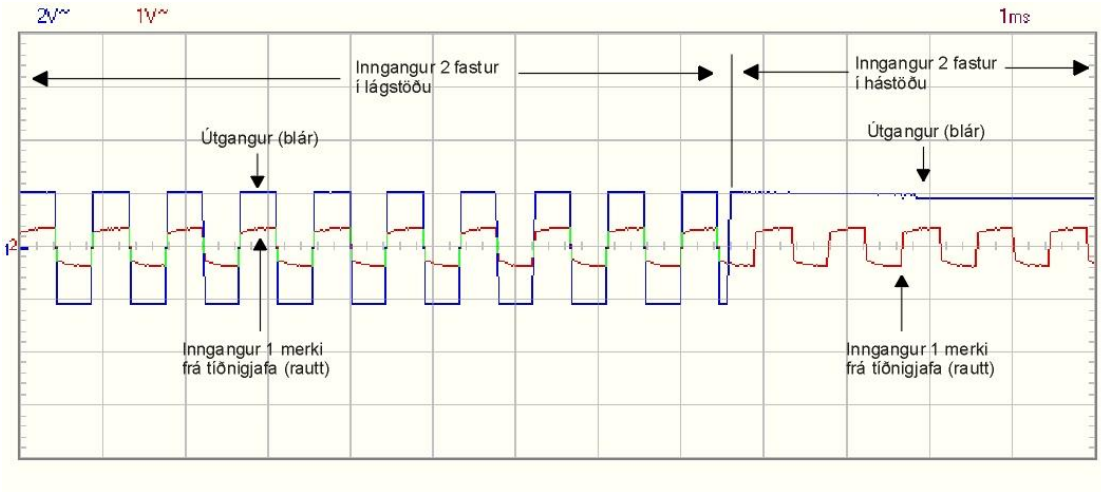
Til þess að leita að bilunum í raunverulegum rásum er algengast að nota stafrænar sveiflusjár eða svokallaðar próbur.



Uppsetning til kennslu og prófana á rökrásum.

Myndirnar hér að neðan sýna hvað við ættum að sjá á sveiflusjá ef hliðin eru í lagi.





Mæling á OR hliði 7432. Skjámynd af stafrænni sveiflusjá. Sjáum að henni ber saman við myndirnar tvær í þriðju röð yfir hvað við ættum að sjá á sveiflusjá.

Annar inngangurinn er festur í upphafi í lágstöðu sem er hlutlaus staða fyrir OR hlið. Útgangur á þá að líta út eins og inngangurinn ef rásin er í lagi.

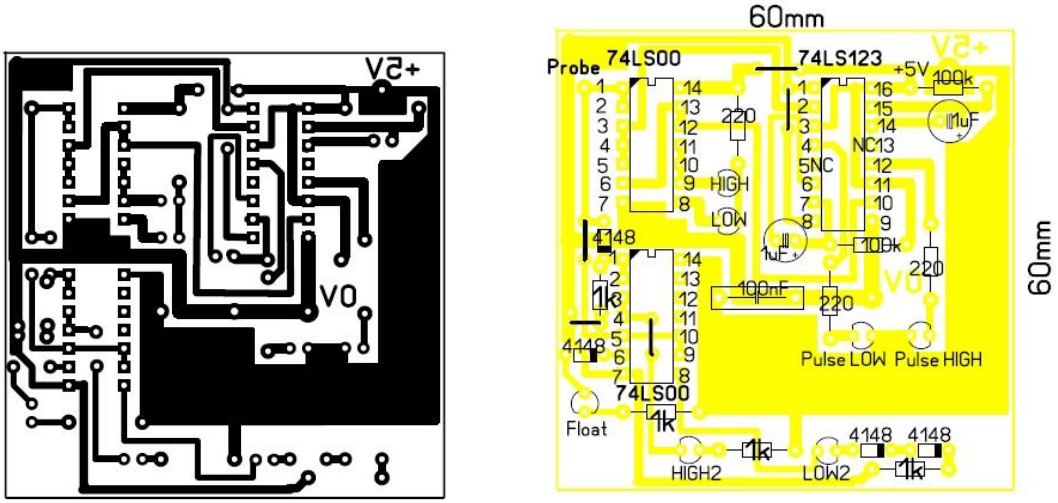
Hinn inngangurinn er tengdur við tíðnigjafa sem gefur út kassapúls c.a. 1kHz (rauður ferill). Sjáum að útgangurinn hermir eftir innganginum eins og við vildum. Þegar inngangurinn sem var í lágstöðu er tengdur við hástöðu þá verður hann ráðandi og útgangurinn festist í hástöðu. Þetta hlið er því greinilega í lagi.

2.3 Próbur

Sveiflusjá er ekki alltaf tiltæk svo gott er að hafa eitthvað nettara bilunarleitartæki t.d. próbu.
Til eru margar gerðir af próbum og við getum látið þær sýna okkur hvort tengipunktur sé í hástöðu, lágstöðu, púlsandi eða fljótandi. Próbur er hægt að kaupa eða einfaldlega smíða sér þær sjálfur.



Mynd af próbu. Svarta og rauða klemman fara í 0V og +5V. Pinninn er settur á þann punkt í rásinni sem mæla á.



Prent af próbu sem auðvelt er að smíða og setja í kassa

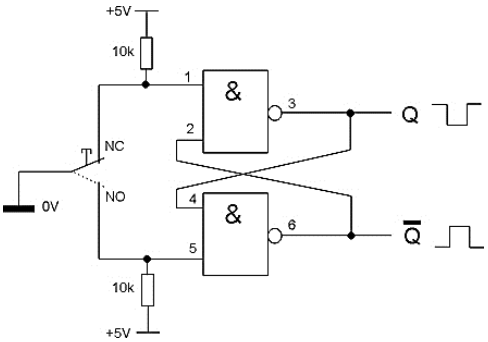
3. Lásar og vippur

3.1 Lásar

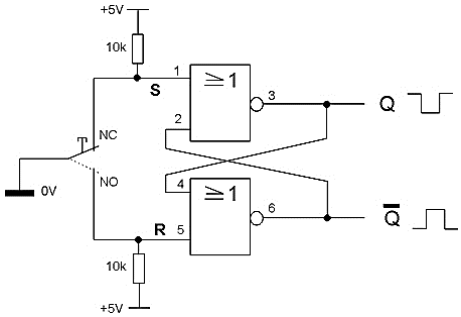
RS lásinn er notaður til þess að geyma stöður. Hann virkar svipað og hin klassíska segulliðastýring með start, stopp og sjálfheldu. Ef S inngangurinn fær púls þá fer Q útgangurinn í 1 og helst þar og ef R inngangurinn fær púls þá fer Q útgangurinn í 0 og helst þar. Stöðurnar læsast og þaðan kemur nafnið lásar.

Til eru 2 grunngerðir af lásum. Það eru annarsvegar NAND lás og hinsvegar NOR lás.

Á myndinni hér að neðan er NAND lás þ.e. hann er búinn til úr tveimur NAND hliðum. Til þess að setja hann þá þarf að setja 0 á S innganginn. („active low“) og til þess hreinsa hann þá þarf að setja 0 á R innganginn. NAND lásinn má ekki fá stöðuna $S = R = 0$. Þá fara báðir útgangar hans í stöðuna $Q = \bar{Q} = 1$ sem er bönnuð staða.



NAND lás



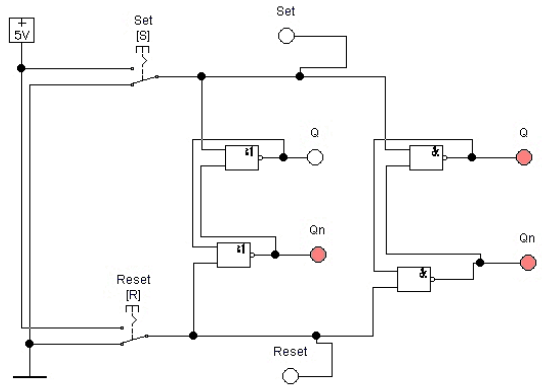
NOR lás

NOR lásinn er líka settur og hreinsaður með 0 („active low“ inngangar). NOR lásinn má ekki fá stöðuna $S = R = 1$. Þá fara báðir útgangar hans í stöðuna $Q = \bar{Q} = 0$ sem er bönnuð staða.

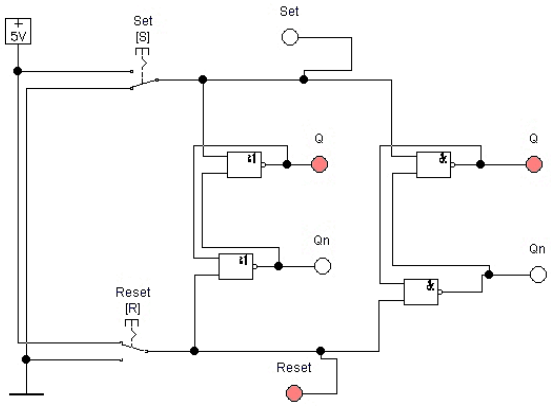
3.2 Sanntöflur

RS - NAND lás					RS - NOR lás				
S	**R**	***Q***	$\overline{Q}$		**S**	**R**	***Q***	$\overline{Q}$	
0	0	1	1	Bannað	0	0	Hold	Hold	
0	1	1	0		0	1	1	0	
1	0	0	1		1	0	0	1	
1	1	Hold	Hold		1	1	0	0	Bannað

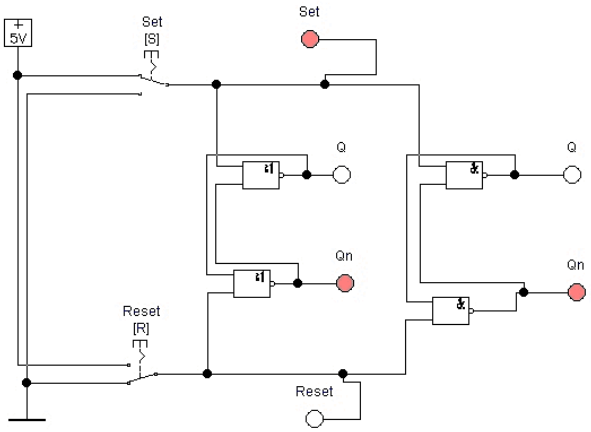
3.3 NOR og NAND lás í Electronic Workbench



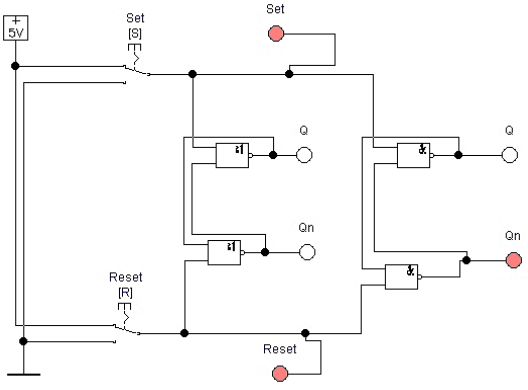
S = 0, R = 0



S = 0, R = 1

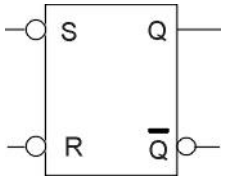


S = 1, R = 0



S = 1, R = 1

RS lás hefur táknið:



3.4 RS vippur

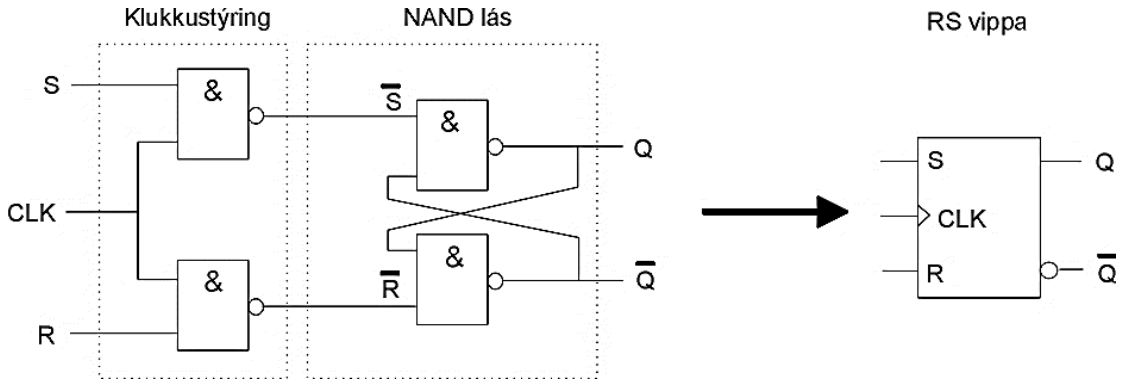
Vippur eru klukkustýrðar. Það þýðir að það er ekki nóg að breyta inngöngum þeirra til þess að fá breytingu á útgangi eins og á lásum og venjulegum rökrásum, heldur verða þær líka að fá klukkupúls. Þetta kallast Syncron rásir. Vippur þekkjast á því að þær hafa klukkuinngang. Til eru tvær gerðir af klukkuinngöngum, active high og active low.



Active high Active Low

Active high inngangurinn er virkur þegar klukkupúlsinn breytist úr 0 í 1 eða á forkanti. Active low inngangurinn er virkur þegar klukkupúlsinn breytist úr 1 í 0 eða á bakkanti. Þetta er líka kallað randstýrðar rásir. Klukkuinngangar eru auðkenndir með þríhyrningi.

RS vippa er eins og NAND lás með klukkuinngangi.



Í syncron rásum breytast útgangarnir ekki um leið og inngangar breytast. Við þurfum því að skilgreina nýja gerð af sanntöflu svokallaða stöðubreytingatöflu (Transition table). Ný staða á útgangi getur nefnilega verið háð því hvaða staða var á útganginum þegar klukkupúlsinn kom. Núverandi útgangur, táknaður Q^n, er því talinn með inngöngunum í stöðubreytingatöflu. Útgangur töflunnar verður staðan eftir næsta klukkupúls og er táknaður með Q^{n+1}.
Taflan verður þá:

S	**R**	**Q^n**	**Q^{n+1}**
0	0	0	0 (Hold)
0	0	1	1 (Hold)
0	1	0	0 (Reset)
0	1	1	1 (Reset)
1	0	0	1 (Set)
1	0	1	1 (Set)
1	1	0	1 (Bannað)
1	1	1	1 (Bannað)

Sé þessi tafla sett í K-kort fæst eftirfarandi útgangsjafna fyrir RS vippu:

$$\boldsymbol{Q^{n+1} = S + \bar{R}Q^n}$$

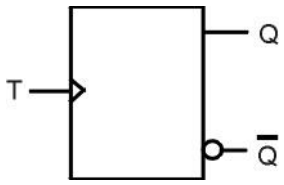
Í bönnuðu stöðunni S = R = 1 fara báðir útgangarnir Q og $\bar{Q}$ í hástöðu.

3.5 T vippur

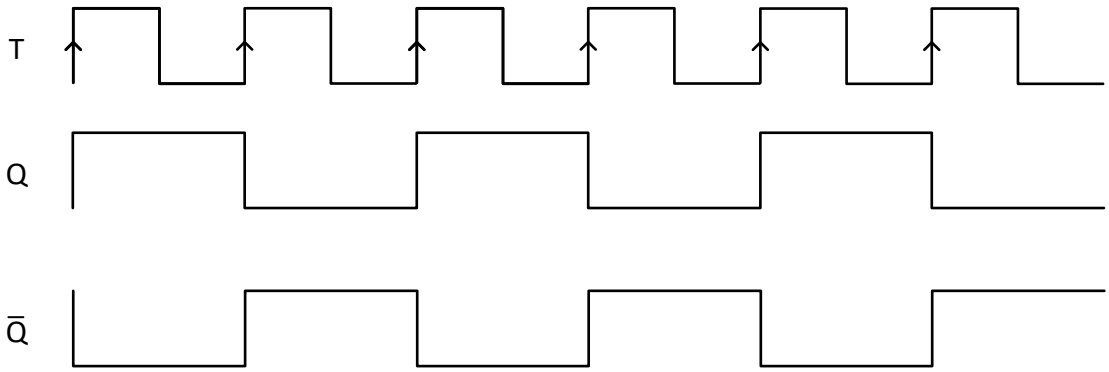
T vippa (Toggle Flip Flop) er einföldust allra vippa. Hún einfaldlega skiptir um stöðu við forkant hvers klukkupúls.

Hún hefur útgangsjöfnuna $\boldsymbol{Q^{n+1} = \overline{Q^n}}$

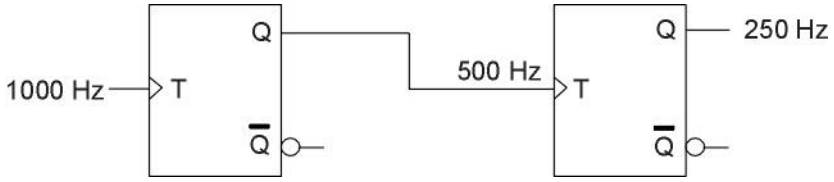
Tákn hennar er:



Púlsaritið er:



Það er hægt að nota hana t.d. til að lækka tíðni. Sjáum að Q útgangurinn hefur tvisvar sinnum lægri tíðni en T inngangurinn. Við getum tengt nokkrar T vippur saman á eftirfarandi hátt til að lækka tíðni.

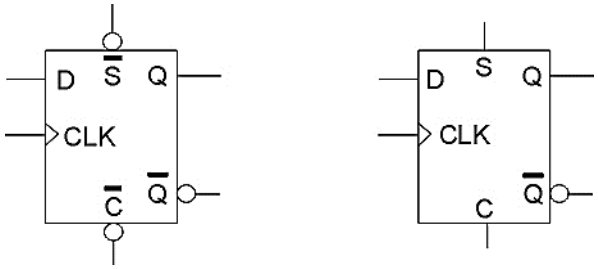


3.6 D vippur

D vippa (Data flip flop) er notuð í teljara og register. (kemur seinna) Hún hefur einn samfasa inngang (syncron) sem merktur er D. Hún hefur líka oftast tvo ósamfasa (asyncron) innganga, Set og Clear. Ósamfasa þýðir að þeir inngangar eru ekki háðir klukkupúlsinum. Ef þeir fá merki þá setja þeir útganginn í 1 eða hreinsa hann í 0 óháð klukkunni. Þeir hafa því forgang á klukkuinnganginn.

Útgangsjafna D vippu er: $\boldsymbol{Q^{n+1} = D}$ þ.e. núverandi staða á D flyst yfir á útgang við næsta klukkupúls.

Tákn D vippu er:



active low *active high*

Ef við viljum ekki nota ósamfasa inngangana S og C, þá verðum við að fasttengja þá í 1 ef þeir eru *active low* (vinstri myndin) eða í 0 ef þeir eru *active high* (hægri myndin).

3.7 JK vippur

JK vippa virkar svipað og RS vippa með J = S og K = R, nema hún hefur enga „bannaða“ stöðu.

Bannaða staðan R = S = 1 verður Toggle (Víxlun) á JK vippu ef J = K = 1.

Stöðubreytingatafla JK vippu er:

J	K	Q^n	Q^{n+1}
0	0	0	0 (Hold)
0	0	1	1 (Hold)
0	1	0	0 (Reset)
0	1	1	1 (Reset)
1	0	0	1 (Set)
1	0	1	1 (Set)
1	1	0	1 (Toggle)
1	1	1	0 (Toggle)

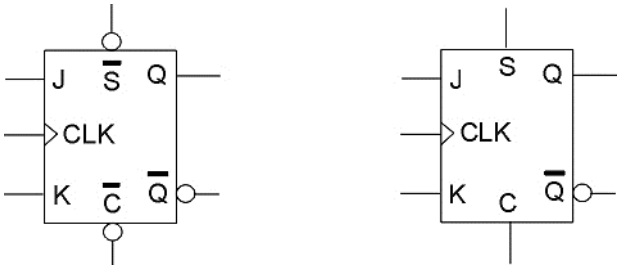
Ef sanntaflan er sett í K kort þá sést að JK vippa hefur útgangsjöfnuna :

$$Q^{n+1} = J\overline{Q^n} + \overline{K}Q^n$$

Til er einfölduð útgáfa af stöðubreytingatöflunni sem notuð er í teljararásum þ.e. ef við höfum bara áhuga á stöðunum Hold og Toggle.

J	K	Q^n	Q^{n+1}
0	X	**0**	**0** (Hold)
X	0	**1**	**1** (Hold)
1	X	**0**	**1** (Toggle)
x	1	**1**	**0** (Toggle)

Tákn JK vippu er:



S og C inngangarnir eru ósamfasa og hafa því forgang á J og K inngangana sem eru háðir klukkunni.

3.8 Einföldun á sannleikstöflu JK vippu

Stöðubreytingatafla JK vippu er:

J	**K**	**Qn**	**Qn+1**
0	0	**0**	**0**
0	**0**	**1**	**1**
0	1	**0**	**0**
0	**1**	**1**	**0**
1	0	**0**	**1**
1	**0**	**1**	**1**
1	1	**0**	**1**
1	**1**	**1**	**0**

→

J	**K**	**Qn**	**Qn+1**
0	*0*	0	0
0	*1*	0	0
1	*0*	0	1
1	*1*	0	1
0	1	1	0
1	1	1	0
0	0	1	1
1	0	1	1

Ef við skoðum línur í sannleikstöflunni þar sem breytingin frá Q^n í Q^{n+1} er eins þ.e. 0 → 0, 0 → 1, 1 → 0, 1 → 1 þá sést að stundum skiptir ekki máli hvað J er og stundum ekki máli hvað K er.

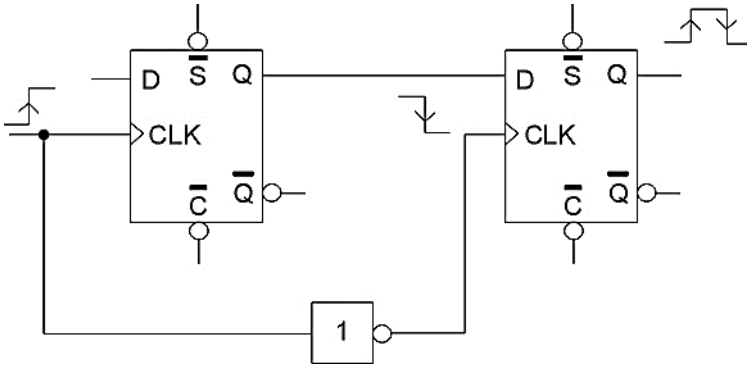
Það sem skiptir máli er feitletrað í töflunni að ofan.

T.d. í breytingunni 0 →0 þá skiptir ekki máli hvað K er svo við getum táknað það með x. (don’t care) Þannig er hægt að fara í gegnum allar stöður. Einfaldaða sannleikstaflan verður þá:

J	K	Q^n (núverandi)	Breyting	Q^{n+1} (næsta)
0	x	**0**	→	**0**
x	**0**	**1**	→	**1**
1	x	**0**	→	**1**
x	**1**	**1**	→	**0**

3.9 Master Slave vippur

Master Slave vippur eru búnar til með því að raðtengja tvær D vippur og snúa klukkuinnganginum á seinni vippunni við með NOT hliði.



Fyrri vippan (Master) stýrir þeirri seinni (Slave). Breytingin frá inngangi út á útgang tekur heilan klukkupúls í Master Slave vippu en ekki bara forkant eða bakkant eins og í venjulegri D vippu. Á forkantinum þá les fyrri vippan stöðuna inn og geymir hana. Á bakkantinum er hún lesin út á Q útganginn á seinni vippunni.

3.10 Breyting úr einni vippugerð í aðra

Hægt er að breyta einni vippugerð í aðra einfaldari með því að nota útgangsjöfnur þeirra:

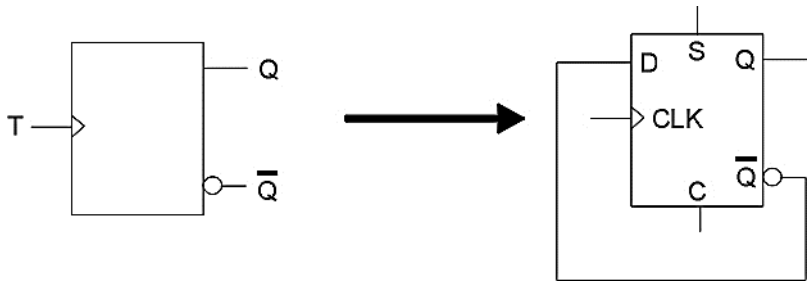
T vippa: $\boldsymbol{Q^{n+1} = \overline{Q^n}}$

D vippa: $\boldsymbol{Q^{n+1} = D}$

JK vippa: $\boldsymbol{Q^{n+1} = J\overline{Q^n} + \overline{K}Q^n}$

Búum til T vippu úr D vippu: Setjum = á milli útgangsjafna þeirra.

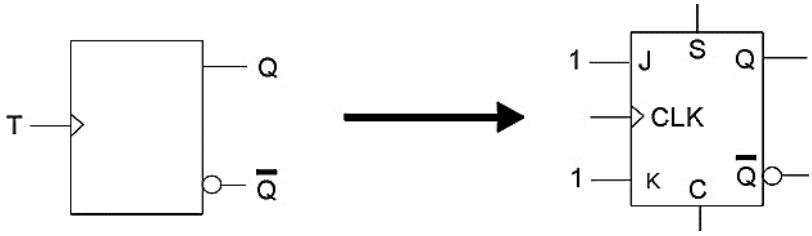
$$\boldsymbol{Q^{n+1} = \overline{Q^n} = Q^{n+1} = D \Rightarrow \overline{Q^n} = D}$$



Búum til T vippu úr JK vippu: Setjum = á milli útgangsjafna þeirra.

$$\boldsymbol{Q^{n+1} = \overline{Q^n} = Q^{n+1} = J\overline{Q^n} + \overline{K}Q^n \Rightarrow \overline{Q^n} = J\overline{Q^n} + \overline{K}Q^n}$$

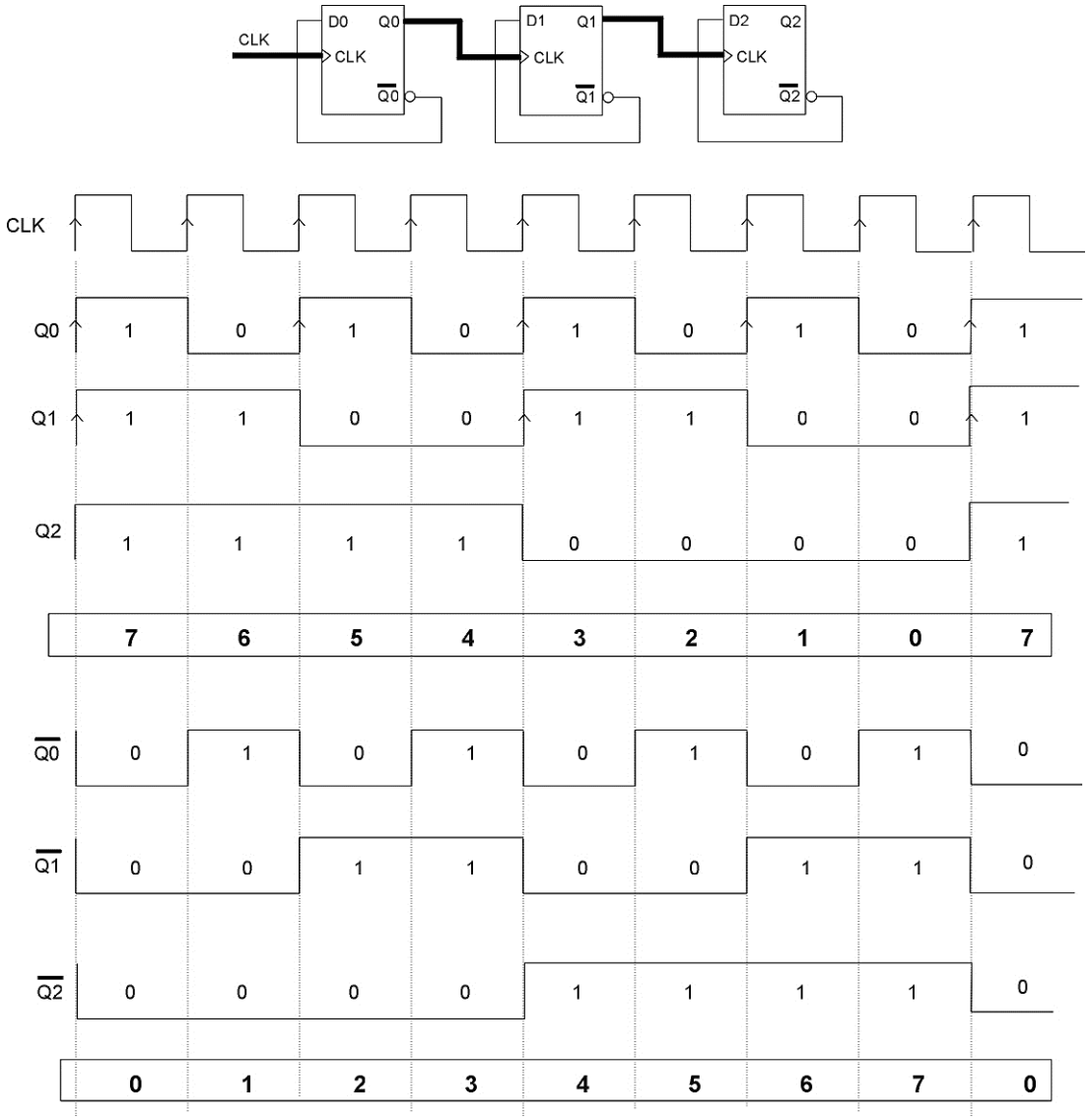
Þetta gefur að J = K = 1 setur JK vippuna í Toggle stöðu.



4. Ósamfasa og samfasa teljarar

4.1 Ósamfasa teljarar

Til eru tvær gerðir af teljurum, ósamfasa (asyncron) og samfasa (syncron). Ósamfasa teljarar eru þannig að klukkupúlsinn kemur bara inn á fyrstu vippuna og útgangur fyrstu vippunnar er síðan klukkuinngangur vippu númer 2 o.s.frv. Skoðum eftirfarandi ósamfasa teljara:



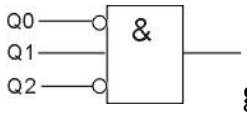
Sjáum að D vippurnar eru tengdar í Toggle tengingu. Fyrsta vippan víxlar á hverjum klukkupúls, önnur á öðrum hverjum og þriðja á fjórða hverjum. Q útgangarnir mynda því rununa 111, 110, 101, 100, 011, 010, 001, 000, 111, .. eða 7, 6, 5, 4, 3, 2, 1, 0, 7, … Þetta er því þriggja bita niðurteljari.

Hinsvegar búa $\bar{Q}$ útgangarnir til öfuga runu við Q þ.e. þeir mynda rununa 000, 001, 010, 011, 100, 101, 110, 111, 000,… eða rununa 0, 1, 2, 3, 4, 5, 6, 7, 0, … Þeir mynda því þriggja bita uppteljara.

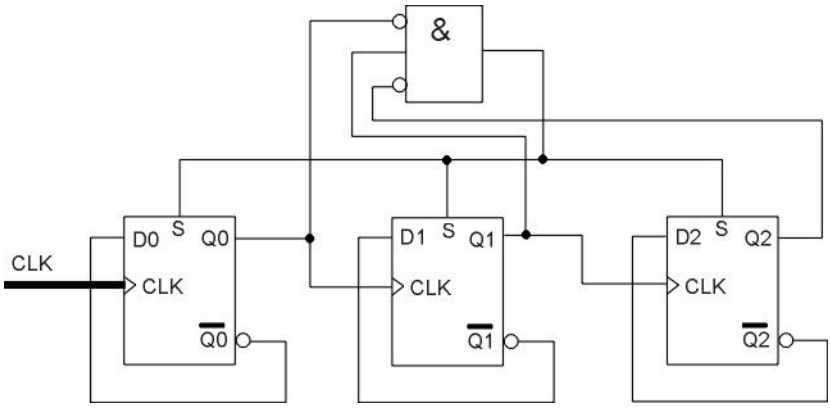
Gallinn við ósamfasa teljara er að það er seinkun í hverri vippu og ef tíðni klukkupúlsins er há þá er ekki víst að þeir nái allir að fylgja eftir.

4.2 Hönnun ósamfasa teljara

Prófum að hanna 3 bita ósamfasa teljara sem telur 7, 6, 5, 4, 3, 7, 6, …. Við þurfum því að „setja" teljarann í stöðunni 2 þ.e. þvinga alla útgangana í 1 svo hann byrji í 7 aftur (7 = 111). Staðan 2 er 010. Þegar hún kemur upp þá þurfum við að fá 1 inn á Set inngangana (gerum ráð fyrir að þeir séu active high).

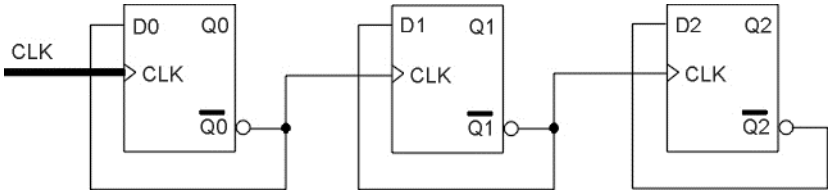


Rásin gefur út 1 í stöðunni 010 en 0 annars. Við getum því notað hana til þess að setja vippurnar í 111.

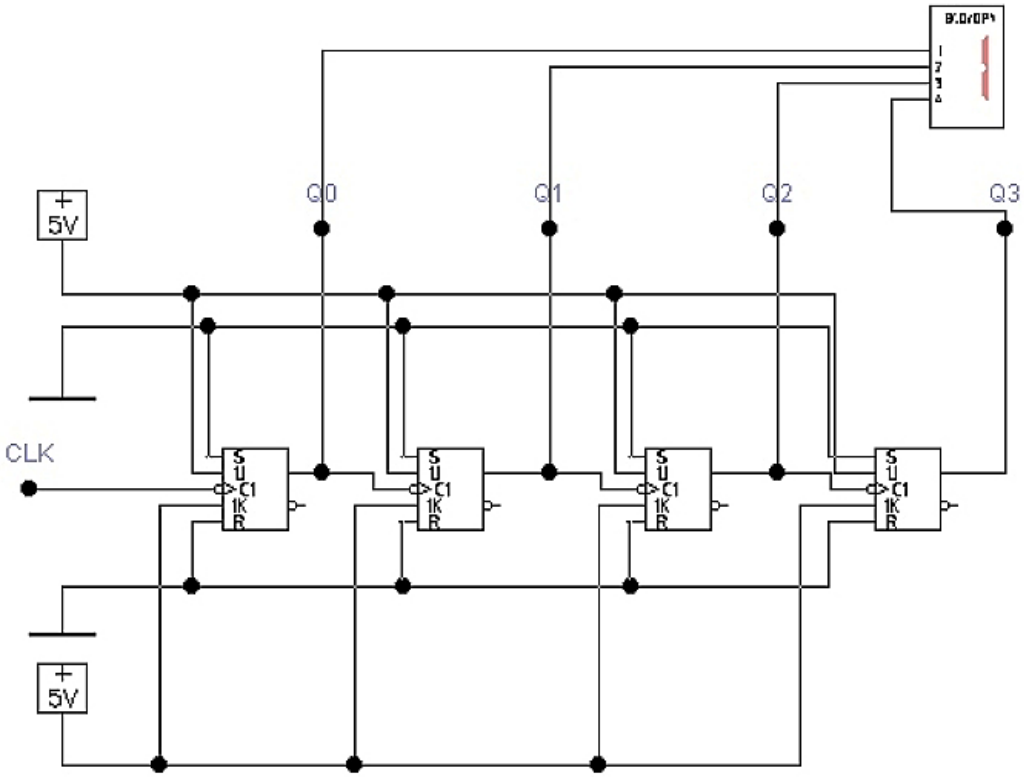


Þessi teljari telur því : 7, 6, 5, 4, 3, 7, 6, ….

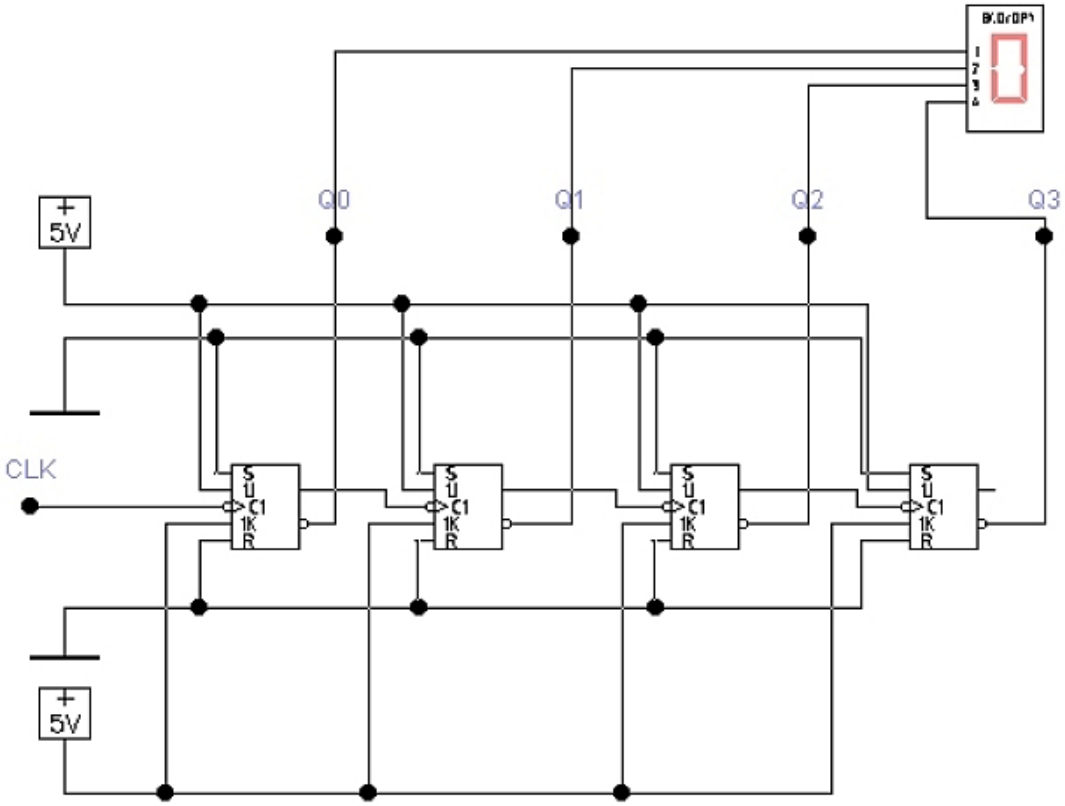
Ef við viljum hanna uppteljara þá þarf að breyta tengingunum í rásinni á eftirfarandi hátt.



eða nota $\overline{Q}$ útgangana inn á display.



JK 4 bita ósamfasa niðurteljari í Electronics Workbench



JK 4 bita ósamfasa uppteljari í Electronic Workbench.

4.3 Hönnun samfasa teljara

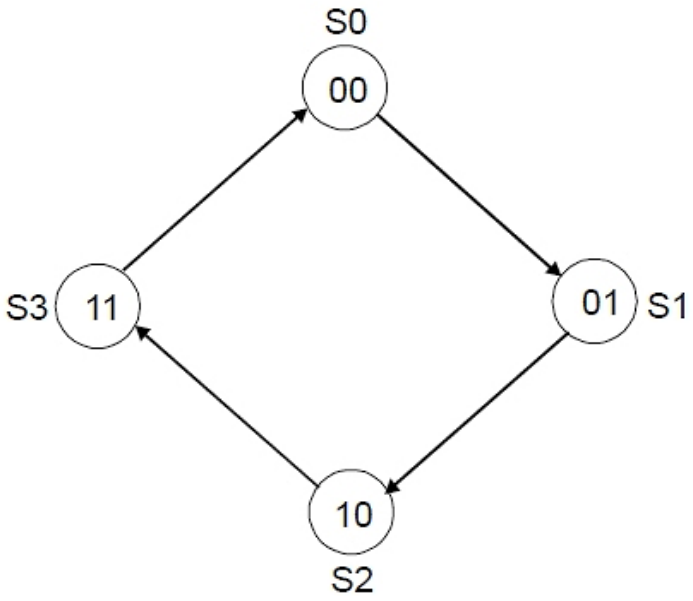
Hönnun samfasa teljara fer fram í 6 skrefum:

1. Stöðurit (state diagram)
2. Stöðutafla (state table)
3. Stöðubreytinga tafla (transition table)
4. Örvunarkort (exitation maps)
5. Örvunarjöfnur (exitation equations)
6. Teikna rásina

Dæmi 1:

Hönnum samfasa teljara sem telur 0-1-2-3-0-1-2-3….

Stöðurit



Stöðutafla

S^n er núverandi staða, S^{n+1} er næsta staða.

S^n	S^{n+1}
S0	S1
S1	S2
S2	S3
S3	S0

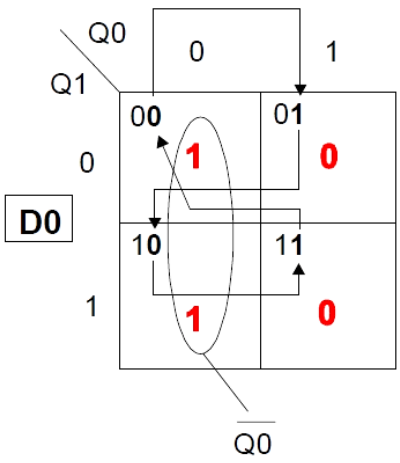
Stöðubreytingatafla

Hér tengjum við hverja stöðu í stöðutöflunni við útganga vippanna. Þetta er tveggja bita teljari svo við notum tvær vippur og köllum útgangana Q_0 (LSB) og Q_1 (MSB). Skref 1 til 3 eru óháð vippugerð.

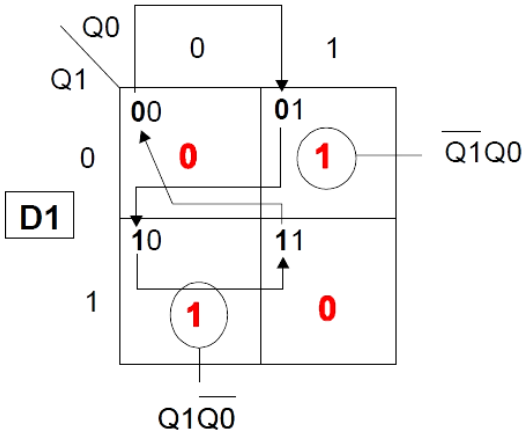
	Q_1^n	Q_0^n	Q_1^{n+1}	Q_0^{n+1}
S0	0	0	0	1
S1	0	1	1	0
S2	1	0	1	1
S3	1	1	0	0

Örvunarkort

Hér þurfum við að ákveða hvaða vippugerð við notum. Notum D vippu. Það er einfaldara að hanna með D vippum, en rásirnar vilja verða flóknari en ef við notum JK vippur. Örvunarkortið er nokkurskonar K-kort þar sem hver reitur inniheldur gildið á D innganginum sem þarf til þess að vippan skipti úr stöðu n yfir í stöðu n+1 við næsta klukkupúls. Við erum með tvo innganga D0 og D1 og þurfum að búa til sitthvort kortið fyrir þá.



D0 = $\overline{Q0}$



D1 = $Q1\overline{Q0} + \overline{Q1}Q0 = Q0 \oplus Q1$

Sjáum að í fyrra kortinu þá höfum við bara áhuga á seinni bitanum. Það er vegna þess að D0 inngangurinn getur bara breytt honum, hann er Q0.
Til þess að breyta Q0 úr 0 í 1 þarf D0 að vera 1 þegar klukkupúlsinn kemur.
Útgangsjafna D vippu er Q^{n+1}= D.
Næst grúppum við saman reiti sem innihalda 1, lóðrétt eða lárétt.
Í fyrra kortinu getum við grúppað saman tvo reiti lóðrétt. Þeir innihalda stöðurnar 00 og 10. Sjáum að Q1 breytist (er 0 og 1) og dettur því út. Q0 útgangurinn er hinsvegar 0 í báðum reitum og verður því $D0 = \overline{Q0}$ í jöfnunni.

Í seinna kortinu náum við ekki að grúppa neitt saman og reitirnir sem innihalda 1 verða því stakir. Sá sem er efst til hægri inniheldur 01 og verður því $\overline{Q1} \cdot Q0$. Hinn inniheldur 10 og verður því $Q1 \cdot \overline{Q0}$.

Til samans mynda þeir jöfnuna:

$$D1 = \overline{Q1} \cdot Q0 + Q1 \cdot \overline{Q0} = Q1 \oplus Q0.$$

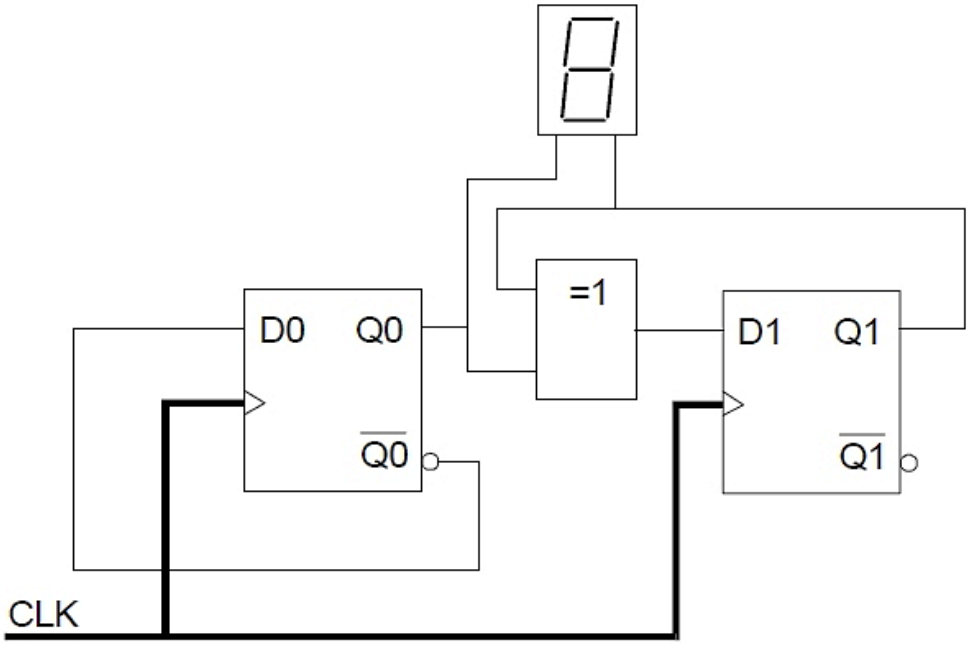
Þetta er því XOR hlið.

Örvunarjöfnur

Fundum þær í skrefi 4

$$D0 = \overline{Q0} \text{ og } D1 = \overline{Q1} \cdot Q0 + Q1 \cdot \overline{Q0} = Q1 \oplus Q0$$

Teiknið rásina



Sjáum að fyrri D vippan er tengd í Toggle mode þ.e. næsta staða verður öfugt við núverandi stöðu.

Þetta er frekar einfaldur teljari. Þegar við höfum náð góðu valdi á þessari hönnun er nóg að byrja í skrefi 4 þ.e. fara beint í örvunarkortið. Fyrstu þrjú skrefin eru frekar einföld.

4.4 Fleiri teljarar

Hönnum nú flóknari teljara. Hönnum BCD teljara með JK vippum.

Hann telur frá 0 upp í 9 þ.e. tugakerfistölurnar.

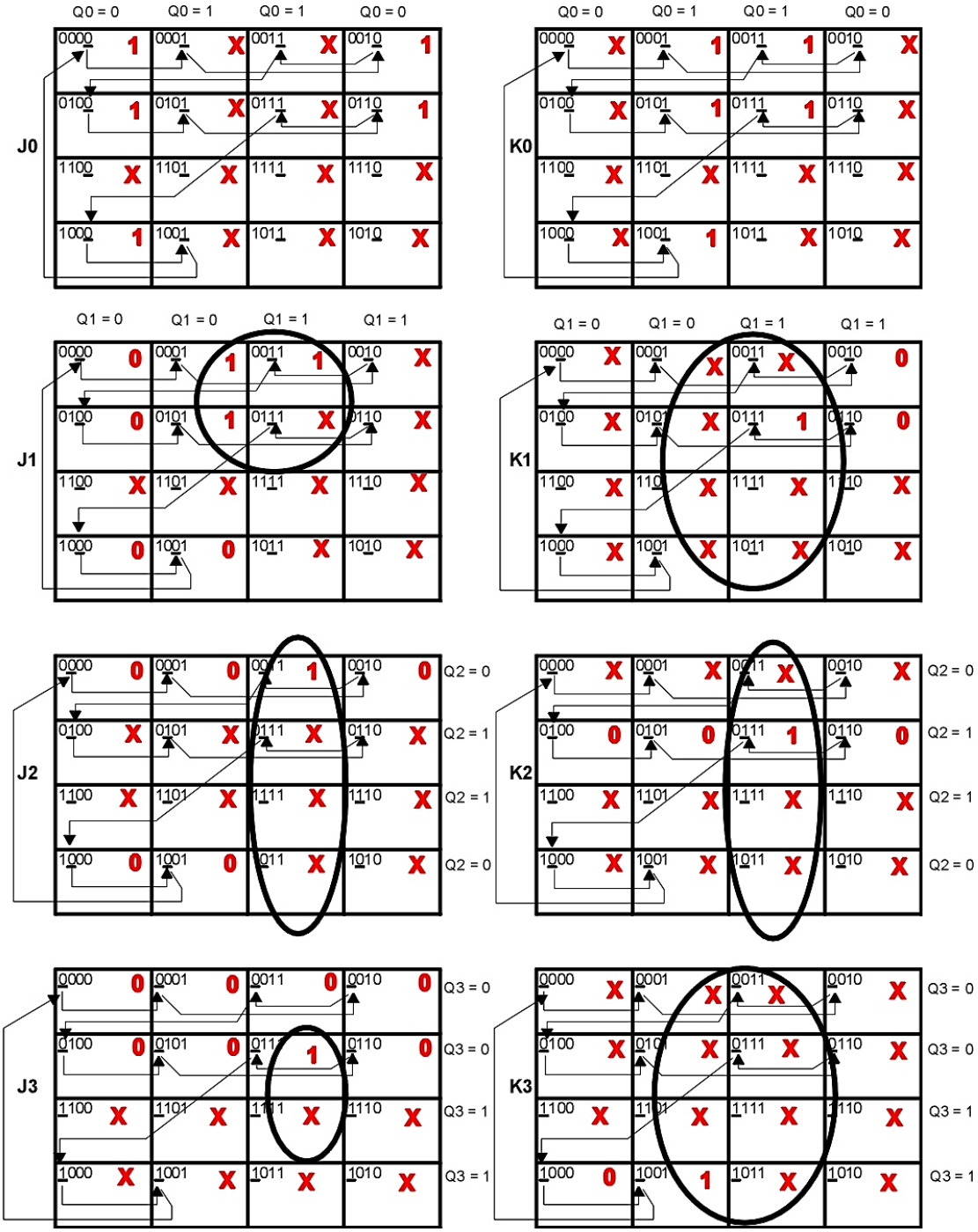
JK vippa hefur útgangsjöfnuna $Q^{n+1} = \overline{JQ^n} + \overline{K}Q^n$.
Út frá þessari jöfnu má setja upp stöðubreytingatöfluna: (x = skiptir ekki máli)

J	**K**	**Q^n**	**Q^{n+1}**
0	X	0	0 (Hold)
X	0	1	1 (Hold)
1	X	0	1 (Toggle)
X	1	1	0 (Toggle)

Við höfum bara áhuga á stöðunum Toggle og Hold, þess vegna getum við notað þessa einfölduðu sannleikstöflu fyrir JK vippu. Það er nefnilega kostur að fá mikið af stöðum sem skipta ekki máli (x = don´t care) því við getum ráðið því hvort við notum þær sem 1 eða 0 í K-kortinu.

Til þess að telja 0 … 9 þurfum við fjórar JK vippur. Við þurfum því að búa til átta örvunarkort þ.e. fyrir J0, K0, …, J3, K3.

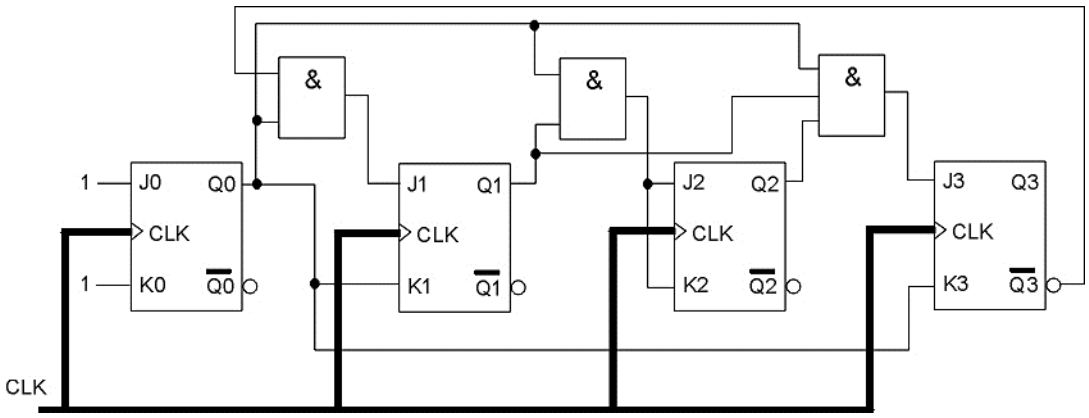
Þær stöður sem við notum ekki þ.e. tölurnar frá 10 til 15 (1010 til 1111) setjum við inn sem X, eða stöður sem skipta ekki máli (don’t care).



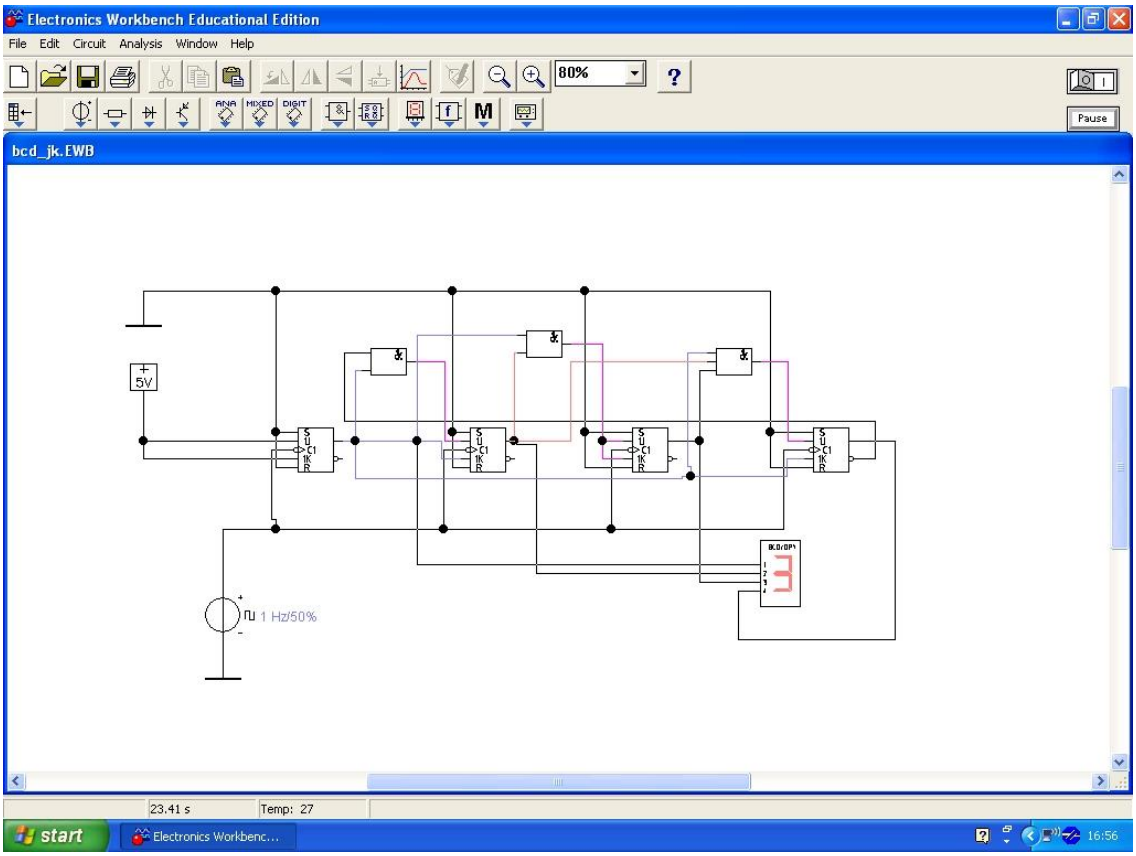
Út frá þessu korti koma jöfnurnar:

$$J0 = 1, \qquad K0 = 1, \qquad J1 = Q0 \cdot \overline{Q3}, \qquad K1 = Q0, \qquad J2 = Q0 \cdot Q1,$$
$$K2 = Q0 \cdot Q1, \qquad J3 = Q0 \cdot Q1 \cdot Q2, \qquad K3 = Q0$$

Þá er hægt að teikna teljarann:



Útgangana Q0 til Q3 má síðan tengja inn á display. Þetta er líka gott verkefni til þess að stilla upp í Multisim eða Electronic Workbench.

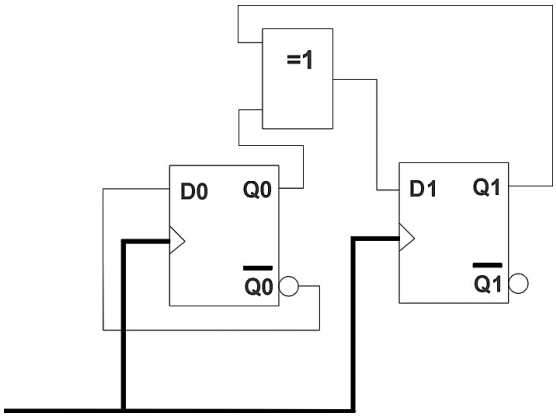


S og R inngangarnir eru Active High og því þarf að tengja þá í 0V. Sjáum líka að klukku-inngangurinn á JK vippunum er Active Low. Það kemur ekki að sök, þeir skipta þá á bakkantinum á klukkupúlsinum.

4.5 Greining samfasa teljara

Ef við fáum í hendurnar mynd af samfasa teljara hvernig getum séð hvernig hann telur?

Við þurfum að hugsa þetta vandamál sem öfuga hönnun þ.e. við rekjum okkur frá teikningunni af rásinni að forsendum hönnunarinnar þ.e. hvernig teljarinn skal telja. Tökum dæmi:



I. Örvunarjöfnur:

a) $D_0 = \overline{Q_0}$ b) $D1 = Q_0 \oplus Q_1$

II. Stöðubreytingajöfnur: **Útgangsjafna D vippu er: $Q^{n+1} = D$**

a) ${Q_0}^{n+1} = D_0 = \overline{Q_0} \Rightarrow {Q_0}^{n+1} = \overline{Q_0}$

b) ${Q_1}^{n+1} = D_1 = Q_0 \oplus Q_1 \Rightarrow {Q_1}^{n+1} = Q_0 \oplus Q_1$

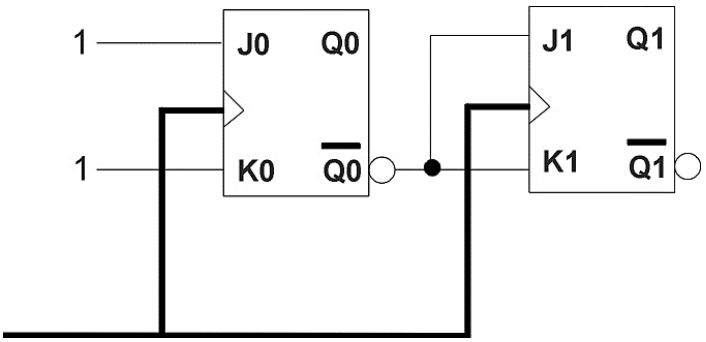
III. Stöðubreytingatafla:

a) ${Q_0}^{n+1} = \overline{{Q_0}^n}$ b) ${Q_1}^{n+1} = {Q_0}^n \oplus {Q_1}^n$

$Staða^n$	${Q_1}^n$	${Q_0}^n$	${Q_1}^{n+1} = {Q_0}^n \oplus {Q_1}^n$	${Q_0}^{n+1} = \overline{{Q_0}^n}$	$Staða^{n+1}$
S0	0	0	0	1	S1
S1	0	1	1	0	S2
S2	1	0	1	1	S3
S3	1	1	0	0	S0

Sjáum hér að talningin er S0 → S1 → S2 → S3 → S0 ….. eða 0,1,2,3,0……

Dæmi 2:
Greinið teljarann á myndinni.



Lausn:

Útgangsjafna vippu er: $Q^{n+1} = J \cdot \overline{Q^n} + \overline{K} \cdot Q^n$
$J_0 = 1, K_0 = 1, J_1 = \overline{Q_0}, K_1 = \overline{Q_0}$

$$Q_0{}^{n+1} = J_0 \cdot \overline{Q_0{}^n} + \overline{K_0} \cdot Q_0{}^n = 1 \cdot \overline{Q_0{}^n} + \overline{1} \cdot Q_0{}^n = \overline{Q_0{}^n}$$

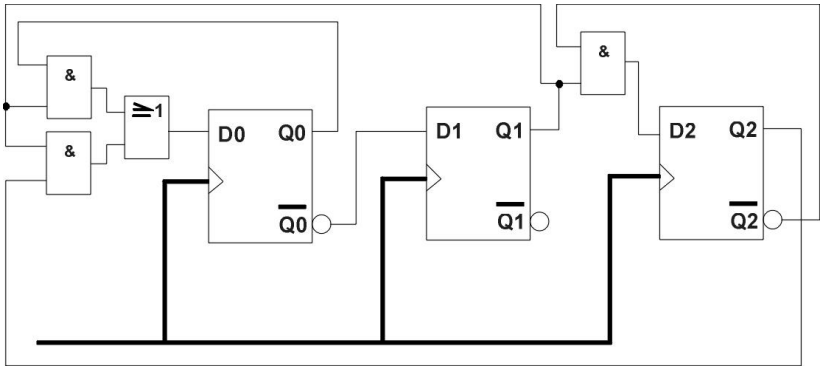
$Q_1{}^{n+1} = J_1 \cdot \overline{Q_1{}^n} + \overline{K_1} \cdot Q_1{}^n = \overline{Q_0{}^n} \cdot \overline{Q_1{}^n} + \overline{Q_0{}^n} \cdot Q_1{}^n = \overline{Q_0{}^n \oplus Q_1{}^n}$ (XNOR)

$\boldsymbol{Q_1{}^n}$	$\boldsymbol{Q_0{}^n}$	$\boldsymbol{Q_1{}^{n+1} = \overline{Q_1{}^n \oplus Q_1{}^n}}$	$\boldsymbol{Q_0{}^{n+1} = \overline{Q_0{}^n}}$
0	0	1	1
0	1	0	0
1	0	0	1
1	1	1	0

S0 → S3, S1 → S0, S2 → S1, S3 → S2, → 0,3,2,1,0,3 …..

Verkefni

Greinið teljarann á myndinni.



D0 =

D1 =

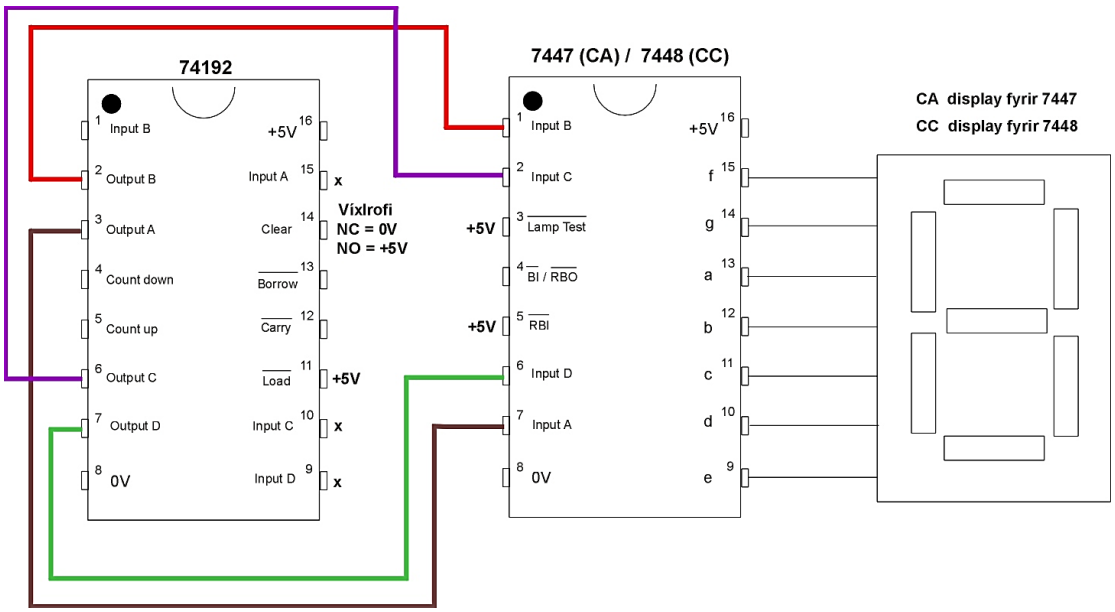
D2 =

5. Notkun teljara

5.1 74192 BCD teljari

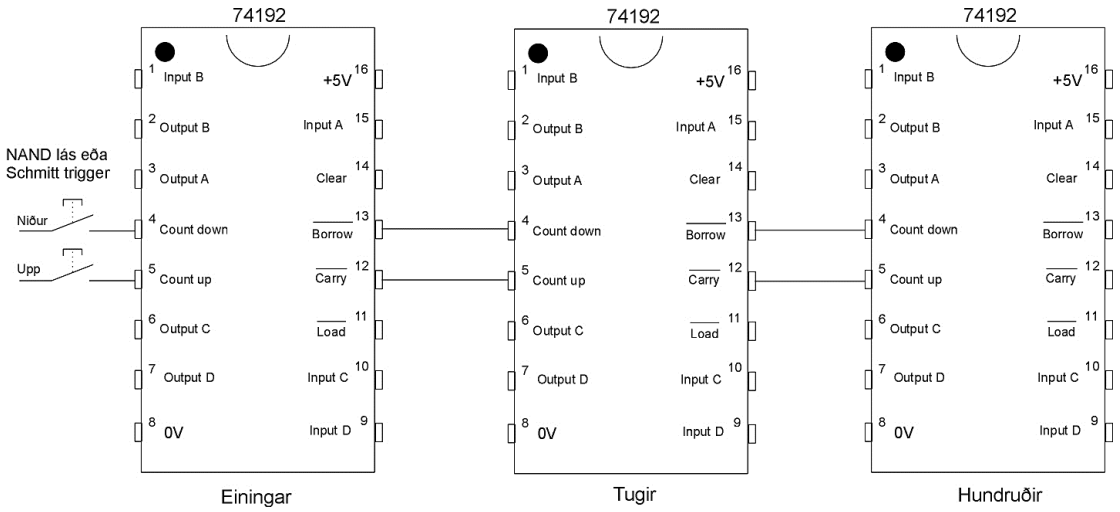
74192 er tilbúinn MOD10 eða BCD (0 – 9) teljari. Hann getur talið bæði upp og niður og auðvelt er að tengja nokkra saman ef telja á stærri tölur. Við þurfum einn fyrir 0 – 9, tvo fyrir 0 – 99, þrjá fyrir 0 – 999 o.s.frv. Hann hefur fjóra gagnainnganga (Input A til Input D) til þess að setja inn fyrirfram ákveðna stöðu (preset). Hún er lesin inn með því að setja 0V inn á $\overline{LOAD}$ innganginn (active low). Ef við ætlum ekki að nota fyrirfram gefnar stöður þá festum við $\overline{LOAD}$ innganginn í +5V og gerum hann þannig óvirkan.

Teljarinn er endursettur með því að setja +5V inn á Clear innganginn (active high). Klukkuinngangarnir eru tveir, Count down (niðurtalning) og Count up (upptalning). $\overline{Borrow}$ og $\overline{Carry}$ útgangarnir stýra klukkuinngöngunum á næsta teljara á eftir til að lækka eða hækka gildi hans. Útgangarnir Output A til Output D eru síðan teknir inn á sjö ráka ákóðunarrás (7447 eða 7448) sem er síðan tengd inn á sjö ráka staf, (display).



Tenging á einum 74192 teljara við sjö ráka afkóðunarrás og sjö ráka staf. Stöðurnar á inngöngum teljarans eru x (don't care) því þær skipta ekki máli ef $\overline{LOAD}$ er í +5V.

Upp og niðurteljari sem telur 0 – 999 eða 999 – 0.



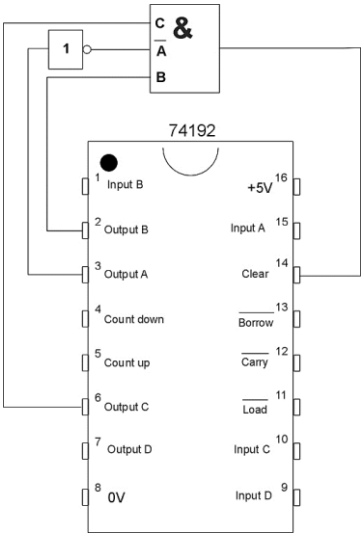
Sjáum hvernig $\overline{Carry}$ og $\overline{Borrow}$ stýra klukkuinngöngum næsta teljara á eftir. Þegar fyrsti teljarinn skiptir úr 9 í 0 þá sendir $\overline{Carry}$ útgangurinn púls inn á næsta teljara og hækkar hann um 1 (hækkun á tug). Þegar teljari nr.2 skiptir frá 9 – 0 þá sendir hann púls inn á þriðja teljarann og hækkar hann um 1, (hækkun á hundraði).

5.2 MOD6 teljari búinn til úr 74192 BCD teljara

Hægt er að búa til MOD6 teljara (0 – 5) úr 74192 BCD teljara. Hann endursetur sig (Clear) á tölunni CBA = 110 sem er 6 í tugakerfi. Hann telur því 0-1-2-3-4-5-0-1-2-3-4…

A útgangurinn er 0 og þarf því að fara í gegnum NOT hlið til að fá stöðuna 111 inn á AND hliðið. Þá gefur AND hliðið 1 inn á Clear innganginn og endursetur teljarann í 0 stöðu. AND hliðið gefur alltaf út 0 nema í stöðunni 110 þá gefur það út 1.

MOD6 teljarinn ásamt BCD teljaranum er nauðsynlegur í hönnun á klukku þar sem sekúndur og mínútur endursetja sig í stöðunni 59 þ.e. þær fara úr 59 í 00.



Það er hægt að búa til hvaða MOD teljara sem er frá 1 til 8 með þessum teljara. Stöðurnar sem eiga að hreinsa teljarann eru settar inn á AND hlið sem gefur +5V inn á Clear við rétta stöðu. ATH: Teljari sem telur 0 – 7 endursetur teljarann í stöðunni 8 = 1000. Það er alltaf staðan sem er einum hærri en talan sem við viljum enda lotuna á. Þess vegna heitir teljari sem telur 0 – 9 MOD10 og teljari sem telur 0 – 5 MOD6. MOD10 hefur 10 stöður 0 – 9 og MOD6 hefur 6 stöður 0 – 5.

MOD10 er líka kallað BCD (Binary Coded Decimal) því hann sýnir bara tugakerfistölur.

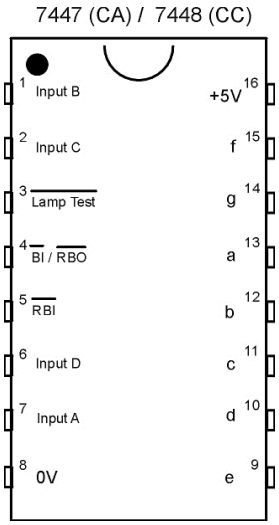
Líka er til teljari 74193 sem er MOD16 eða HEX teljari og hefur 16 stöður.

5.3 7447 og 7448 sjö ráka afkóðunarrás
Seven Segment Decoder CA og CC.

Seven Segment Decoder rásirnar breyta fjögurra bita innmerki í sjö útganga fyrir sjö ráka tölu. Inngangarnir heita Input A (LSB), Input B, Input C og Input D (MSB)

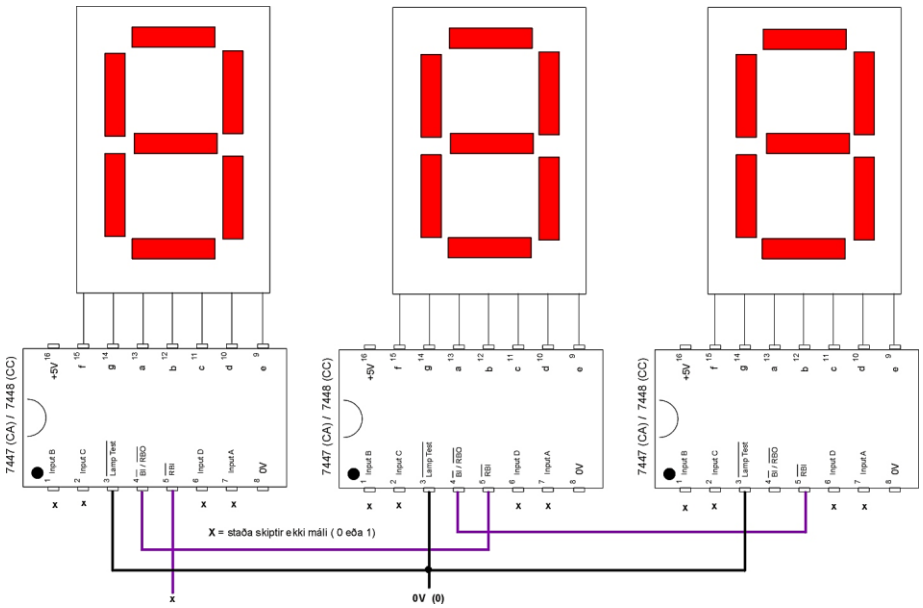
Afkóðunarrásirnar sýna tölustafina 0 til 9 fyrir inngangana DCBA = 0000 til 1001. Það er síðan misjafnt eftir gerðum afkóðunarrása hvað þær sýna fyrir inngangana DCBA = 1010 – 1111 eða 10 – 15. Sumar rásir eins og t.d. 7447 og 7448 sýna einhver tákn fyrir 10 – 14 og staðan 1111 = 15 slekkur á stafnum.

Aðrar afkóðunarrásir sýna HEX tölur (sextándukerfi) fyrir ofan 9 (A,b,c,d.E.F).



Lamp Test ($\overline{LT}$) inngangurinn (Stafprófun)

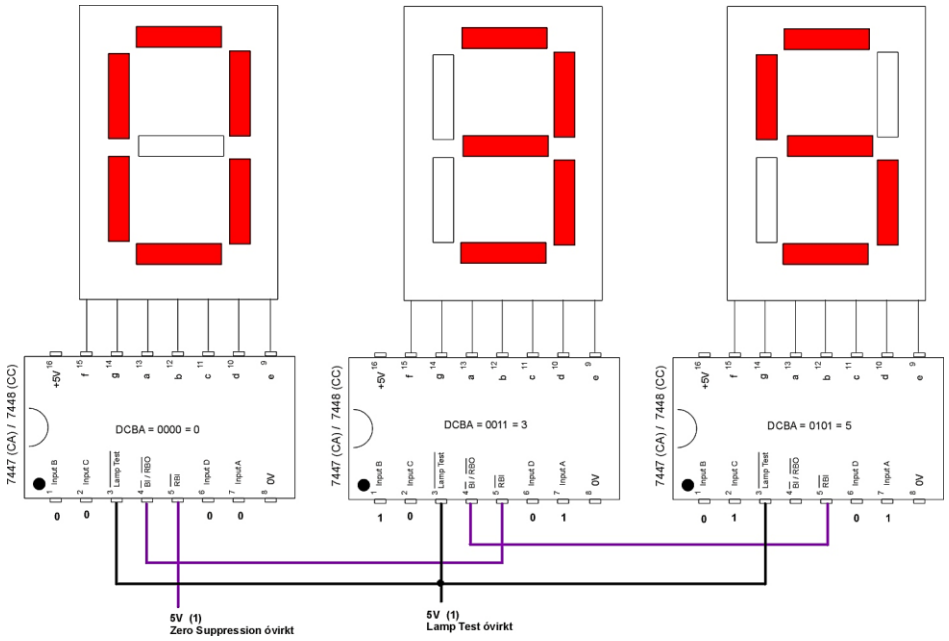
Lamp Test inngangurinn er active Low ($\overline{LT}$) og verður því virkur í lágstöðu. Sé hann settur í lágstöðu þá kveikir hann á öllum útgöngum afkóðunarrásarinnar og ef tölustafurinn 8 birtist þá er í lagi með stafinn. Sé ekki verið að nota Lamp Test verður $\overline{LT}$ inngangurinn að vera fastur í 5V (1). Þá er hann óvirkur eða hlutlaus.



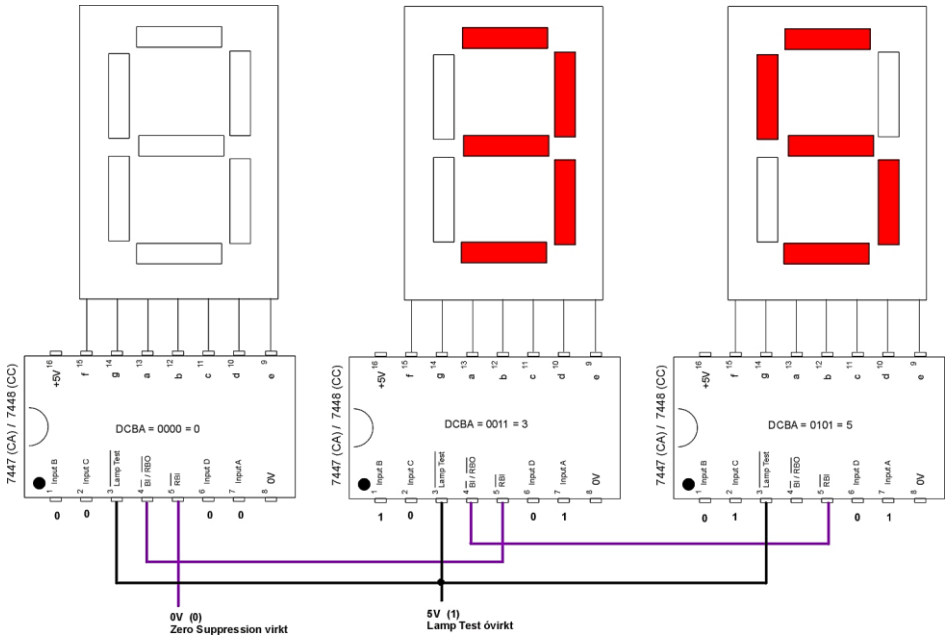
Notkun á **Lamp Test.** Stöður sem skiptir ekki máli hvort eru 0 eða 1 (don’t care) eru merktar með x. Setjum 0V inn á $\overline{LT}$ innganginn á öllum rásunum.

RBI, RBO inngangarnir og Zero suppression (Núll þvingun)

Þegar sýna á tölustaf eins og t.d. 35 á þriggja stafa skjá þá myndu þeir að öllu óbreyttu sýna 035.

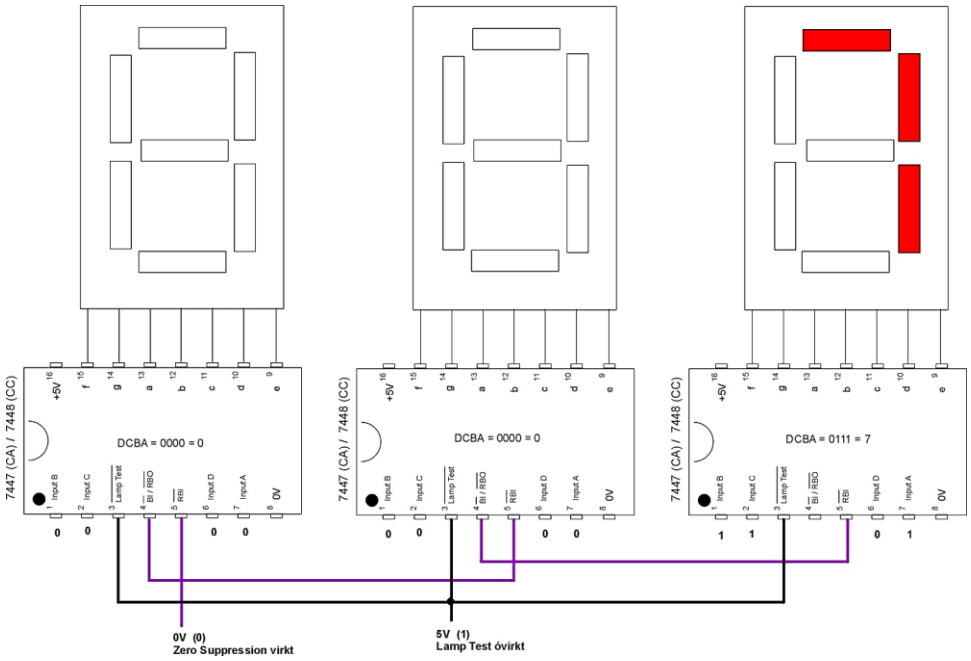


Til þess að sleppa því að sýna óþarfa 0 t.d. hundruð í tugtölu eða hundruð og tugi í einingatölu notum við Zero Suppression.



Reglan er sú að ef $\overline{BI}$ = 0 og DCBA = 0000 þá slekkur afkóðunarrásin á útgöngum sínum og þá slokknar á stafnum. Hún sendir um leið stöðuna 0 frá $\overline{RBO}$ útganginum inn á $\overline{BI}$ á næsta afkóðunarrás fyrir neðan í röðinni.

Tökum annað dæmi og skrifum út stafinn 7 (007) með Zero Suppression.

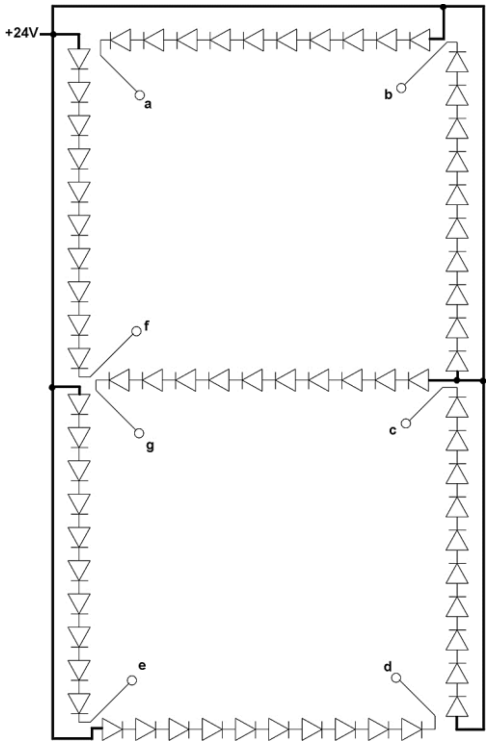


Hér fær afkóðunarrásin fyrir hundrað fasta 0V spennu inn á $\overline{BI}$ innganginn og Zero Suppression er því virkt á henni. Þegar gagnainngangar hennar verða DCBA = 0000 þá slekkur hún á útgöngunum a til f og sendir 0V frá $\overline{RBO}$ útganginum inn á $\overline{BI}$ innganginn á afkóðunarrásina fyrir tugi. Þegar gagnainngangar tugarins verða líka DCBA = 0000 þá slekkur afkóðunarrásin fyrir tugina á útgöngunum a til f og sendir 0V inn á $\overline{BI}$ innganginn á afkóðunarrásinni fyrir einingar. En þar sem gagnainngangar afkóðunarrásarinnar fyrir einingar er DCBA = 0111 en ekki 0000 þá birtir hún töluna 7.

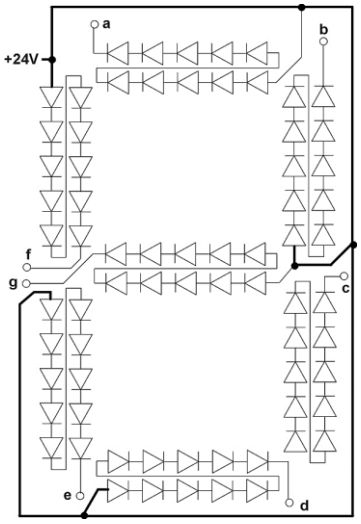
5.4 Seven segment display búið til úr LED díóðum

Stundum þarf stærri stafi en hægt er að kaupa tilbúna. Þá er hægt að smíða sína eigin sjö ráka stafi úr ljósdíóðum.

Útgáfa 1.
10mm LED 2,4V, 50mA
Einföld röð, 10 ljósdíóður í röð.
Samtals 70 ljósdíóður.
Tengt í Common Anode mode (Sameiginlegur +).

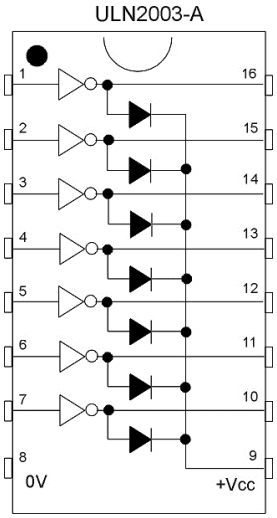


Útgáfa 2.
10mm LED 2,4V, 50mA
Tvöföld röð, 5 ljósdíóður í röð.
Samtals 70 ljósdíóður.
Tengt í Common Anode mode (Sameiginlegur +).

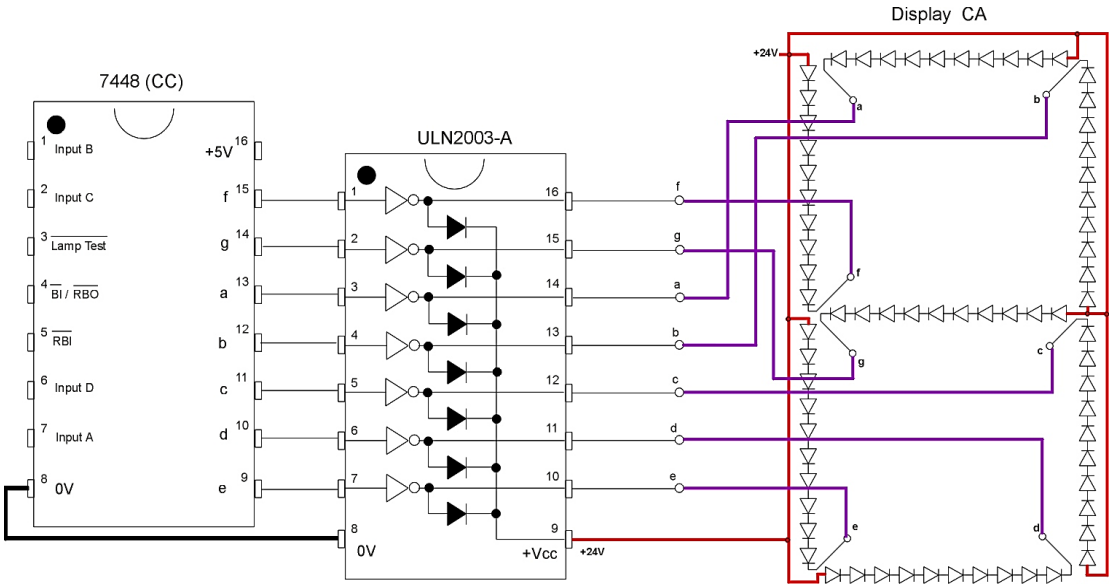


Það er ljóst að 7448 rásin keyrir ekki svona stóra stafi beint og því þarf að setja Darlington driver ULN2003 á milli.

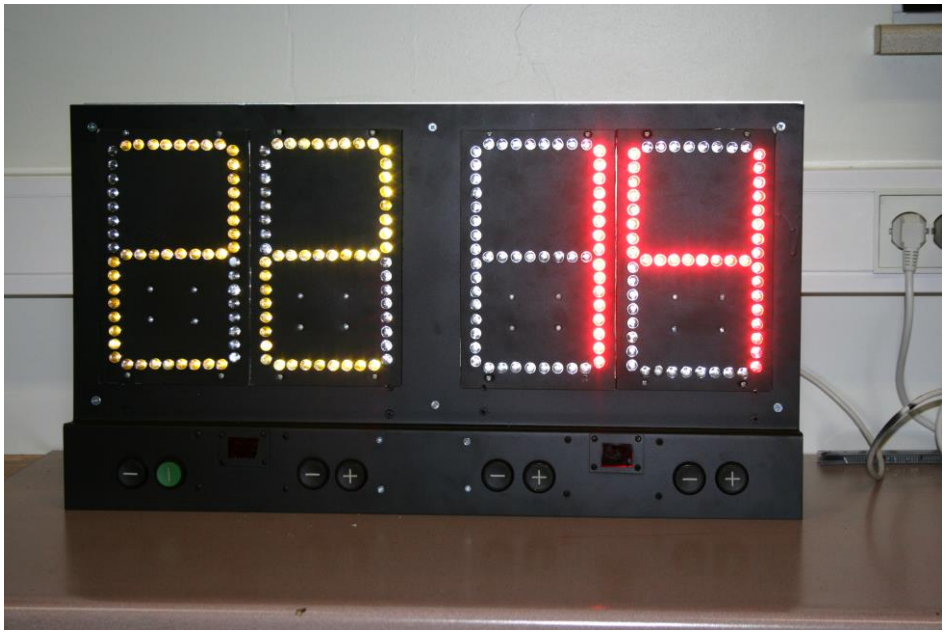
5.5 Darlington ULN2003A



Þótt stóru ljósstafirnir séu tengdir í CA mode þá þarf samt 7448 (CC) rásina til að kóða stafina. Það er vegna þess að ULN2003-A rásin er með viðsnúnum (inverted) útgangi og 7448 og ULN2003-A vinna því saman sem 7447 rás með margfalda aflgetu.



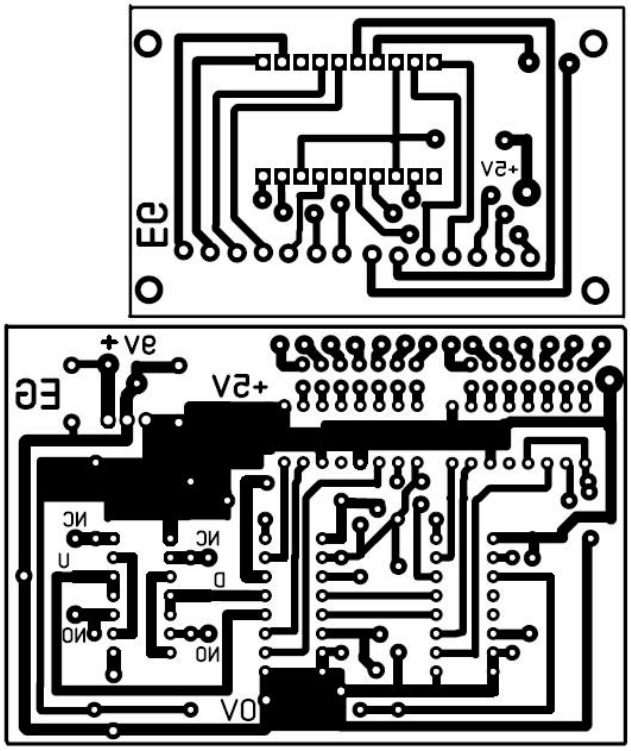
$+V_{CC}$ inngangurinn á ULN2003 rásinni þarf að fá sömu + spennu og stafurinn fær eða +24V. ULN2003 rásin tengir síðan hverja rák (segment) við 0V og þannig kviknar á rákinni.



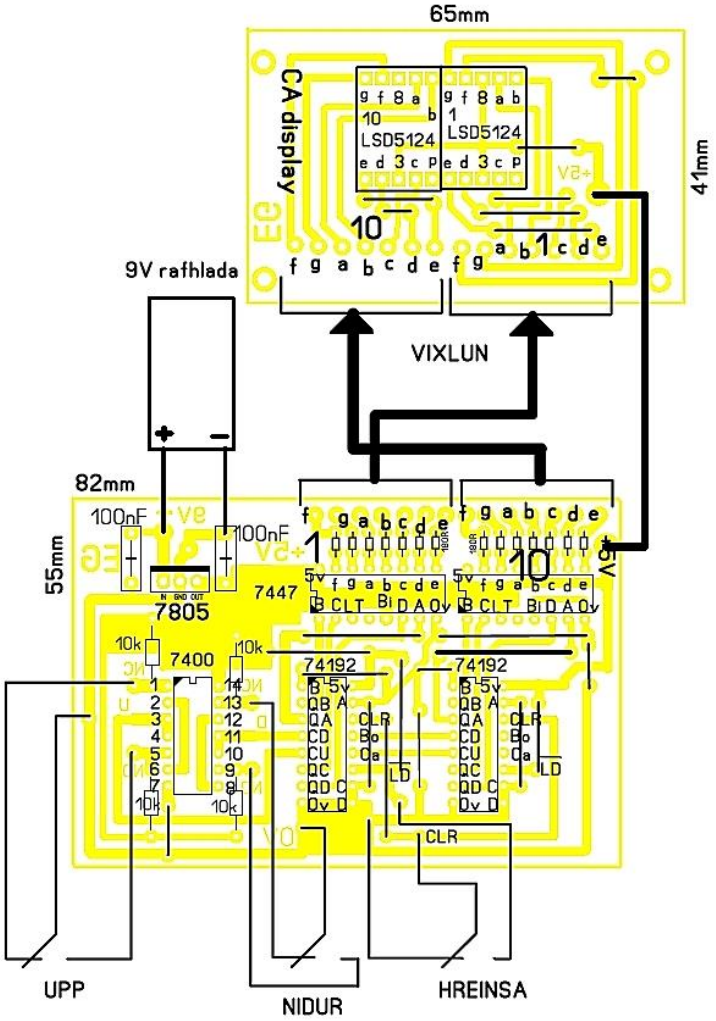
Innáskiptispjald fyrir fótbolta. 8 stk. 10mm LED í hverri rák.

Verkefni. Teljari 0 - 99

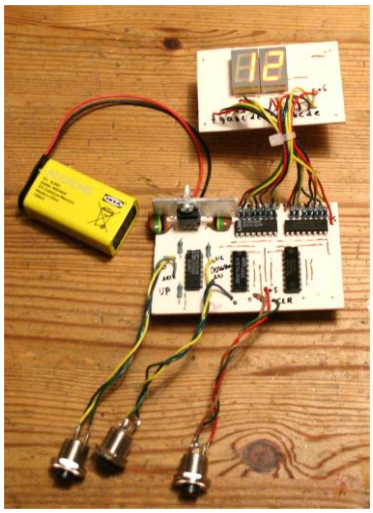
Hér er prentplata fyrir teljara sem telur 0 – 99 bæði upp og niður.



Prent fyrir teljararás. Stærri platan er 82mm x 55mm og sú minni 65mm x 41mm.



Tengimynd fyrir teljarann.

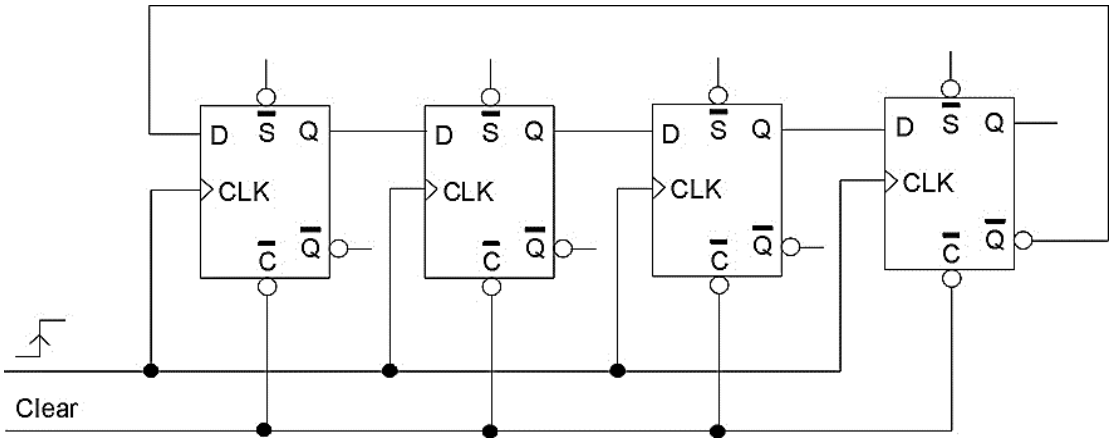


Teljari 0-99 tilbúinn. Þessa rás er síðan hægt að setja í lítinn kassa. Jafnvel má nota tvo saman og þá er komið stigaborð fyrir spurningakeppni ☺

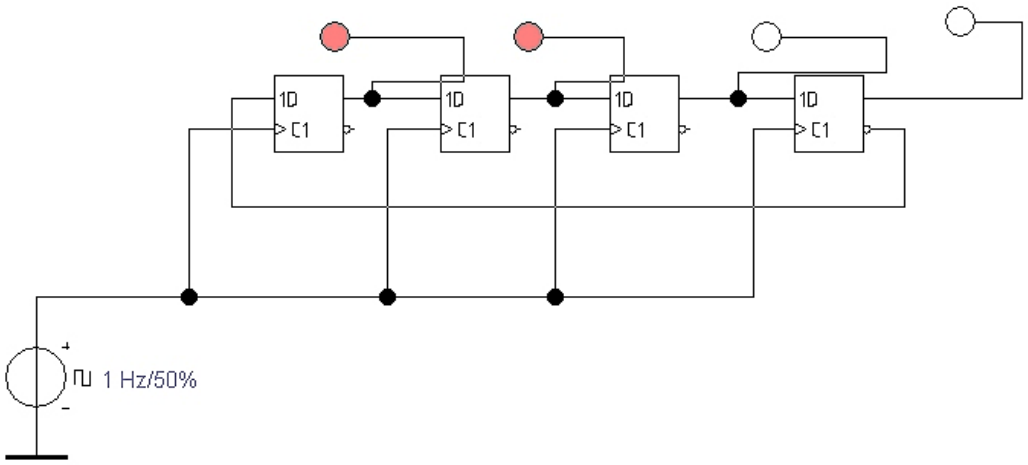
5.6 Sérstakir teljarar

5.6.1 Johnson teljari

4 bita Johnson teljari telur:
0000 – 1000 – 1100 – 1110 – 1111 – 0111- 0011 – 0001 – 0000… Hann er búinn til með fjórum D vippum sem tengdar eru saman í halarófu en samfasa.



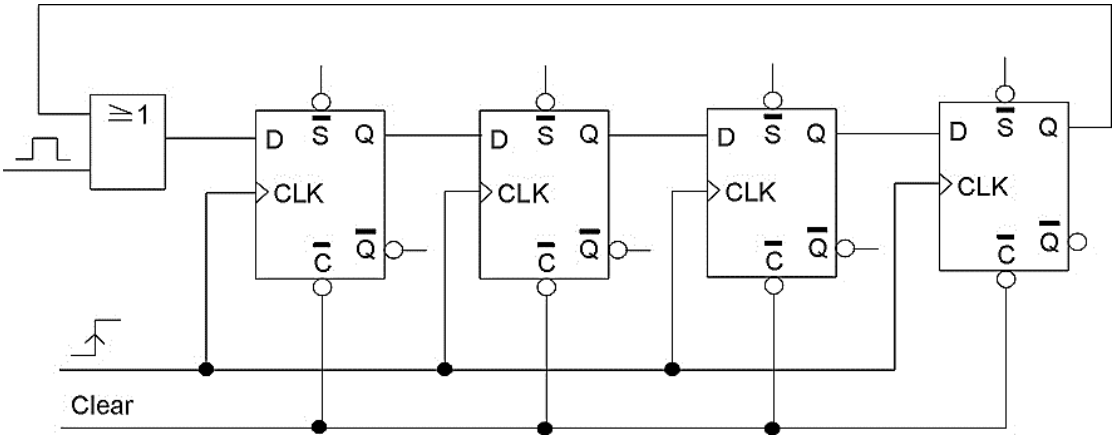
Johnson teljari tengimynd



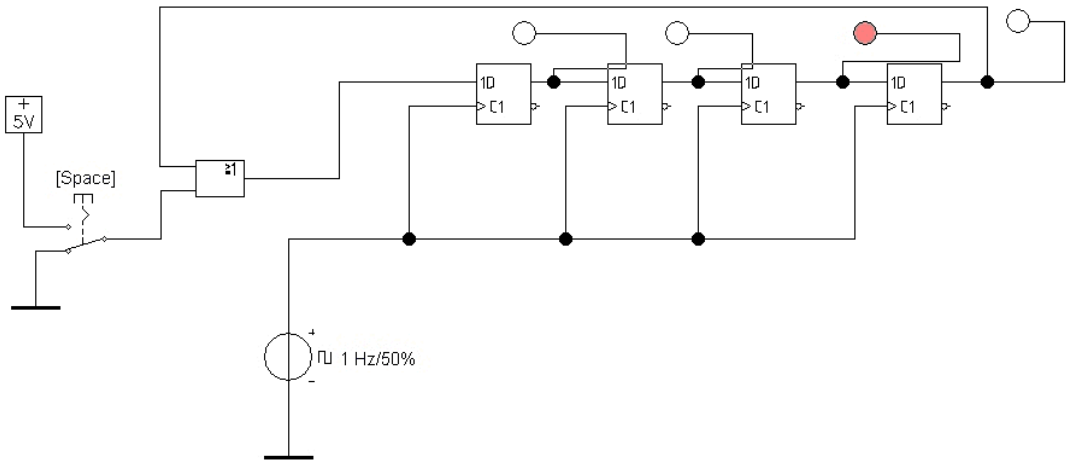
Johnson teljari úr hermiforriti

5.6.2 Overbeck teljari

4 bita Overbeck teljari telur 1000 – 0100 – 0010 – 0001 – 1000 …. Það er alltaf bara einn útgangur með stöðuna 1 í einu. Hann þarf þó að fá einn púls inn á D inngang fyrstu vippu til að fara í gang.



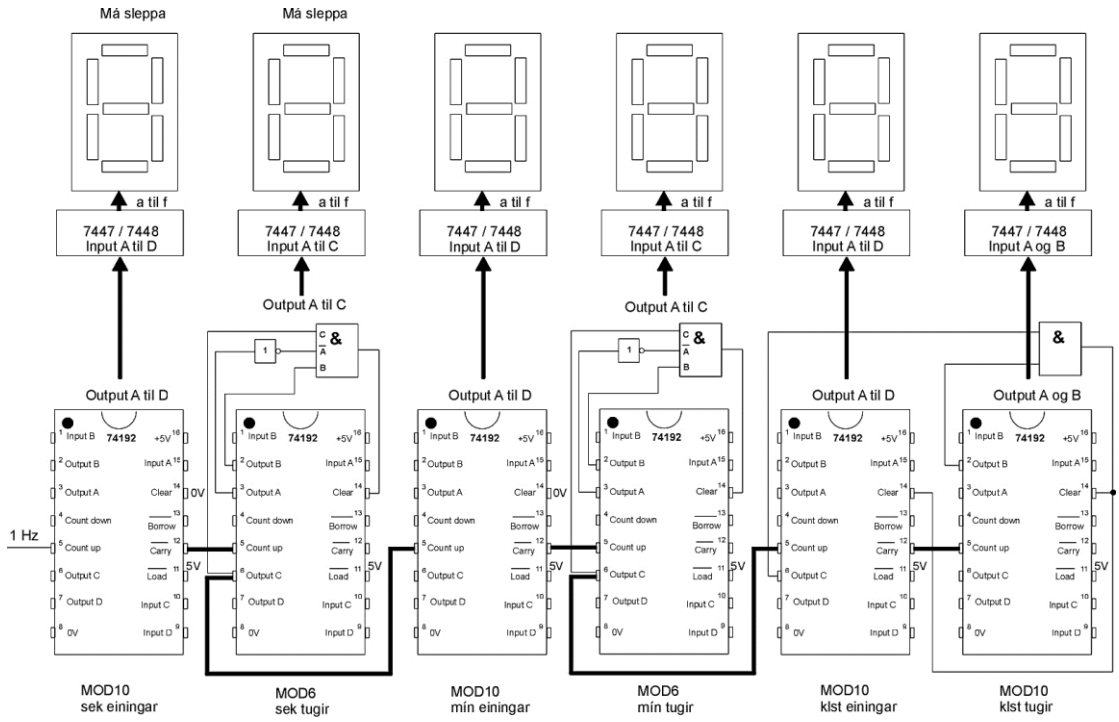
Overbeck teljari tengimynd



Overbeck teljari úr hermiforriti

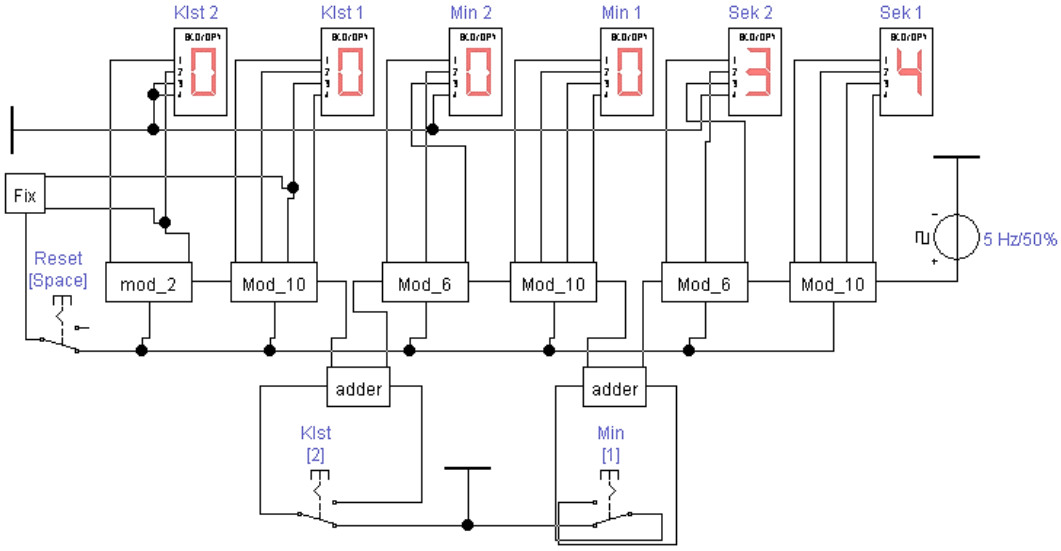
5.6.3 Klukka

Til að búa til klukku þurfum við þrjá MOD10 teljara, tvo MOD6 og einn MOD2 teljara. Klukkan á að endursetja sig eftir stöðuna 23:59:59 og fara í 00:00:00. Við þurfum því að skoða sérstaklega MOD2 og MOD10 teljarann á klukkustundunum. Þeir endursetja sig þegar klukkan reynir að fara í 24:00:00. Það er því staðan 24 á klukkustundunum sem við notum til að endursetja þær. Mínútur og sekúndur endursetja sig sjálfar. 24 = 0010 0100. Getum því notað þá bita sem eru 1 akkúrat í þessari stöðu. Það er Q1 á MOD2 teljaranum og Q2 á MOD10 teljaranum. Í sjálfu sér þarf ekki MOD2 teljara á klukkustundirnar. Hægt er að nota MOD10 því hann er hvort eð er endursettur í stöðunni 24:00:00.



Klukka búin til með 74192 BCD teljararásum. Raða þarf stöfunum í rétta röð.

Ef við skoðum klukkustundirnar sérstaklega þá sést að þær endursetja sig þegar 23 breytist í 24. Breytingin er frá 0010 0011 yfir í 0010 0100. Við getum því notað bitana sem eru 1 í stöðunni 24 til ð endursetja klukkustundirnar. Það er Output B á tugunum og Output C á einingunum. Tökum þessa útganga inn á AND hlið sem sendursetur síðan tvo öftustu teljarana og staðan fer úr 24:00:00 í 00:00:00.



24 stunda klukka úr hermiforriti. Hér er 74192 rásin ekki notuð heldur eru allir teljarar hannaðir sérstaklega með JK vippum. Þeim er síðan breytt í Sub Circuit í forritinu og gefið nafnið Mod_2, Mod_6 og Mod_10. Klukkuna má stilla á hærri tíðni en 1Hz til að flýta fyrir hermun.

6. Gistur (register)

Gistur eru rásir sem geta geymt eina tvíundartölu 8, 16, 32 eða 64 bita og eru því nokkurskonar minni. T.d. ef tölva leggur saman tvær tölur sem geymdar eru í vinnsluminni þá les hún þær fyrst inn í gistur og sækir þær þangað til þess að leggja þær saman í reiknirás. Gistur eru búin til úr vippum, yfirleitt D vippum. Gistur tapa innihaldi sínu við straumleysi og eru því óstöðugt (Volatile) minni.

Vippur geyma einn bita
Gistur geyma eina tvíundartölu af ýmsum stærðum
Minni geyma margar tvíundartölur

Til eru tvær megingerðir af gistrum:

1. **Samhliða gistur** (Parallell register) Talan er öll lesin inn í einum klukkupúls. Eingöngu notað á stuttum vegalengdum, þ.e. á móðurborði.
2. **Raðgistur** (Serial register) Talan er lesin inn bita fyrir bita í mörgum klukkupúlsum. Mest notað fyrir samskipti á löngum vegalengdum þ.e. á milli tölva.

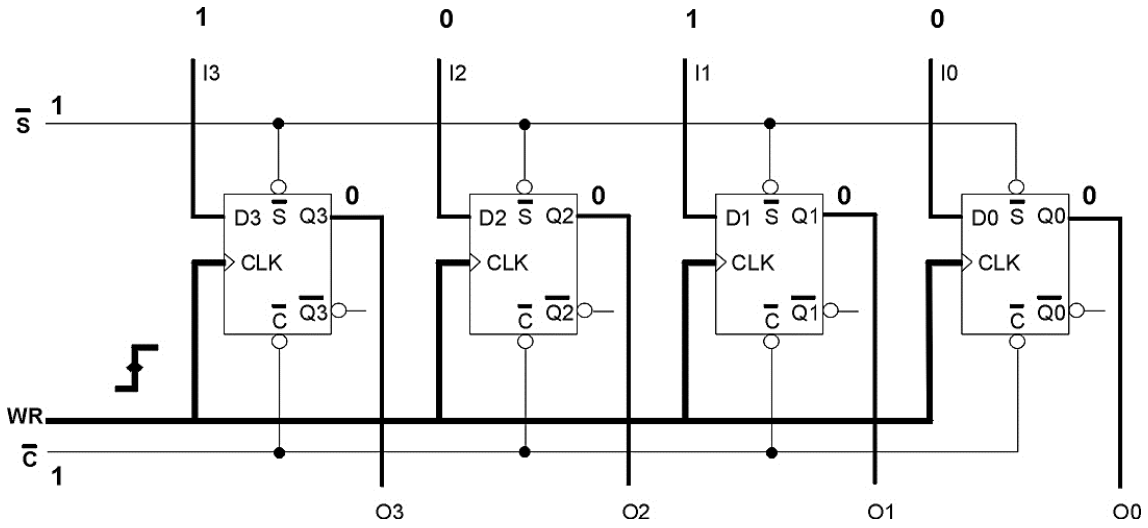
Síðan er til sambland af þessu tvennu þ.e:

Samhliða inn / Rað út og Rað inn / Samhliða út.

6.1 Samhliða inn / Samhliða út gistur

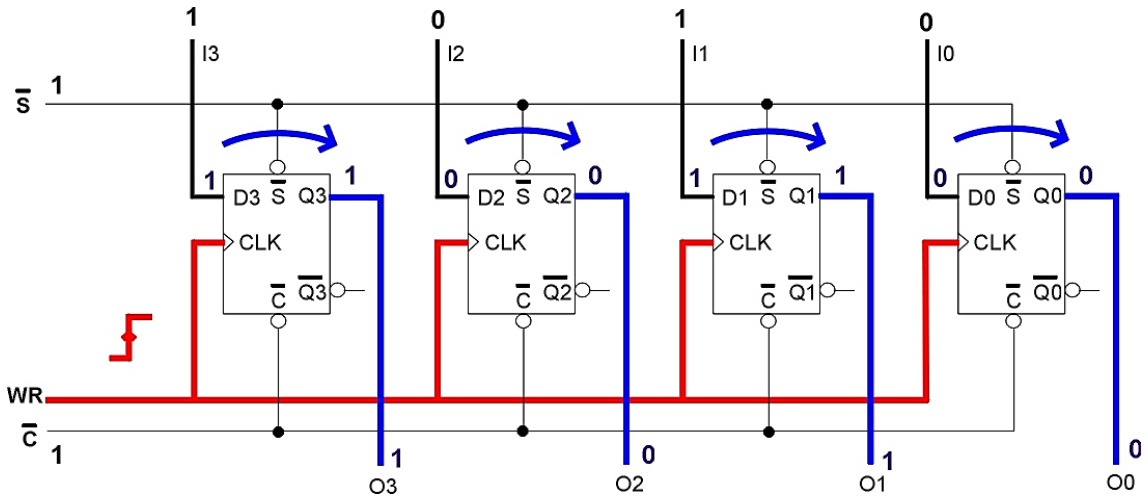
Skoðum fyrst samhliða inn / Samhliða út gistur: Miðum við 4 bita, hugsunin breytist ekkert þótt bætt sé við bitum (vippum).

Skoðum þessa rás:

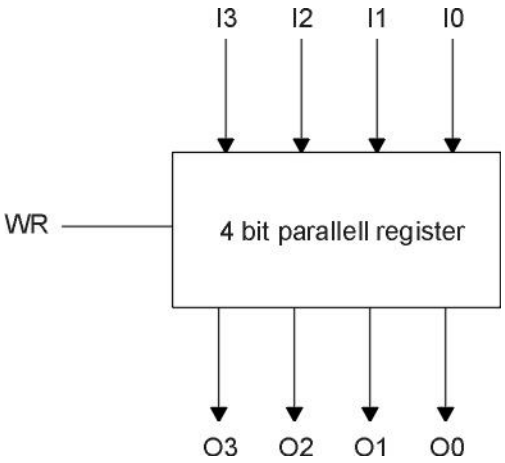


Uppbygging rásarinnar.

Munum að útgangsjafna D vippu er $Q^{n+1} = D$ þ.e. næsti útgangur Q er staðan sem bíður á D innganginum. Þegar WR (write) inngangurinn breytist úr 0 í 1 þá flyst staðan sem bíður á inngöngum I3 til I0 (1010) yfir á útgangana O3 til O1. Við höfum því lesið töluna 1010 inn í gistrið og þar verður hún þangað til næsti WR púls kemur og les inn nýja stöðu frá innganginum.



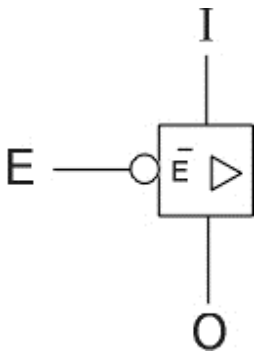
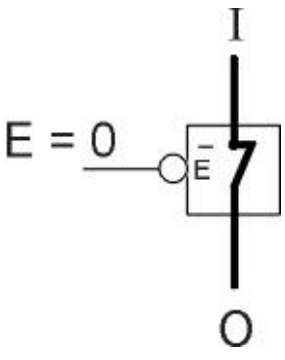
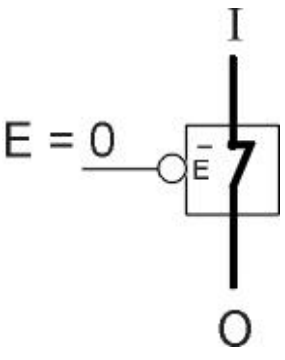
Við lesum því alla bitana inn í einum púlsi.

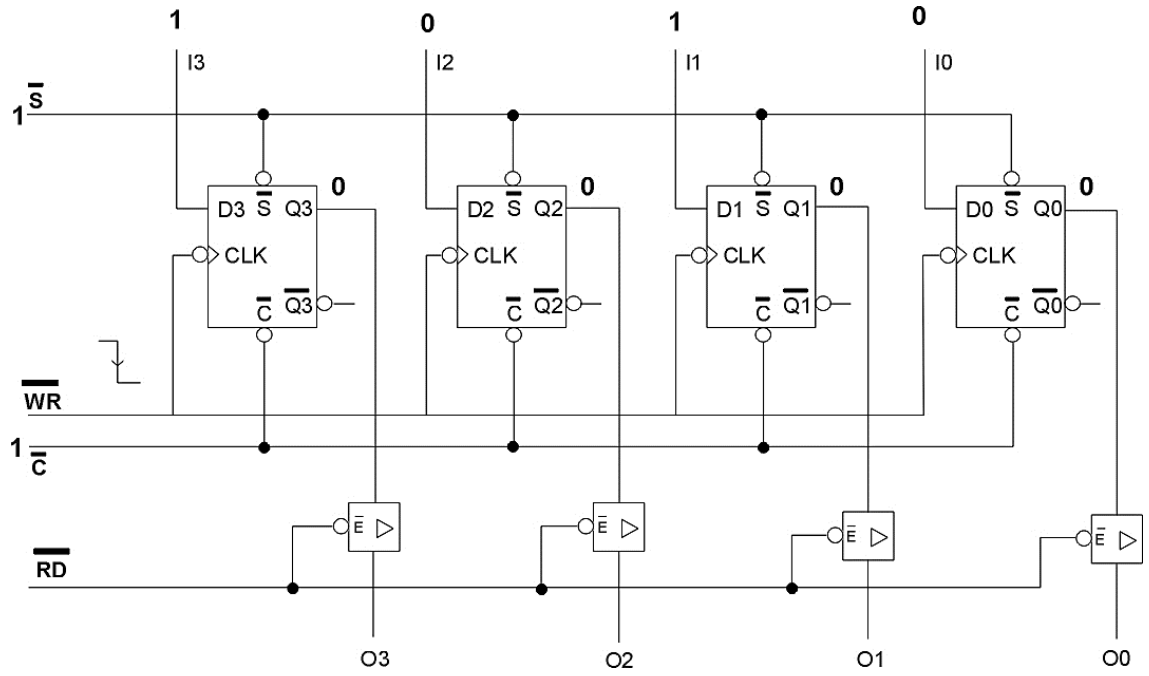


Tákn fyrir 4 bita Samhliða inn / Samhliða út gistur

6.2 Samsíða gistur með þristöðu útgangi (Tristate logic , buffer)

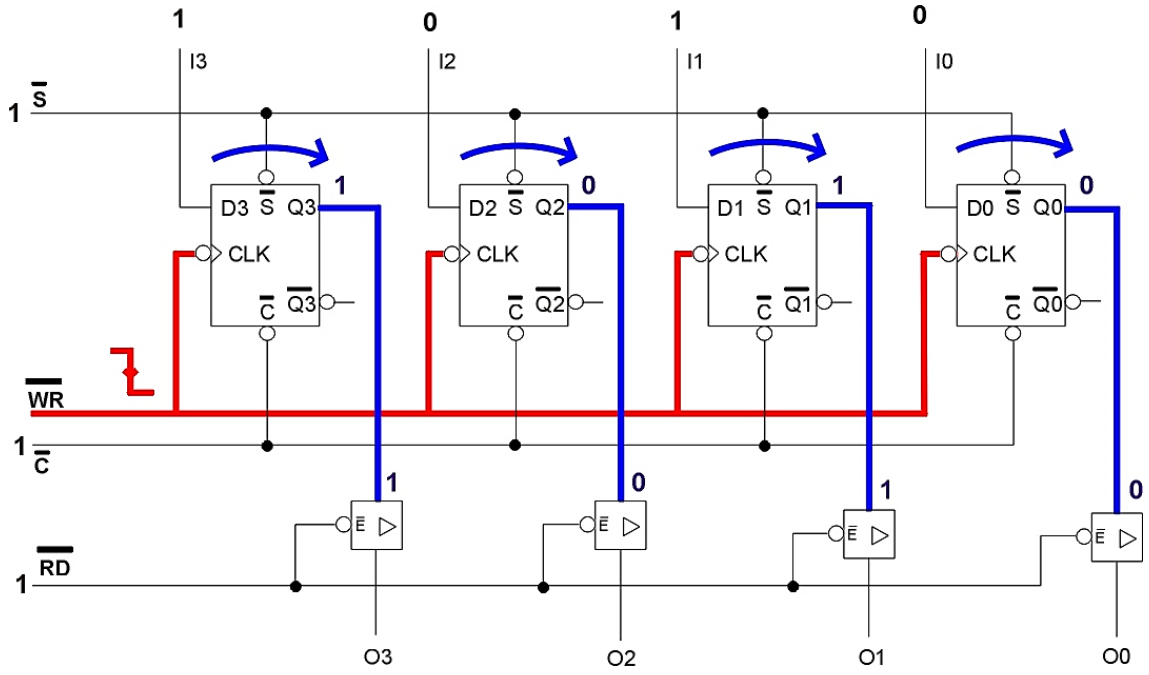
Stundum viljum við stjórna því hvenær innihald gisturs verður aðgengilegt á gagnabraut. Það er gert með því að nota svokallaða **Þristöðurás.** Hún virkar þannig að hún er með tvo innganga og einn útgang. Annar innganganna er stýriinngangur (Enable) og er ýmist active low eða high. Hann stýrir því hvort rásin leiðir eða ekki. Ef stýriinngangurinn er virkur þá leiðir rásin beint á milli inn og útgangs. Ef hann er óvirkur þá er útgangurinn aftengdur frá rásinni.

		
Tákn fyrir þristöðurás með active low Enable inngang.	$E = 0 \Rightarrow O = I$ $I = 1 \Rightarrow O = 1$ $I = 0 \Rightarrow O = 0$	$E = 1$ O=opin rás, aftengd frá gagnabraut

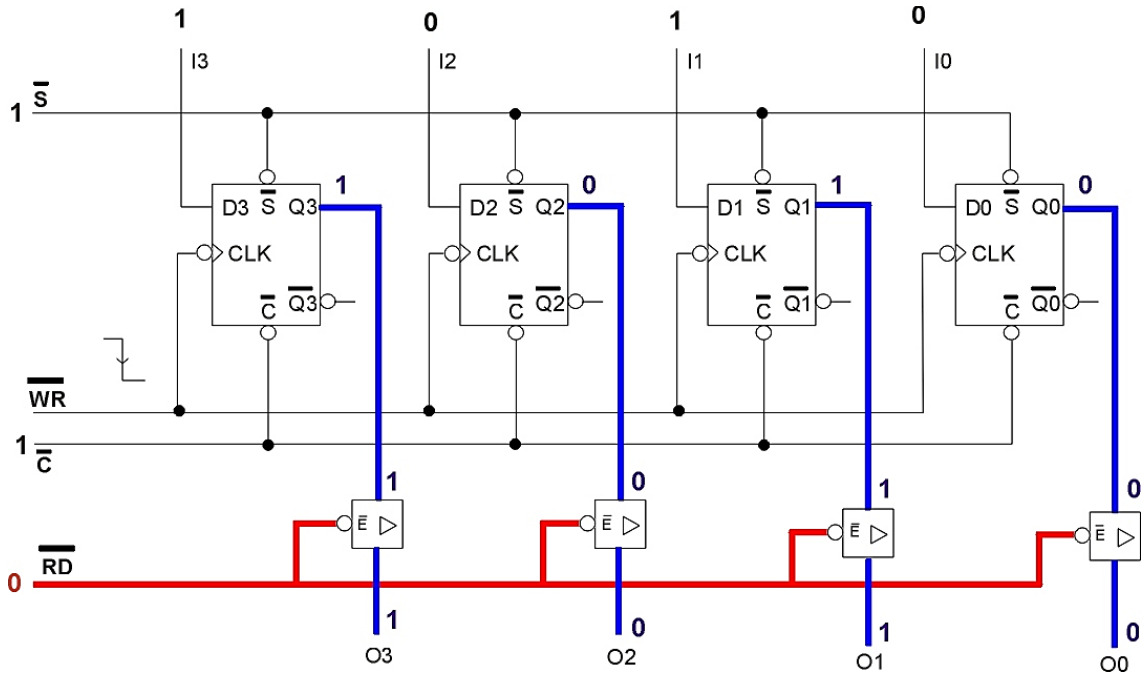


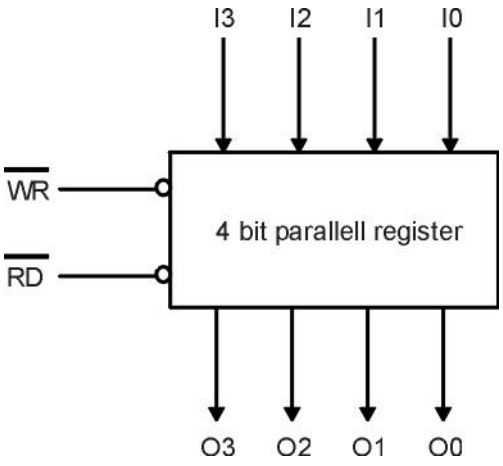
Uppbygging rásarinnar.

Við skrifum í gistrið með því að setja $\overline{WR}$ innganginn í lágstöðu (gefum einn klukkupúls). Eftir það er staðan 1010 komin á innganga þristöðurásarinnar.



Ef við viljum lesa innihaldið út þá er nóg að setja $\overline{RD}$ innganginn í lágstöðu og staðan 1010 birtist á O3 til O0 útgöngunum.

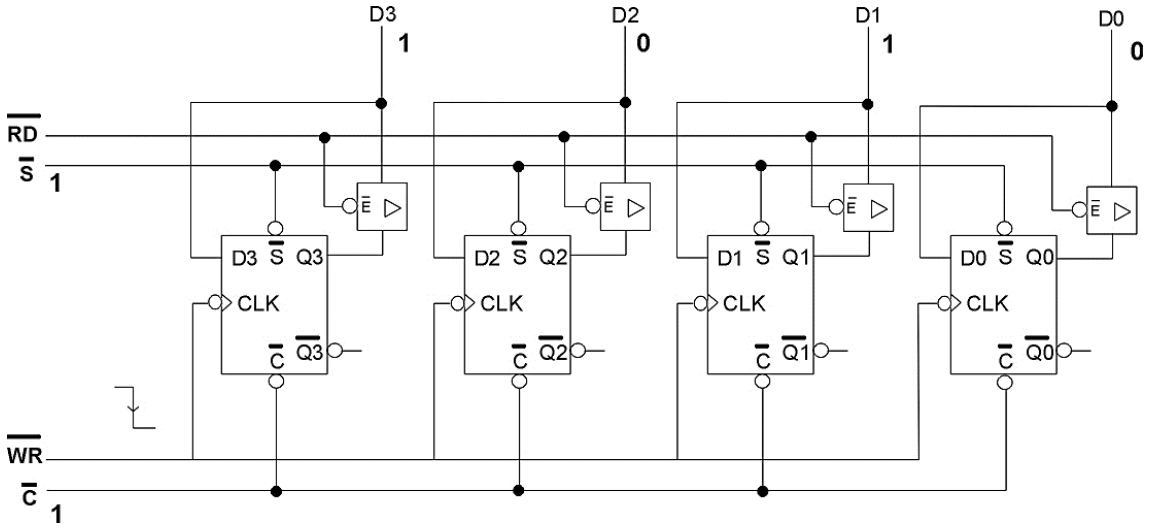




Tákn fyrir samhliða inn/út gistur með þrístöðurás á útgangi.

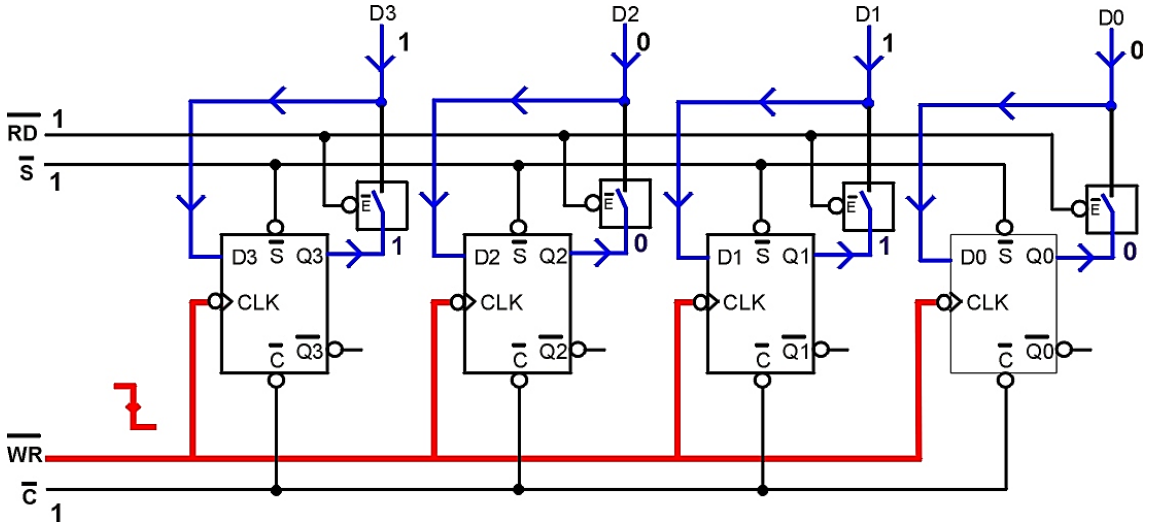
6.3 Tvístefnd gistur (Bidirectional registers)

Tvístefnd gistur hafa sameiginlega inn og útgangana. Þeir kallast D (data) í stað I og O áður og eru tengdir beint við gagnabrautina. Við notum stýriinnganga til þess að stjórna því hvort þeir virka sem inn eða útgangar.

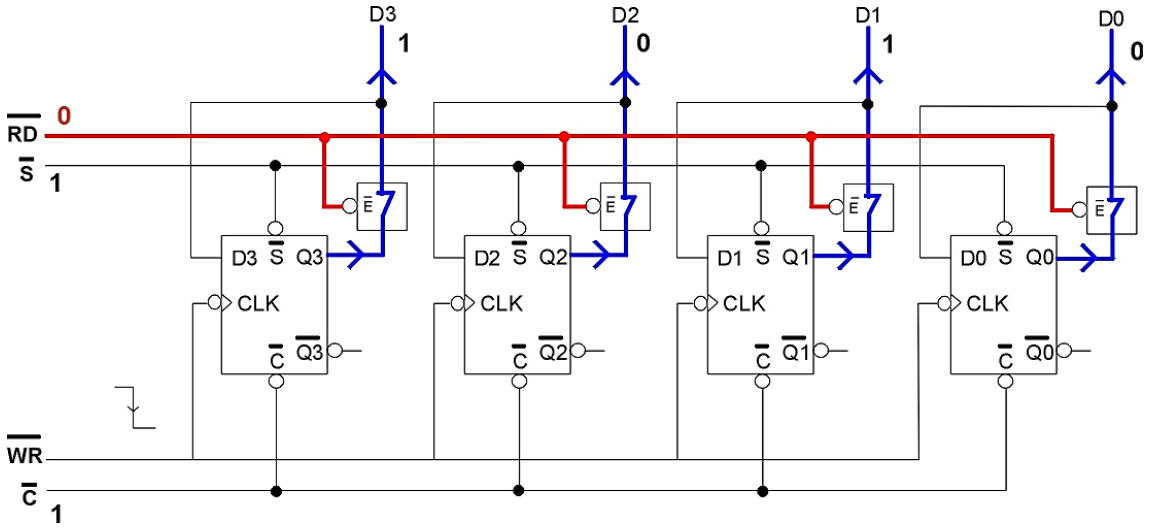


Uppbygging rásarinnar.

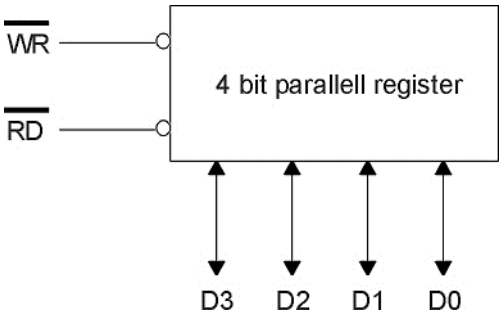
Til þess að lesa inn í gistrið er $\overline{WR}$ inngangurinn settur í lágstöðu í augnablik. Þá færast tölurnar af innganginum inn á útganga vippanna en stoppa við þrístöðurásina.



Tala lesin inn í gistrið. $\overline{WR}$ (write) línan er virk og D3 til D0 er lesið inn.

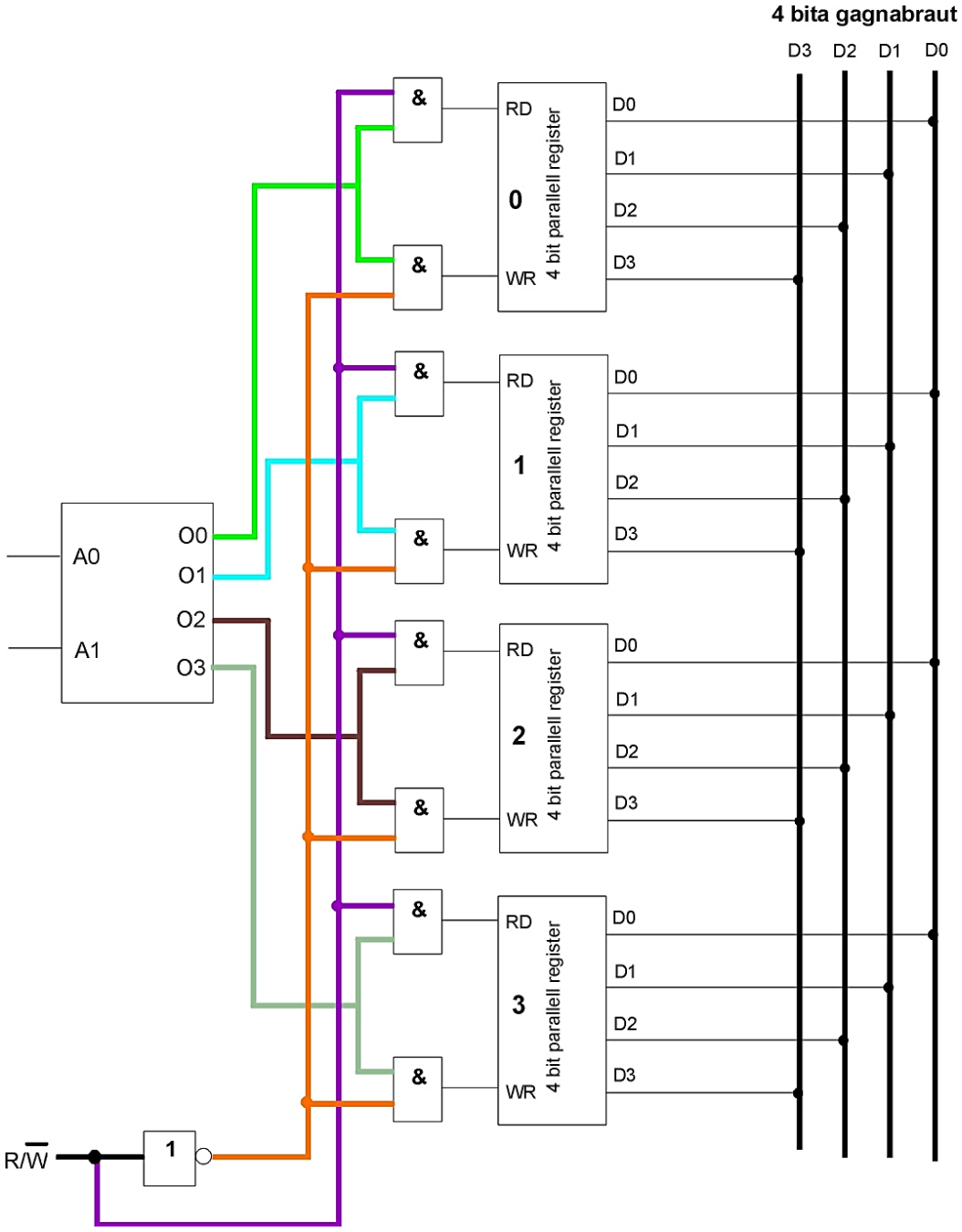


Til þess að lesa út er $\overline{RD}$ (read) inngangurinn settur í lágstöðu. Þristöðurásin verður þá virk og tölurnar verða núna aðgengilegar á útganginum.

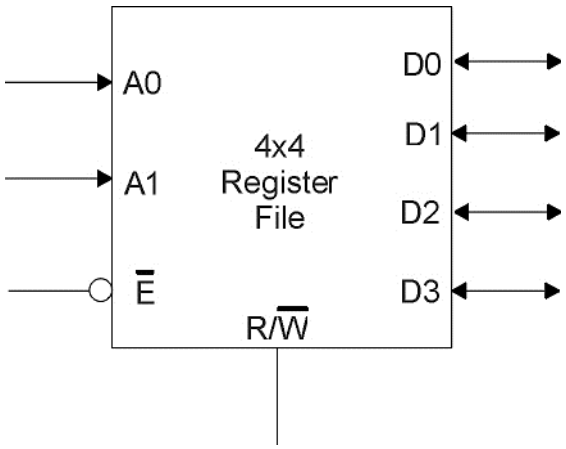


Tákn fyrir 4 bita tvístefnt samhliða gistur.

Oftast eru gistur sett upp sem blokkir og þá þarf að setja upp einhverja stýringu sem stjórnar því í hvaða gistur á að skrifa og úr hvaða gistri á að lesa.



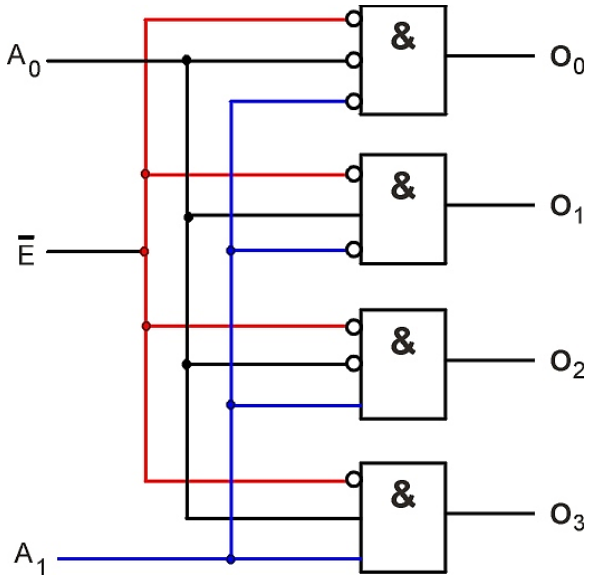
Hönnum stýringu sem gerir okkur kleift að koma öllum gistrunum í eina rás með 4 inngöngum og 4 útgöngum. $A0$, $A1$ stýra því hvaða gistur af fjórum er virkt. $\bar{E}$ stjórnar hvort gistrin sem heild eru virk eða óvirk og $R/\bar{W}$ inngangurinn ákvarðar hvort við skrifum inn gögn ($R/\bar{W} = 1$) eða lesum þau út ($R/\bar{W} = 0$).



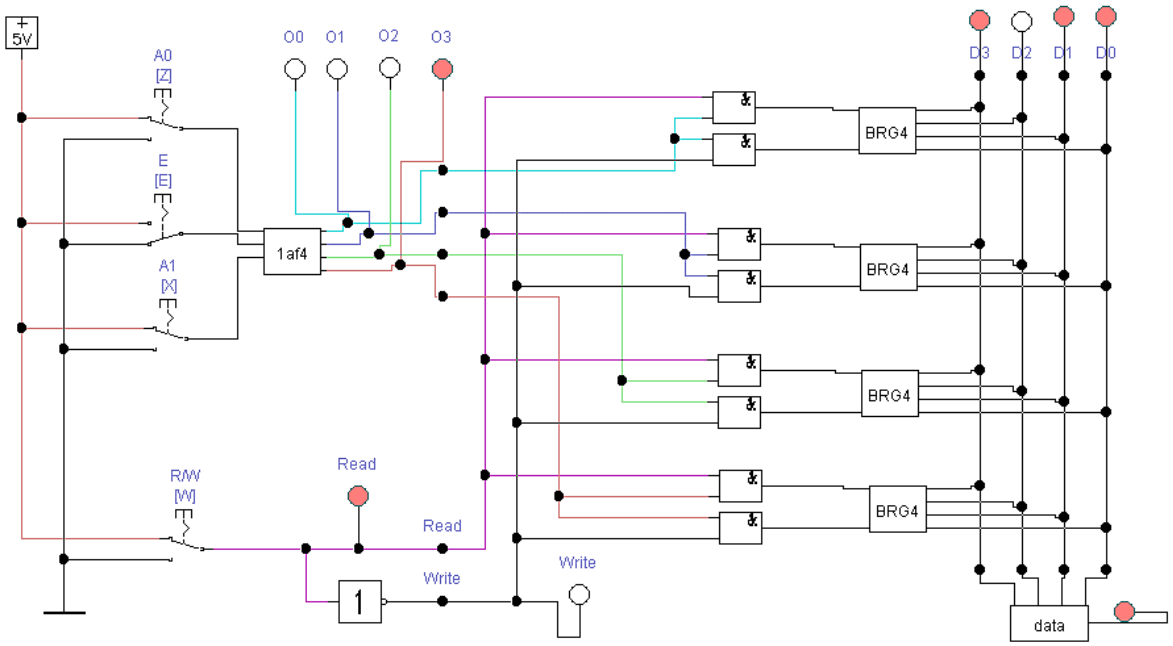
Hönnum rásina sem ákveður hvaða gistur er virkt. Sanntafla hennar lítur svona út:

Gistur	$\bar{\bar{E}}$	A_1	A_0	O_0	O_1	O_2	O_3
Gistur 0	0	0	0	1	0	0	0
Gistur 1	0	0	1	0	1	0	0
Gistur 2	0	1	0	0	0	1	0
Gistur 3	0	1	1	0	0	0	1
Ekkert	1	x	x	0	0	0	0

$O_0 = \bar{\bar{E}} \cdot \overline{A_1} \cdot \overline{A_0}$ $O_1 = \bar{\bar{E}} \cdot \overline{A_1} \cdot A_0$ $O_2 = \bar{\bar{E}} \cdot A_1 \cdot \overline{A_0}$ $O_3 = \bar{\bar{E}} \cdot A_1 \cdot A_0$



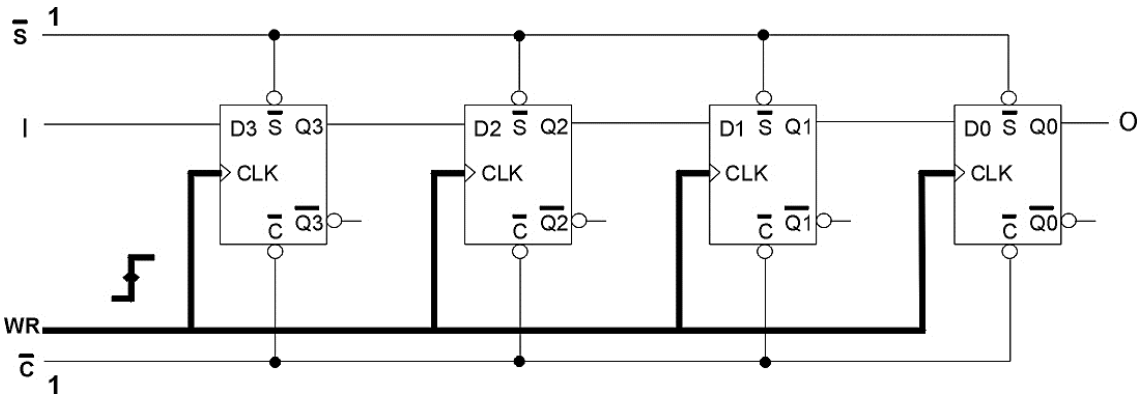
1 af 4 rás



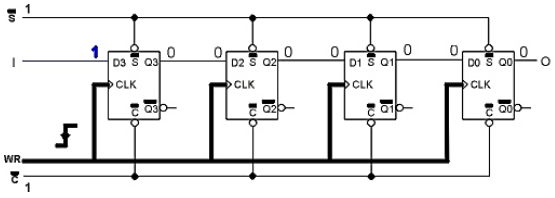
Uppsetning úr Electronic Workbench.

6.4 Rað inn / Rað út gistur

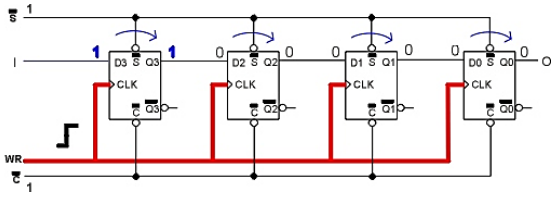
Skoðum þessa rás:



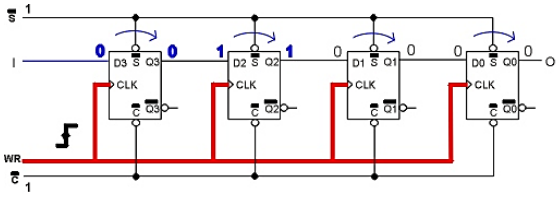
Sjáum að vippurnar eru raðtengdar þ.e. útgangur vippu nr. n tengist inngangi vippu nr. n-1. Skoðum hvað þarf til þess að koma stöðunni **1101** inn í svona gistur. Viljum enda með **1** á Q0, **0** á Q1, **1** á Q2 og **1** á Q3.



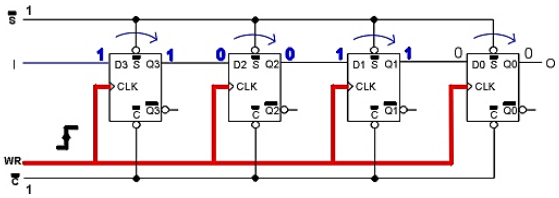
Upphafsstaða: **0000**



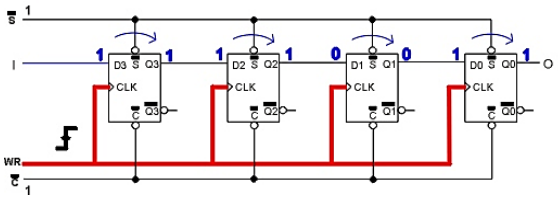
Staðan eftir 1 klukkupúls: **1000**



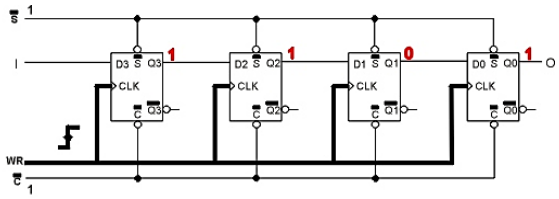
Staðan eftir 2 klukkupúlsa: **0100**



Staðan eftir 3 klukkupúlsa: **1010**



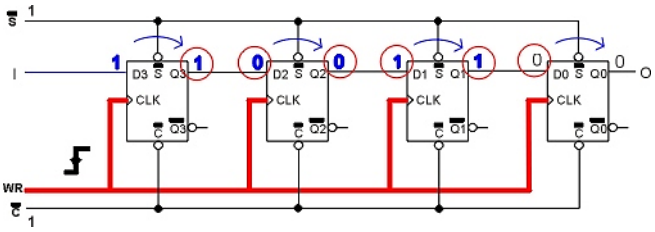
Staðan eftir 4 klukkupúlsa: **1101**



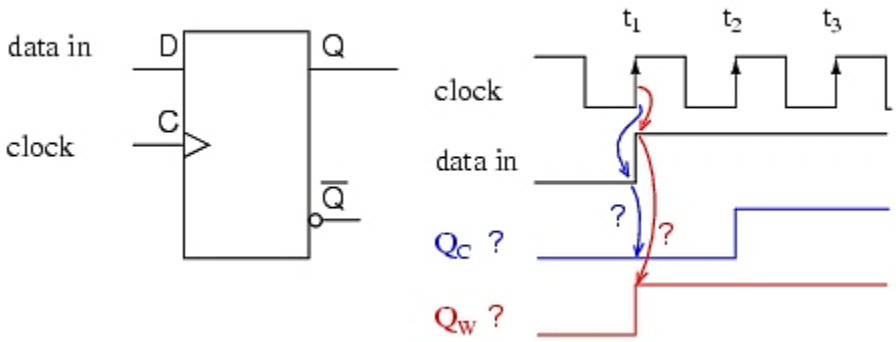
Lokastaða: **1101**

Það þurfti fjóra klukkupúlsa til þess að lesa inn 4 bita tölu í stað eins klukkupúls áður. Gistrið tekur inn nýja tölu með því að lesa inn bitana frá vinstri til hægri. (Shift Right Register).

En það er eitthvað vandamál sem þarf að skoða en það er sú staðreynd að útgangar eru tengdir við innganga. Kemur ekki upp sú staða að sama lína vill vera bæði 0 og 1 á sama tíma?

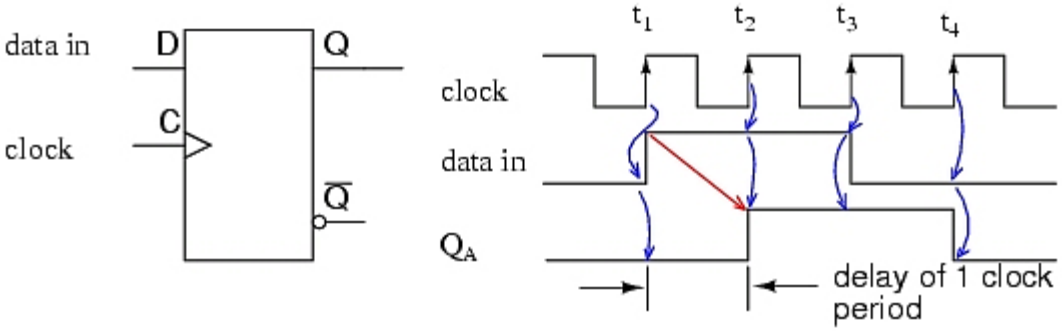


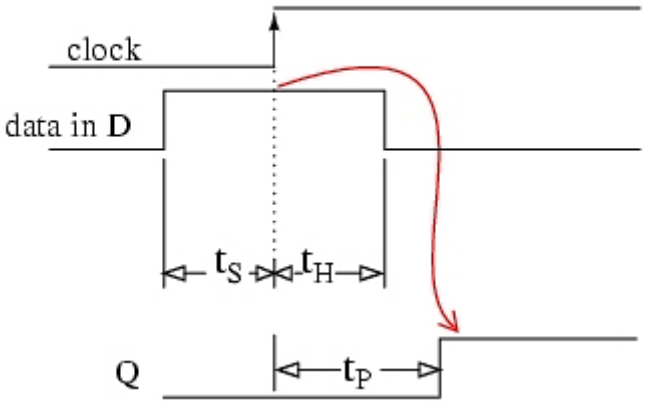
Hvaða stöðu sér vippan þegar klukkupúlsinn kemur, 0 eða 1?



Hvort svarið er rétt Q_C eða Q_W ?
Þegar púlsinn kom við tímann t_1 þá var staðan á D = 0. En við t_2 er hún orðin 1 og vippan skiptir þá úr 0 í 1. Q_C er því rétt svar.

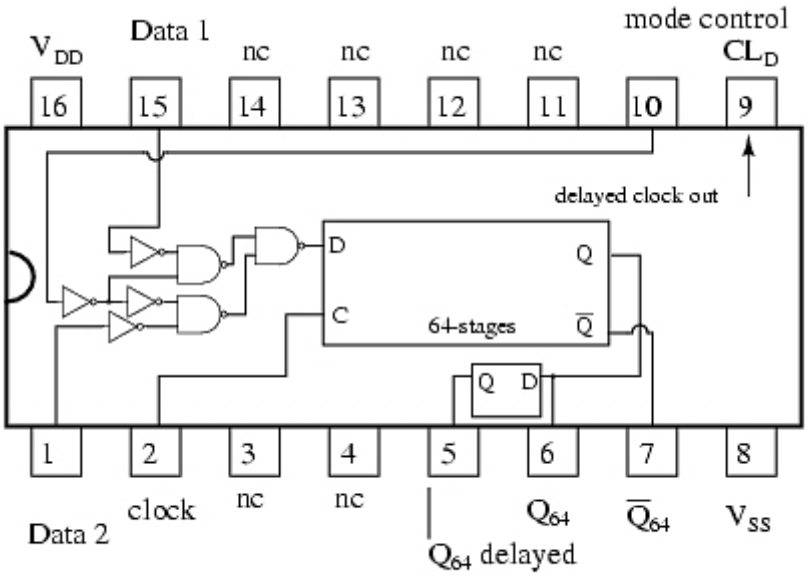
Einfaldasta svarið er að hún sér stöðuna á D innganginum áður en klukkupúlsinn kom. Sú staða er flutt yfir á útganginn við tímann t_1. D vippan seinkar því öllum gögnum um einn klukkupúls en heldur réttu bylgjuformi á gögnunum.





t_s = 100 ns (uppsetningartími, setup time)
t_h = 60 ns (hald tími, hold time)
t_p = 200 – 400 ns (flutnings seinkunn, propagation delay) (typical data)

Gögnin verða að vera klár tímanum t_s áður en klukkupúlsinn skiptir í hástöðu og tímanum t_h eftir að klukkupúlsinn skipti í hástöðu. Seinkunnartíminn frá D til Q er t_p.



CD4031 64-bit serial-in/ serial-out shift register

6.5 Parity

Til þess að tryggja að gögn skili sér á áfangastað í serial samskiptum er sendur einn aukabiti með gögnunum, svokallaður parity biti. Möguleikarnir á parity eru tveir:

1. Sent er með even parity. Þá er heildarfjöldi af **1** sem sendur er slétt (jöfn) tala. Ef við viljum t.d. senda gögnin **1101** með even parity þá verður parity bitinn að vera **1** til þess að gera fjöldann af **1** slétta tölu. Gögnin verða því **1**1101 þar sem fremsti bitinn er þá parity bitinn.
2. Sent er með odd parity. Þá er heildarfjöldi af **1** sem sendur er oddatala. Ef við sendum gögnin **1101** með odd parity þá er parity bitinn **0** því það er oddatölufjöldi af **1** í gögnunum. Gögnin verða því **0**1101.

Á móttökuenda er síðan skoðað hvort gögnin hafi skilað sér rétt. Ef svo er ekki er send beiðni til baka til sendanda og hann beðinn um að senda þau aftur. Gallinn við parity gagnasendingar er sá að ef tveir bitar breytast á leiðinni þá skynjar móttökuendinn það ekki sem villu. Í dag eru notaðar stærðfræðilegar aðferðir til þess að villuleita samskipti.
Hvernig verður þessi parity biti til? Ef við köllum even parity bitann P_e og odd parity bitann P_o þá er ljóst að $P_o = \overline{P_e}$.

Fjöldi af 1	D3	D2	D1	D0	P_e
0	0	0	0	0	**0**
1	0	0	0	1	**1**
1	0	0	1	0	**1**
2	0	0	1	1	**0**
1	0	1	0	0	**1**
2	0	1	0	1	**0**
2	0	1	1	0	**0**
3	0	1	1	1	**1**
1	1	0	0	0	**1**
2	1	0	0	1	**0**
2	1	0	1	0	**0**
3	1	0	1	1	**1**
2	1	1	0	0	**0**
3	1	1	0	1	**1**
3	1	1	1	0	**1**
4	1	1	1	1	**0**

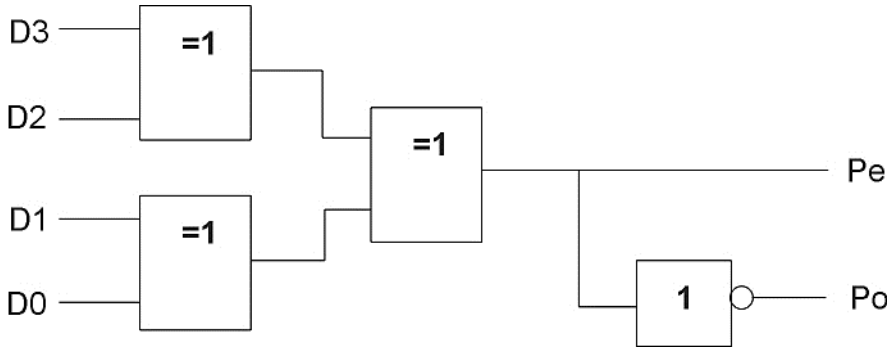
Ef við setjum niðurstöðuna í K kort þá fæst:

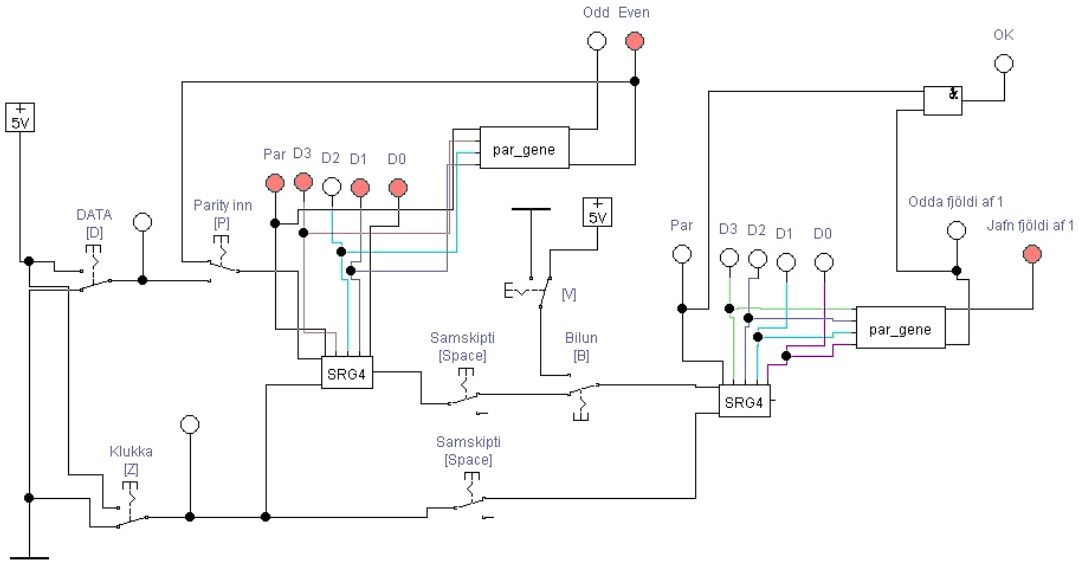
	$\overline{D_1} \cdot \overline{D_0}$	$\overline{D_1} \cdot D_0$	$D_1 \cdot D_0$	$D_1 \cdot \overline{D_0}$
$\overline{D_3} \cdot \overline{D_2}$	0	1	0	1
$\overline{D_3} \cdot D_2$	1	0	1	0
$D_3 \cdot D_2$	0	1	0	1
$D_3 \cdot \overline{D_2}$	1	0	1	0

Getum ekkert lykkjað saman. Útkoman verður: (Eftir mikla Boolean algebru ☺)

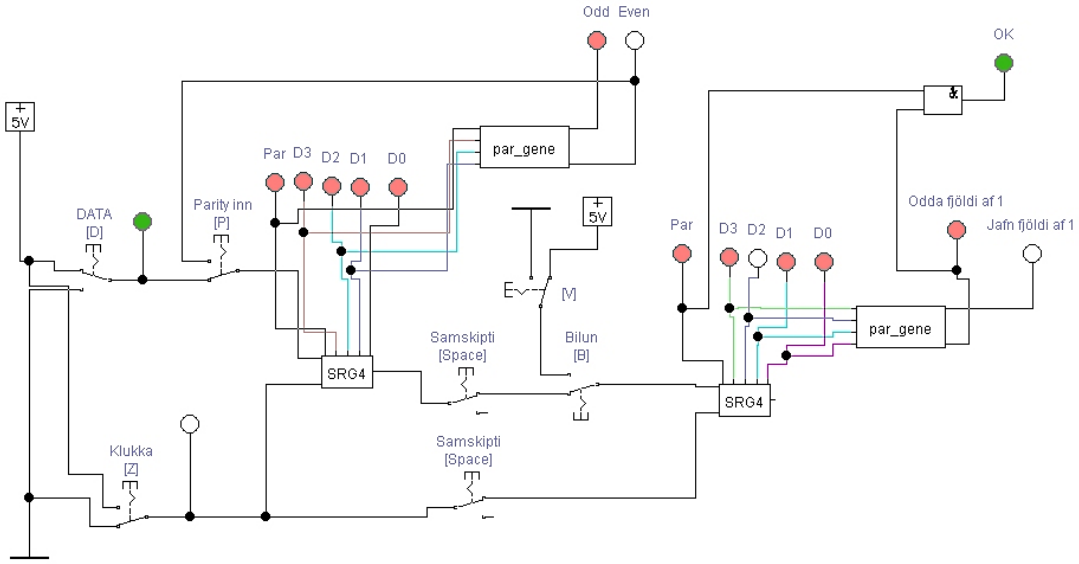
$$P_e = (D_1 \oplus D_0) \oplus (D_3 \oplus D_2)$$

Parity rásin lítur þá svona út (4 bita Parity Generator):

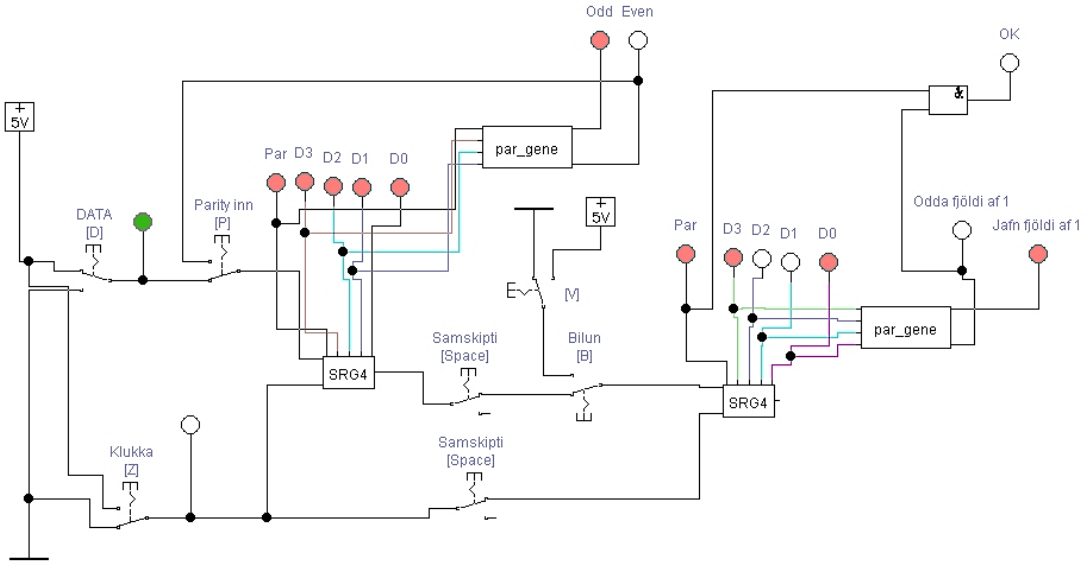




Uppsetning í Workbench. Serial samskipti með even parity. Staðan **1011** *skrifuð inn með even parity bitanum* **1** *og flutt yfir í annað gistur á móttökuenda og parity kannað.*



Odd parity gaumljós á móttökuenda sýnir okkur að talan sem var flutt var odd parity tala og þess vegna var even parity bitinn **1**.



Hér var sett inn bilun þ.a. bita D1 sem átti að vera **1** *var breytt í* **0**. *Þá skilar staðan* **1001** *sér á áfangastað í stað* **1011** *og við fáum ekki grænt OK ljós á móttökuenda*.

7. Minni

Minni má skipta upp í tvo flokka:

A. Hraðvirkt vinnsluminni **RAM**

Hraðvirkt lesminni **ROM**

B. Geymsluminni, stórt og hægvirkt t.d. harðir diskar, geisladiskar, segulbönd….

RAM minni (Random Access Memory) er minni sem hægt er að lesa í og skrifa úr á rauntíma (meðan tölvan vinnur). Þess vegna er vinnsluminni í tölvu alltaf RAM. Stundum er þó búið til sýndarminni (virtual memory) á harða disknum.

Ein minniseining (cell) getur geymt einn bita **0** eða **1**. Ein röð getur geymt eina tölu (word). Þessi tala er yfirleitt 8 bitar (1 bæt), 16 bitar, 32 bitar eða 64 bitar. Alltaf er ein röð (tala) lesin inn eða út í einu.
4 x 4 minni gæti litið svona út:

0	0	1	1	0
1	1	1	1	0
2	0	1	0	1
3	1	1	1	1

Hægt er að skipta um fjóra bita (eina röð) í einu. Minni sem er 1024 x 8 geymir 1024 bæt eða 1KB. Til þess að geta farið inn í hvaða röð sem er þarf 1024 vistföng (addressur) eða 10 bita (línu) vistfangsbraut (address bus) því $2^{10} = 1024$. Í minninu á myndinni að ofan þarf 2 línu vistfangsbraut því við erum með fjögur vistföng og $2^2 = 4$.

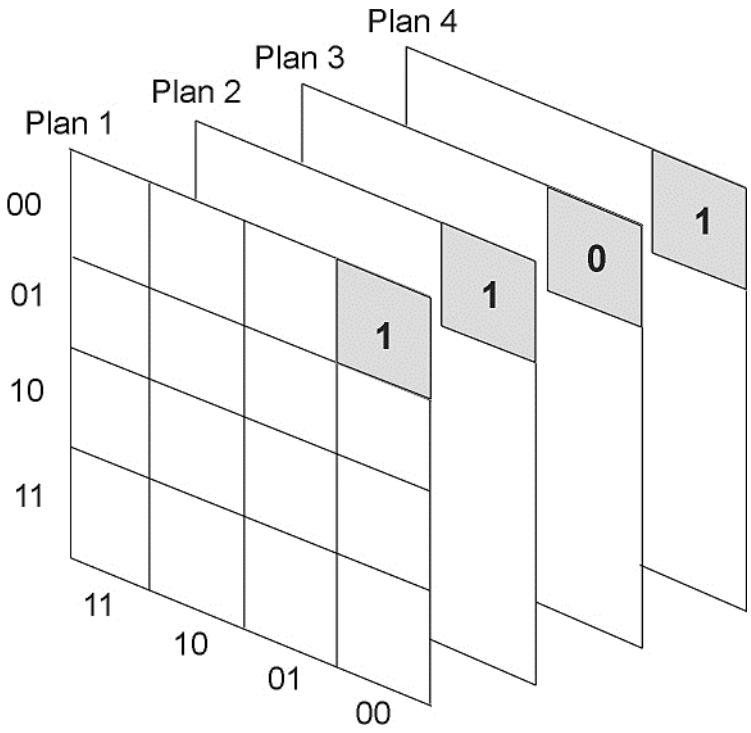
ATH:
1.024 = 1K. 1.024·1.024=1.048.576=1M.
1.024·1.024·1.024=1.073.741.824=1G
Tvíundartölur ganga ekki upp í tugakerfið, því ekki er hægt að skrifa 10 sem 2^n. Þess vegna kemur þessi skekkja fram.

Minnið á myndinni hér að framan er einvítt þ.e. línur. Einnig er hægt að skipuleggja minni í tvívídd þ.e. línur og dálkar og í þrívídd þ.e. línur, dálkar og plön.

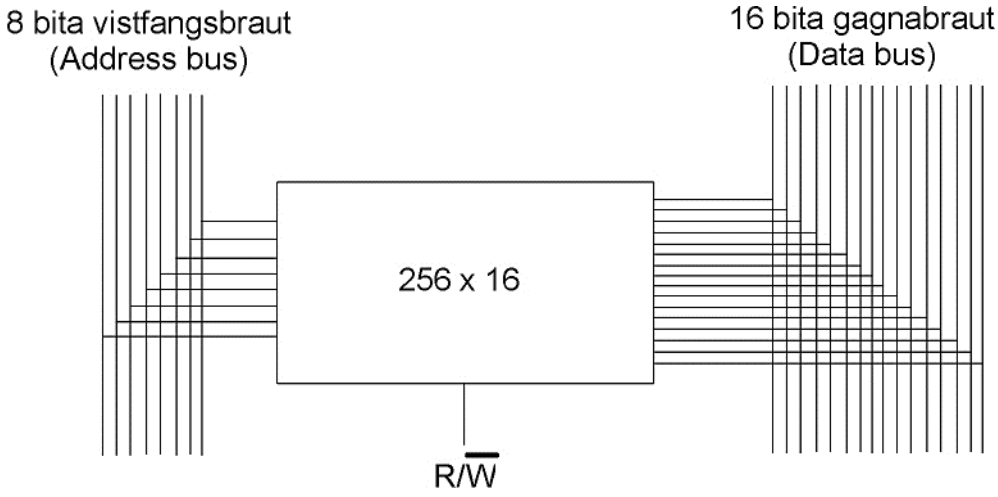
00	3	2	1	0
01	7	6	5	4
10	B	A	9	8
11	F	E	D	C
	11	**10**	**01**	**00**

Til þess að breyta bitanum í hólfi D þá er staðan **1101** sett á vistfangsbrautina. Tveir fyrstu bitarnir eru línan og seinni tveir eru dálkurinn. Þetta minni geymir 16 bita eða 2 bæt.

Ef við viljum að hvert vistfang geymi meira en einn bita þá búum við til plön og förum í þrívídd. Átta bita tala, 1 bæt, er þá geymd í átta plönum.



Í þessu minni er 4 bita talan 1101 geymd í vistfangi 0000 á fjórum plönum. Minnið geymir 16 x 4 bita eða samtals 8 bæt.



Þetta minni geymir 256 (2^8) 16 bita tölur á 16 plönum eða samtals 512 bæt.

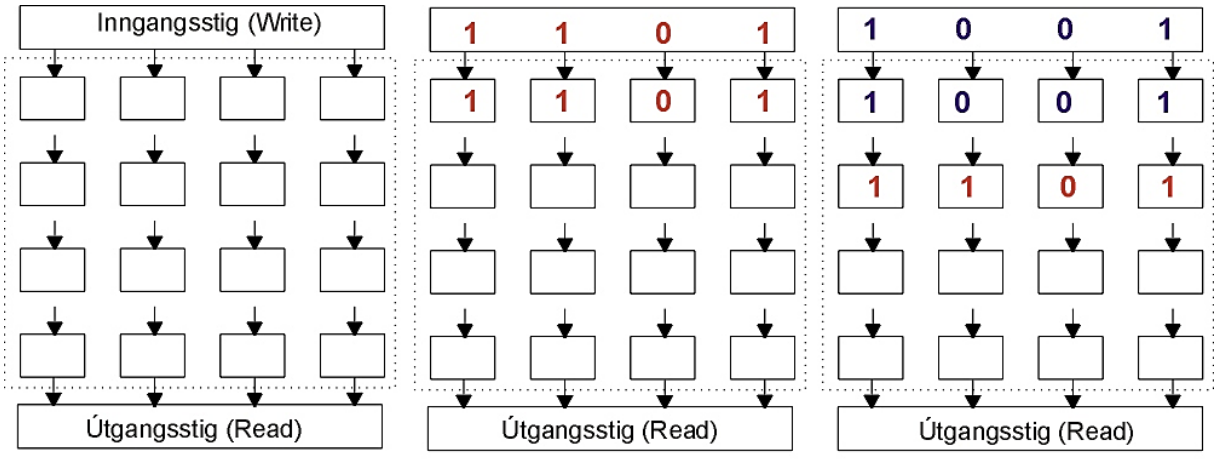
Sóknartími (Access time)

Sóknartími er sá tími sem líður frá því að vistfangið kemur á vistfangsbrautina þar til innihald minnishólfsins er komið á gagnabrautina. Þessi tími er fastur fyrir þrívítt minni, yfirleitt mældur í ns. Þegar við getum farið inn í hvaða vistfang sem er í minninu og lesið út eða skrifað inn á föstum tíma, er það kallað „random accessing“.

7.1 Högun minnis

FIFO (First In First Out)

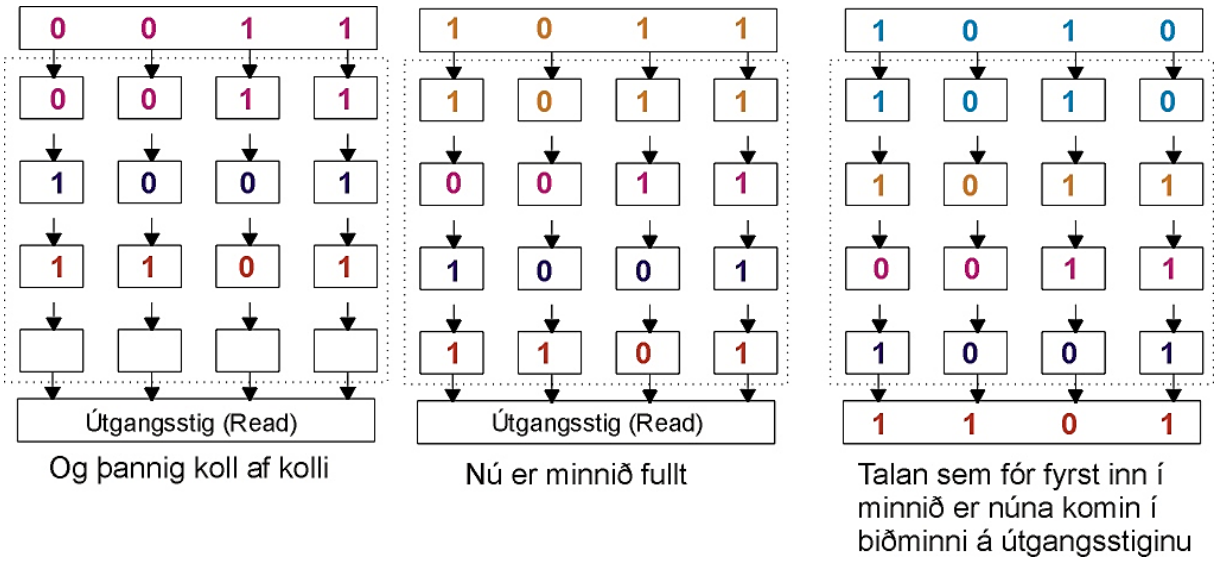
FIFO minni eru dýnamísk þ.e. gögnin eru á hreyfingu innan þess.



4 x 4 dýnamískt FIFO minni

Gögn sem lesin eru inn í tómt FIFO minni eru færð frá toppi til botns

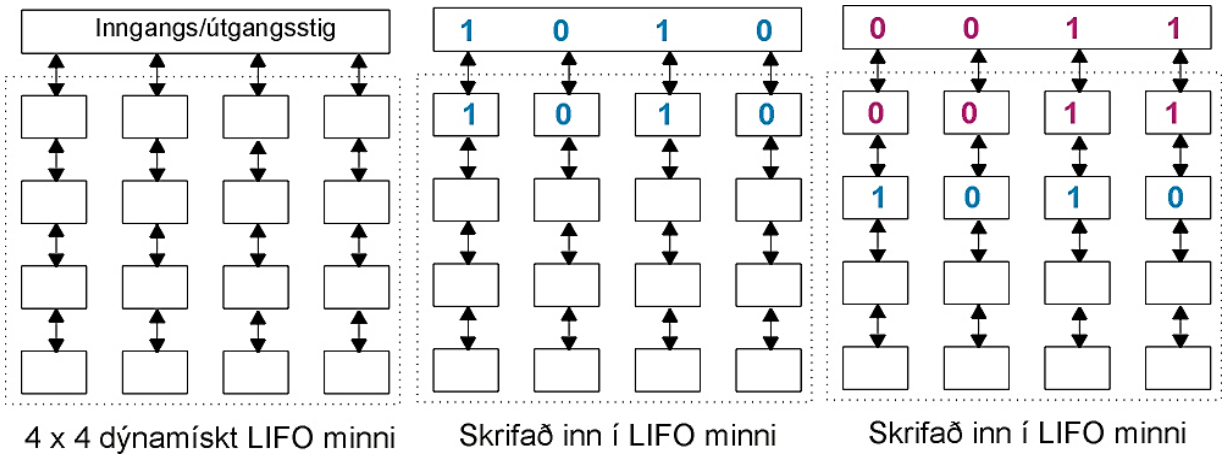
Fyrstu gögnin færast um eitt sæti niður

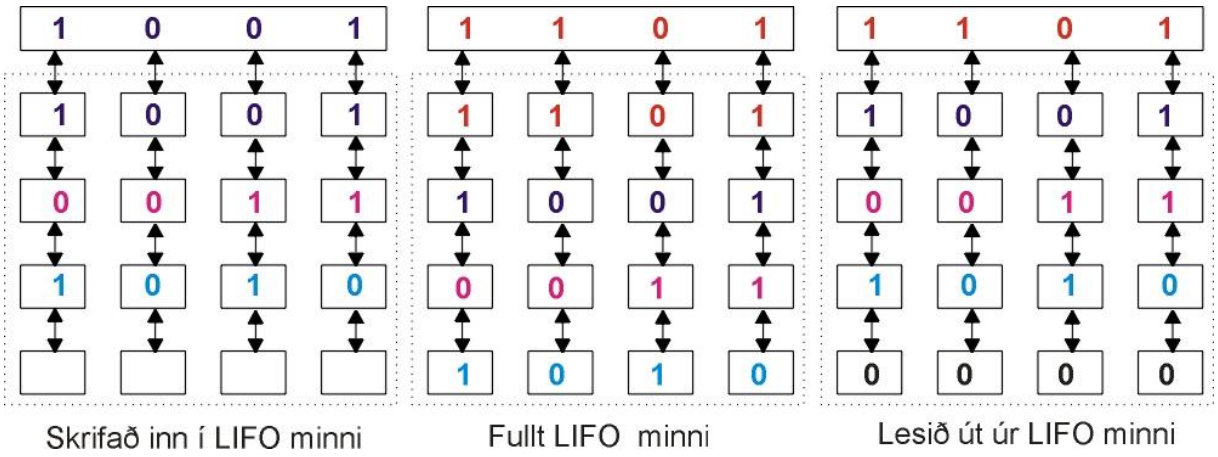


FIFO minni eru byggð upp sem Shift Register. Inngangur þeirra er Samhliða gistur (Parallell register) sem geymir næstu stöðu sem lesa á inn. Útgangurinn er líka Samhliða gistur sem geymir síðustu stöðu sem lesin var út. Ef minnið er fullt og við þurfum að bæta við tölu þá er síðustu tölunni ýtt út. (Push) FIFO minni er t.d. notað sem biðminni (buffer) fyrir prentara.

LIFO (Last In First Out)

LIFO minni er byggt upp sem tvístefnt gistur þ.e. það hefur sameiginlegt inn og útgangsstig. Þess vegna er síðasta tala inn, fyrst út.





Ef LIFO minni yfirfyllist þá týnast neðstu tölurnar (út um botninn, *Bit Bucket*). Því verður að fylgjast vel með hve mikið er komið í minnið. (Vandamál forritara).

R/W minni

R/W minni er samheiti yfir minni sem hægt er að skrifa í og lesa úr á meðan forrit keyrir. Undir þetta flokkast bæði FIFO og LIFO. En R/W minni eru samt ekki RAM minni í strangasta skilningi því í RAM nafninu er innifalin „access“ tæknin þ.a. sóknartíminn er fastur fyrir gögn hvar sem þau eru stödd í minninu. Það gildir ekki fyrir FIFO og LIFO. Þetta eru samt kölluð RAM minni þótt það sé ekki „rétt“.

Óstöðugleiki minnis (Memory volatility)

Minni sem tapa innihaldi sínu við rafmagnsleysi eru kölluð hvikul (volatile). Minni sem halda innihaldi sínu við rafmagnsleysi kallast óhvikul (non volatile). Hægt er að gera RAM minni óhvikul með því að nota rafhlöðu. Það er t.d. gert á móðurborði tölva til þess að viðhalda innihaldi CMOS rásarinnar sem geymir BIOS tölvunnar.

Minni sem heldur innihaldi sínu svo lengi sem rafmagn er til staðar kallast stöðugt (static). Þetta eru svokölluð SRAM minni (Static RAM).

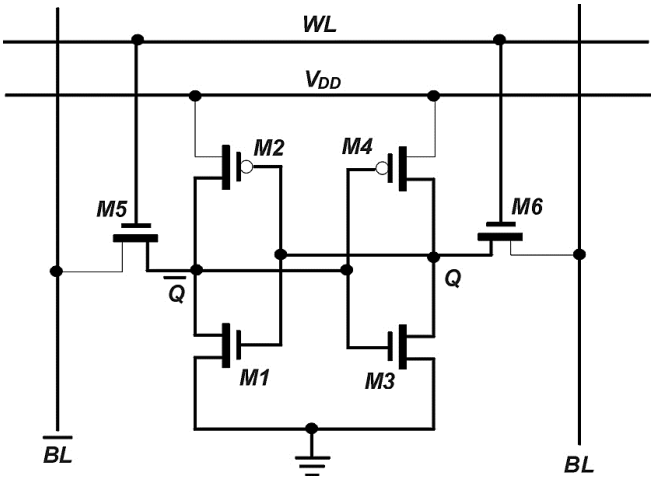
Til eru RAM minni sem eru ekki stöðug þótt þau hafi rafmagn. Þetta eru svokölluð kvik minni (Dynamic RAM eða DRAM). Þau eru hraðvirk og ódýr en gallinn við þau er að þau þurfa að fá „hressingu“ á c.a. 3 ms fresti til þess að viðhalda innihaldi sínu. Hver minniseining er örsmár þéttir sem heldur hleðslu sinni í örstutta stund.

7.2 SRAM (Static Random Access Memory)

Static random access memory (SRAM) er hálfleiðaraminni þar sem orðið „static“ (stöðugt) vísar til þess að ekki þarf að viðhalda (refresh) innihaldi þess reglulega eins og í DRAM minni. SRAM notar vippur (bistable latching) til þess að geyma hvern bita. SRAM minni eru hvikul (volatile) sem þýðir að þau glata innihaldi sínu við straumleysi.

Hugtakið SDRAM sem stendur fyrir „synchronous DRAM“ hefur ekkert með SRAM að gera en er oft ruglað saman.

Hönnun



Sex-transistora CMOS SRAM minnis-sella.

Hver biti í SRAM er geymdur í fjórum transistorum (M1 – M4) sem mynda tvo kross tengda *invertera* (Q og $\bar{Q}$). Þessi geymslueining hefur tvær stöður sem við köllum **0** og **1**. Síðan eru tveir auka transistorar (M5 og M6) sem eru notaðir við innskrift og útlestur. SDRAM er til með fleiri transistorum sem gera þá kleift að vinna með fleiri inn og útganga í einu (Multi Access). Til þess að ná sambandi við minnisröðina (orðið) er WL (Word line) inngangurinn notaður. Hann stýrir transistorunum M5 og M6 sem stýra því að einingin tengist við BL og $\overline{BL}$ línuna (Bit Line). Hún er notuð til þess að flytja gögn bæði í skrift og lestri. Þó það sé í rauninni óþarfi að nota $\overline{BL}$ línuna þá er það yfirleitt gert til þess að minnka suð með því að nota balanseruð samskipti.

Þetta eykur bandvídd samskiptanna miðað við DRAM því balanseruð samskipti eru suðlaus. Einnig gerir tilkoma transistoranna kleift að dæla út meiri straum en í DRAM minni.

Stærð SRAM minnis með *m* vistföngum (addressum) og *n* gagnalínum er 2^m orð (word) eða $2^m \cdot n$ bitar.

> ATH: Word þýðir nokkrir bitar. 8 bita word er sama og 1 bæt.
> Algengar stærðir eru 8 bita orð, 16 bita orð, 32 bita orð og 64 bita orð.

SRAM með 16 vistfangalínum og 32 gagnlínum (32 línu gagnabraut) geymir því 2^{16} = 65.356 orð.
Í bitafjölda er þetta 65.356 · 32 = 2.097.152 bitar, (2 Mb eða 262 kB).

SRAM virkni

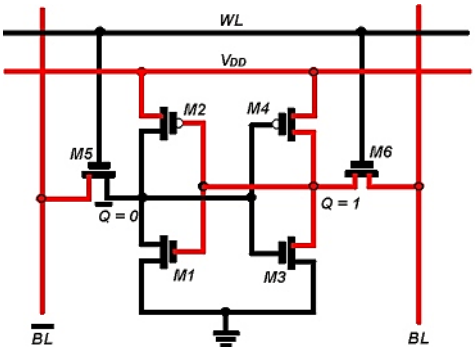
SRAM minniseining hefur þrjár stöður: Biðstaða (standby), útlestur (reading) og innlestur (writing).

Biðstaða (Standby)

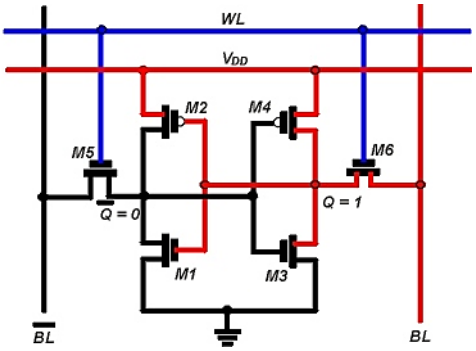
Ef word lína er spennulaus þá aftengja M5 og M6 einninguna frá gagnabrautinni. M1 til M4 halda innihaldi sínu á meðan spenna er á rásinni.

Útlestur

Gerum ráð fyrir að innihald minnissellu sé **1** og geymt á *Q* útgangi. Útlestrarferlið hefst á því að báðar *BL* línurnar eru settar í hástöðu **1**. Síðan er *WL* línan gerð virk þ.a. bæði M5 og M6 fara í ON stöðu. Næst er gildið **1** (*Q* og $\bar{Q}$) flutt yfir á *BL* línurnar með því að setja *BL* í hástöðu og $\overline{BL}$ í lágstöðu. Það er gert með því að afhlaða $\overline{BL}$ til jarðar með M1 og M5 og tengja *BL* með M4 og M6 við V_{DD} (lógískur **1**).
Ef innihald einingarinnar hefði verið **0** þá gerist akkurat öfugt þ.e. *BL* er afhlaðin en $\overline{BL}$ sett í hástöðu.



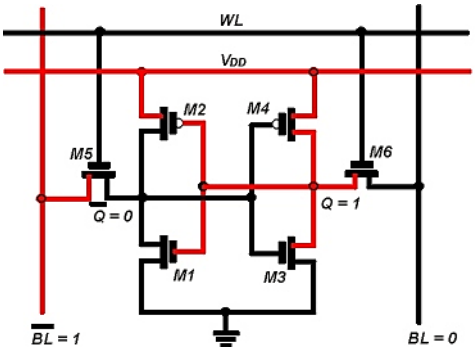
1. $WL = 0, BL = \overline{BL} = 1$



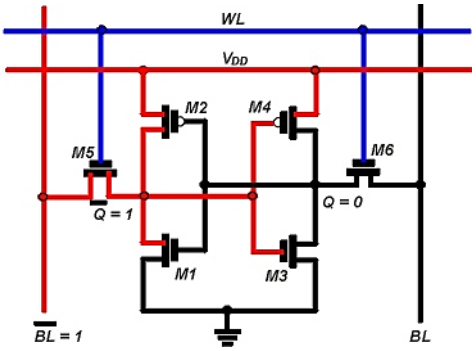
2. $WL = 1, \overline{BL} = 0, BL = 1$

Innskrift

Innskriftarferlið hefst á því að bitarnir sem skrifa á inn eru settir á BL línuna. Ef við viljum skrifa inn **0** þá setjum við **0** á BL línuna og **1** á $\overline{BL}$ línuna. Síðan er WR línan gerð virk og bitarnir skrifaðir inn. Ef við ætlum að skrifa inn **1** þá setjum við **1** á BL og **0** á $\overline{BL}$. Inngangstransistorarnir (M5 og M6) eru miklu sterkari en geymslutransistorarnir (M1 – M4) og geta því auðveldlega þvingað inn nýja stöðu.



1. $BL = 0, \overline{BL} = 1$



2. $WL = 1$ *og stöðurnar skrifaðar inn.*

Hegðun brautar

RAM minni með sóknartíma (access time) 70 ns skilar af sér gögnum í seinasta lagi 70 ns eftir að vistfangsbrautin (address bus) varð virk. Þau eru líka aðgengileg í smástund 5 – 10 ns (hold time) eftir að þau voru lesin út eða skrifuð inn.

SRAM er miklu dýrara, hraðvirkara og eyðir minna afli en DRAM. Það er því mest notað þegar bandvídd er léleg eða lítið afl til staðar. T.d. er flýtiminni (cache) örgjörva alltaf SRAM. Auðveldara er að stýra SRAM minni en DRAM og SRAM er meira „random access“ minni en DRAM. DRAM minnið er hinsvegar miklu ódýrara og því mest notað sem vinnsluminni í tölvum í dag.

7.3 ROM minni (Read Only Memory)

Til eru fjórar gerðir af ROM minni.

1. Mask ROM (MROM)
2. Bræðitengja ROM (PROM fuse link)
3. ROM hreinsanlegt með útfjólubláu ljósi (EPROM)
4. ROM breytanlegt með rafmagni (EEPROM eða E^2PROM)

1. MASK ROM er forritað með filmumaska og er forritað í framleiðslu og er óbreytanlegt. Getur tekið vikur og er í raun úrelt tækni.
2. PROM er hægt að forrita með forritunartæki, svokölluðum PROM programmer. PROM kemur frá verksmiðjunni með 0 í öllum minniseiningum. Til þess að skrifa 1 í minniseiningu er settur straumur á réttan stað í rásinni til þess að bræða tengingu. Þessi aðgerð er endanleg og rásin ónýt ef það mistekst.
3. EPROM er minni sem hægt er að hreinsa innihaldið með útfjólubláu ljósi og endurforrita. EPROM er forritað með EPROM programmer þar sem forritið er flutt frá PC tölvu yfir í forritunartækið. EPROM minni má auðveldlega þekkja á glugganum ofan á því. Þegar búið er að forrita verður að hylja þennan glugga með ógagnsæju efni t.d. límmiða.
4. EEPROM er ROM minni sem hægt er að breyta á meðan unnið er í tölvu. Þetta er þó of hægvirkt til þess að geta kallast RAM. EEPROM er byggt upp sem tvö lög af minni. Efra lagið er venjulegt R/W minni en neðra lagið hægvirkt EEPROM. Ef örgjörvinn skynjar rafmagnsvandræði getur hann skipað svo fyrir að efra lagið sé afritað í neðra lagið og geymt þar sem afrit. Hægt er að afrita í báðar áttir. BIOS er geymdur í EEPROM minni. EEPROM gengur yfirleitt undir nafninu Flash minni.

Lestur í og úr minni.

Til þess að lesa í og úr minni þarf þrennt að gerast.

1. Finna þarf vistfang minnishólfsins og koma því á vistfangsbrautina.
2. Ákveða þarf hvort lesa á út eða skrifa inn.
3. Stýra þarf nákvæmlega hvenær þetta á að gerast.

Örgjörvinn stýrir þessum atburðum. Hann sendir fyrst vistfangið á vistfangsbrautina. Minnið inniheldur vistfangskóðara (address decoder) sem sér um aðgang að réttum minniseiningum. Stuttu síðar sendir örgjörvinn annaðhvort Read eða Write skipun og lesið er út á gagnabrautina eða skrifað inn af gagnabrautinni. Örgjörvinn sér síðan um að koma upplýsingunum af gagnabrautinni á rétta staði. Hann getur t.d. sent þær út á prentara, skjá eða tekið þá inni í reikningseininguna (ALU) og framkvæmt einhverja útreikninga á þeim.

Notkun minnis

Í tölvum eru fimm gerðir af minni

1. Flýtiminni (Scratch pad memory)
2. Biðminni (Buffer)
3. Staflaminni (Stack)
4. Raðminni (Queue)
5. Uppflettiminni (Look up table)

1. Flýtiminni (Scratch pad) er lítið minni sem notað er til þess að geyma gögn í stuttan tíma t.d. í reiknivélum. Öll vinnsla í reiknivélum fer fram í flýtiminni en hægt er að geyma niðurstöðurnar í öðru minni. Í PC tölvum er þetta minni yfirleitt byggt inn í örgjörvann sem flýtiminni (Cache). Reiknieiningin (ALU) notar flýtiminni til þess að geyma þær tölur sem hún er að nota í útreikningum. Flýtiminnið, eins og nafnið bendir til, er hraðvirkara en venjulegt vinnsluminni. Það er líka miklu dýrara og því framleitt sem frekar lítið eða 512k til 1M. Vegna þess að flýtiminnið er byggt í örgjörvann þá teppir það ekki neinar brautir svo tölvan getur unnið í flýtiminninu samhliða því sem hún framkvæmir eitthvað annað á móðurborðinu.
2. Biðminni (Buffer) virkar sem stuðpúði á milli hraðvirks örgjörva og hægvirks jaðartækis t.d. prentara. Biðminni er nær undantekningalaust FIFO. Þannig getur örgjörvinn fyllt öðruhvoru á biðminnið og haldið síðan áfram við annað. Prentarinn mjatlar síðan úr biðminninu á sínum hraða en teppir ekki vinnu örgjörvans.

3. Staflaminni (Stack) er frátekinn hluti af vinnsluminni tölvunnar. Staflinn er notaður til þess að henda í öllum gögnum sem hann er að vinna með ef hann fær beiðni (rof) um að gera eitthvað annað strax. (Interrupt request, IRQ). Þetta er nær undantekningarlaust LIFO minni því það hentar vel að síðasta skipun sem fer inn sé sú sem þarf að framkvæma fyrst eftir að rofið er búið. Í staflann fer líka afrit af innihaldinu úr flýtiminninu svo það glatist ekki. Til þess að fylgjast með stærð staflans er notaður svokallaður staflabendir (Stack pointer). Til þess að fylgjast með vistföngum vinnsluminnisins er notaður svokallaður forritsteljari (Program counter). Ef vistfangsbrautin er 32 bitar þá þurfa staflabendirinn og forritsteljarinn báðir að vera 32 bita.

   Skoðum hvað gerist þegar örgjörvinn fær skipun um að hætta í forriti til þess að sinna jaðartæki. Gerum ráð fyrir að staflinn sé hluti af vinnsluminninu, en settur upp eins og LIFO. Segjum að forritsteljarinn sé staddur í vistfangi 0500_H og staflabendirinn bendi á $FFFF_H$ (16 bita vistfangsbraut). Örgjörvinn er þá að ná í skipun úr minnishólfi 0500_H (1280_{10}) og staflabendirinn bendir á næsta lausa minnishólf sem er efst á staflanum. Örgjörvinn klárar að framkvæma skipunina úr 0500_H og hækkar síðan forritsteljarann um 1 (0501_H) og flytur það gildi í staflann. Síðan flytur hann allt úr flýtiminninu í staflann. Þegar hann er búinn að afgreiða rofið þá flytur hann innihaldið úr staflanum í flýtiminnið og síðan gildi næsta vistfangs (0501_H) inn í forritsteljarann og getur þá haldið áfram að framkvæma næstu skipun í forritinu sem hann var að vinna í fyrir rof.

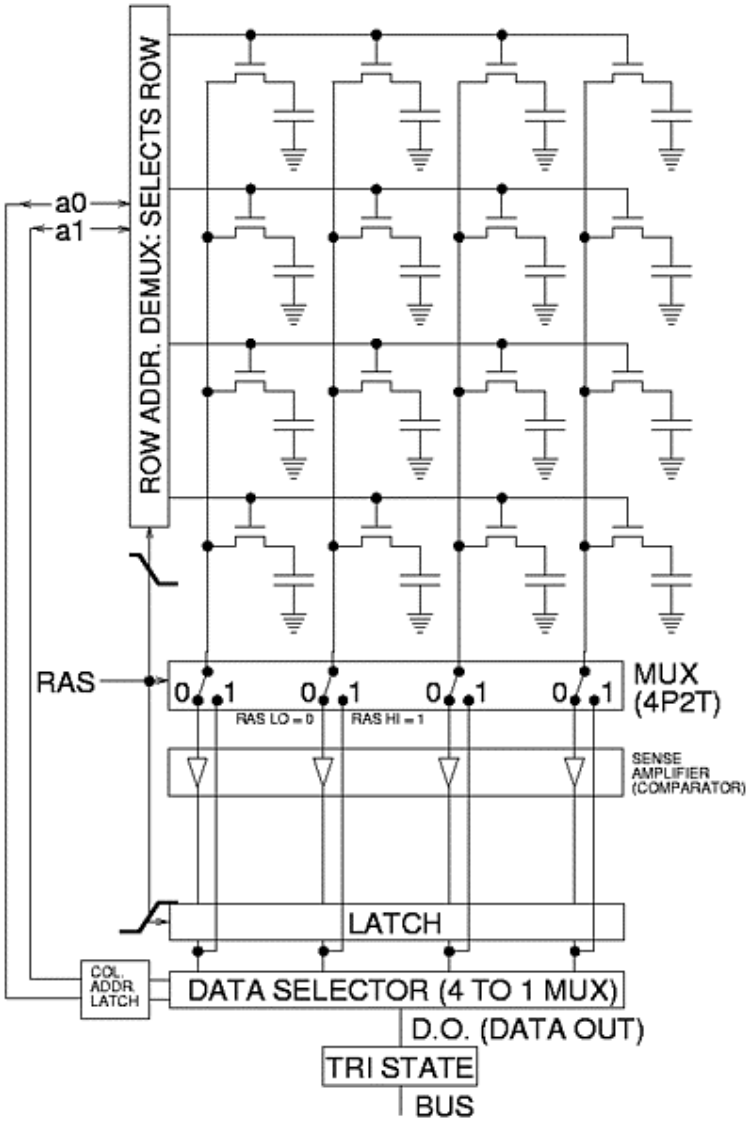
4. Raðminni (Queue). Örgjörvinn færir allar skipanir sem framkvæma á inn í biðminni þaðan sem þær eru lesnar ein og ein inn í skipanagistrið (Instruction Register, IR). Hann fyllir á minnið eftir þörfum og getur notað lausan tíma til þess að framkvæma skipanir innan örgjörvann sem ekki útheimta notkun á vistfanga eða gagnabrautinni.

5. Uppflettiminni (Look Up Table Memory) er minni sem hefur fast innihald og við getum litið á sem uppflettitöflu. Ef við myndum t.d. brenna sannleikstöflu einhverrar rökrásar með 4 inngöngum þá myndi hún geyma 2^4 uppflettilínur þar sem hver lína geymdi stöðu útganganna fyrir þá línu. Sem dæmi um svona væri t.d. seven segment decoder rásin sem hefði þá 4 innganga og 7 útganga. Inngangurinn **0000** gæfi þá okkur útganginn **1111110** út sem myndar töluna **0** á display.

7.4 DRAM (Dynamic random access memory)

DRAM geymir hvern bita í litlum þétti. Vegna þess að þéttar hafa tilhneigingu til að missa hleðslu (leka) þá þarf reglulega að hressa upp á hleðsluna. Það fer þannig fram að sömu gögnin eru skrifuð inn aftur reglulega án breytinga. Dynamic hlutinn af nafninu vísar til þessa, minnið er stöðugt endurskrifað aftur og aftur.

Kostur DRAM er einfaldleikinn í uppbyggingunni Aðeins þarf einn transistor og einn þétti til að geyma einn bita í stað fjögurra transistora í SRAM. Þetta þýðir að DRAM minni geta orðið mjög stór og eru reynd mest notuðu vinnsluminnin í dag.



Uppbygging DRAM minnis

Virkni

DRAM er venjulega sett upp sem ferningslaga fylki (tafla) með einn transistor og einn þétti í hverri minniseiningu. Á myndinni hér að framan er dæmi um 4 x 4 fylki (16 bitar) en í raunveruleikanum erum við með kannski þúsundir eininga í röðum og dálkum.

Langa línan sem tengir einingarnar í hverri röð kallast WL (Word Line). Hver dálkur hefur venjulega tvær bitalínur (BL og $\overline{BL}$) en til einföldunar er bara BL sýnd. Þær eru venjulega kallaðar + og – bitalínan. Þetta er gert til þess að halda minninu stöðugu með jákvæðri afturverkun (positive feedback)
Venjulega gera framleiðendur ráð fyrir að minnið sé endurskoðað með að minnsta kosti 64 ms millibili. Þetta gerir rásina flóknari (útheimtir Refresh logic) en er samt miklu ódýrara en að nota SRAM. Sumar DRAM tegundir eru hresstar við á 64 ms fresti en aðrar taka eina röð í einu. Sem dæmi má taka minni með 2^{13} = 8192 raðir. Þá er hver röð hresst við á 7,8 µs fresti (64 ms / 8192).

Tímar

	PC-3200 (DDR-400)				PC2-6400 (DDR2-800)				PC3-12800 (DDR3-1600)			
	Typical		Fast		Typical		Fast		Typical		Fast	
	cycles	time	cycles	time	cycles	time	cycles	time	cycles	time	cycles	time
t_{CL}	3	15ns	2	10ns	5	12.5ns	4	10ns	9	11.25ns	8	10ns
t_{RCD}	4	20ns	2	10ns	5	12.5ns	4	10ns	9	11.25ns	8	10ns
t_{RP}	4	20ns	2	10ns	5	12.5ns	4	10ns	9	11.25ns	8	10ns
t_{RAS}	8	40ns	5	25ns	16	40ns	12	30ns	27	33.75ns	24	30ns

Þessi tafla spannar yfir 11 ára tímabil og framfarirnar eru ekki dramatískar. Þó má taka fram að DDR3 tæknin hefur 32 sinnum hærri bandvídd og getur sent út tvö orð með 1,25 ns millibili (1600 Mword/s) á meðan EDO RAM getur skilað af sér einu orði á 20 ns (50 Mword(s).

Villur og villuleiðréttingar

Raf og segultruflanir innan tölvu geta valdið því að einn biti í DRAM minni skiptir úr sinni réttu stöðu í andhverfa stöðu. Þetta er vegna þess að DRAM minnin eru að minnka að ummáli, meiru er troðið í þau af gögnum og vinnsluspennan lækkuð. Það þýðir að lágorku agnir geta breytt innihaldi einnar eða fleiri minniseininga.

Þetta vandamál má leysa með því að nota t.d. parity bita og/eða villuleiðréttingu (Error Correcting Code, **ECC**). Gallinn við parity er að það skynjar bara breytingu á einum (eða oddatölufjölda) af bitum. Algengasta villuleiðréttingin er að nota Hamming code. Hann getur leiðrétt einn rangan bita og ef notaður með parity getur hann fundið út hvort bilun er í tveimur bitum í sama orði.

Ekki styðja öll móðurborð villuleiðréttingu. Hún er fyrst og fremst notuð í dýrum mikilvægum tölvum eins og þjónum. Sum móðurborð hafa þá tækni að BIOSinn í þeim lærir að þekkja veilur í minni og forðast að nota þær raðir.

Ástæðurnar fyrir því að nota ekki villuleiðréttingu (ECC) eru:

- Auka parity biti í hverju orði eykur kostnað um 9/8. Í harðri verðsamkeppni er það mikið.
- Móðurborð sem styðja ECC eru dýrari.
- ECC hefur þau áhrif að erfiðarara er að yfirklukka örgjörvann.
- ECC minnisrás þarfnast aukatíma til þess að útfæra villuleiðréttinguna sem getur þýtt töf uppá 0,5% til 2%.

Nýlegar rannsóknir sýna að villur í minni eru frá: einn biti á klst per GB af minni til einn biti á öld per GB af minni. Í öllum tölvum sem notaðar eru í vísindalegum og fjármálalegum tilgangi er það regla að nota ECC frekar en undantekning.

DRAM pakkningar



IP 16 pinna



SIPP 30 pinna



SIMM 30 pinna



SIMM 72 pinna



SDRAM 168 pinna DIMM



DDR1 184 pinna DIMM



EDO RAM 72 pinna (SIMM)



RAMBUS 168 pinna (RIMM)



DDR2 240 pinna DIMM



DDR3 240 pinna DIMM

DIP = Dual Inline Package
SIMM = Single Inline Memory Module

SIPP = Single Inline Pin Package
DIMM = Dual Inline Memory Module

Ósamfasa DRAM (Asynchronous DRAM)

Þetta er grundvallargerðin sem allar gerðir DRAM eru hannaðar útfrá. Hún hefur yfirleitt 12 vistfangslínur og 4 tvístefndar gagnalínur. Einnig hefur hún fjóra active low innganga:

- RAS, kveikir á valinni röð. Röð er valin svo lengi sem RAS inngangurinn er lágur.
- CAS, kveikir á völdum dálki. Dálkur er valinn svo lengi sem CAS inngangurinn er lágur.
- WE, (Write Enable) rásin er tilbúin til að skrifa inn gögn og lesa út gögn svo lengi sem WE er lágur.
- OE, (Output Enable). Aukainngangur sem stýrir aðgangi að I/O pinnunum.

EDO RAM Extended Data Out (EDO) DRAM



EDO DRAM minnisrásir

EDO DRAM er hannað þannig að hægt er að byrja að sækja ný gögn eða skrifa ný gögn inn í það um leið og fyrri gögn eru geymd í I/O einingu.

Double Data Rate (DDR) SDRAM

DDR minni kom á markað í byrjun ársins 2000. Double hlutinn á nafninu þýðir að það vinnur bæði á forkanti og bakkanti klukkupúlsins.

DDR eða DDR1 (2000 – 2002)

Miðað við SDR SDRAM (single data rate) er DDR miklu hraðvirkara bæði vegna betri gagnastýringar og líka að það vinnur á báðum köntum klukkupúlsins. Miðað við sömu klukkutíðni er það því c.a. tvisvar sinnum hraðvirkara. Hámarks flutningsgeta DDR1 er 1600MB/s.

DDR1 minni keyrir á 2,5V miðað við 3,3V á SDRAM. Hækkun á þessari spennu getur aukið hraðann en á kostnað hærri orkunotkunar og jafnvel hættunnar á skemmdum.
Venjulegt DDR1 minni hefur 8 minnisrásir (8 x 8 = 64 bitar). Ef ECC er bætt við bætist níunda minnisrásin við.

DDR2 (2003 - 2004)

Ágóði fram yfir DDR1

Hámarksflutningshraði 3200 MB/s á móti 1600 MB/s. DDR2 keyrir á 1,8 – 2,5V miðað við 2,5V hjá DDR1. DDR2 leyfir hærri brautarhraða með því að lækka innri klukkuhraða niður í helming brautarhraða. DDR2 hefur tvisvar sinnum stærra biðminni (prefetch que) en DDR1.

DDR2 er hvorki samhæft við DDR1 eða DDR3.

DDR3 (2007 - 2008)

Ágóði fram yfir DDR2

Gagnaflutningur er tvisvar sinnum hraðari en fyrir DDR2. Hámarksflutninghraði DDR3 64 bita minnis er 6400 MB/s. DDR3 notar 30% minna afl en DDR2. DDR3 keyrir á 1,35V til 1,5V á móti 1,8 til 2,5V hjá DDR2. Minniskubbarnir eru jafnlangir og hafa jafnmarga pinna. Sökklarnir eru þó ekki eins. DDR3 hefur tvisvar sinnum stærra biðminni (prefetch que) en DDR2 sem flýtir fyrir inn og útlestri.

GDDR3 er stundum ruglað við DDR3 en er sérstök útgáfa af DDR2 fyrir myndvinnslu.

DDR3 er hvorki samhæft við DDR2 eða DDR4

DDR4 (2013 - 2014)



DDR4 288 pinna DIMM. Sama lengd og DDR3 240 pinna DIMM.

Ágóði fram yfir DDR3

Þéttari pökkun á minniseiningum og þannig er hægt að búa til stærra minni, jafnvel tvisvar sinnum stærra. DDR4 notar minna afl en DDR3 og er hraðvirkara. DDR4 keyrir á 1,2V og býður upp á hraða frá 1600MHz til 3200MHz. Low voltage DDR4 keyrir á 1,05V.

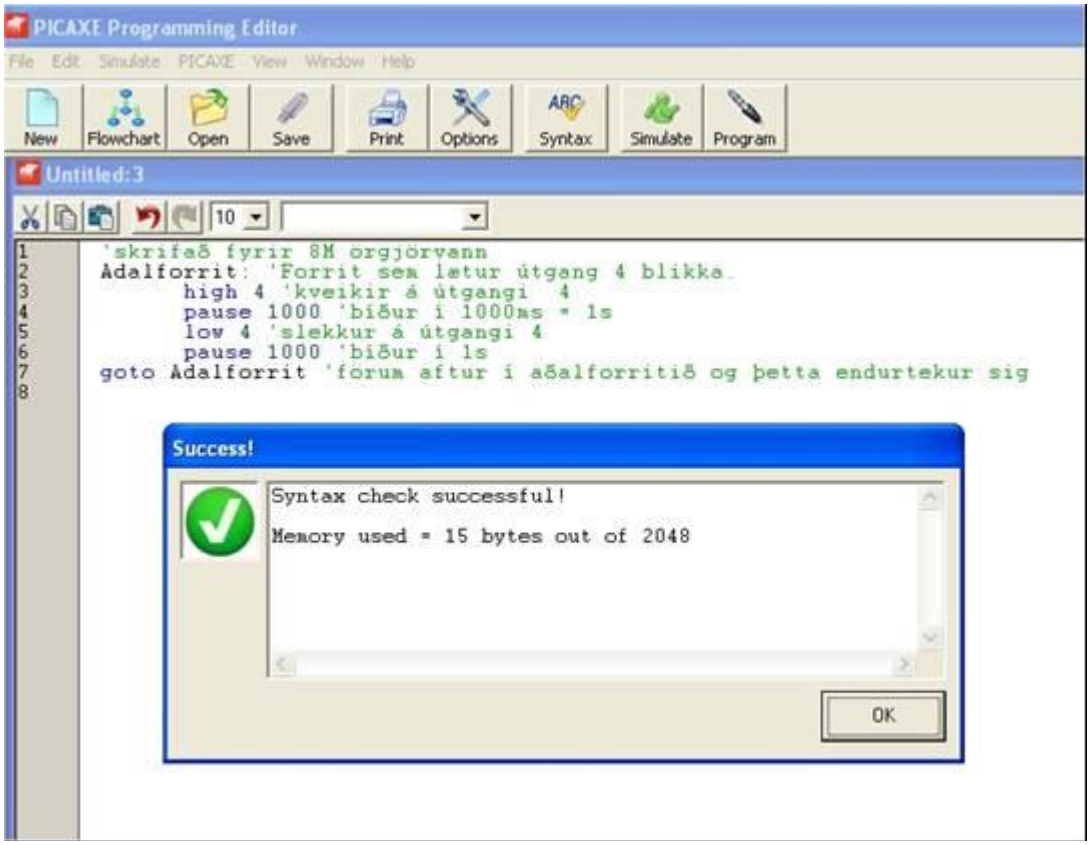
Netheimar halda því þó fram að DDR4 hafi ekkert dramatískt forskot á DDR3. Það muni aðallega henta fyrir stóra tölvuþjóna vegna aflsparnaðar. Byltingin úr DDR2 í DDR3 hafi verið miklu stærri.

8. Forritun stýriörgjörva

Picaxe

Picaxe eru stýriörgjörvar sem mjög auðvelt er að forrita. Þeir eru góðir sem fyrsta val í stýriörgjörvum og forritin eru mjög gegnsæ. Fyrir lengra komna er hægt að fara í Arduino örgjörvann eða Raspberry PI en það er flóknara forritunarumhverfi. Picaxe örgjörvinn er forritaður með forriti sem heitir Picaxe Progamming Editor og hægt er að niðurhala því ókeypis. Forritið er notað til þess að tengjast örgjörvanum en einnig er hægt að herma forritin í því og athuga hvort þau virka. Hægt er að kaupa mikið af tilbúnum verkfærum frá Picaxe til að leika sér með, en einnig er hægt að útbúa eigin serial upphals rás og nota örgjörvann á prentplötur.

Sem dæmi um lítil forrit eru t.d.



Þetta forrit lætur útgang nr.4 á Picaxe 8M2 örgjörvanum blikka með tíðninni 1Hz.

Fyrst veljum við Syntax til að skoða hvort forritið sé villulaust. Græni textinn eru athugasemdir og er ekki hluti af forritinu heldur hjálpartæki fyrir forritarann. Athugasemdir eru gerðar með því að slá inn ‘ á undan (‘ er undir spurningarmerkinu). Þá breytist athugasemdatextinn í grænan.

Ef forritið er villulaust birtist Success! gluggi.
Næst hermum við forritið. Við smellum á hnappinn Simulate og þá birtist þessi mynd. Græna ljósið á útgangi 4 blikkar með tíðninni 1Hz eins og til stóð.

